INSTRUCTOR KIT · MYSQL FOUNDATIONS · COMPLETE ANSWER KEY

MySQL Foundations

Every self-check item answered and explained, plus reference notes for each lab, for all 14 sessions. Use for grading and reteaching. Confidential instructor material.

14 sessions | 168 self-check items + lab solutions | Confidential · do not distribute

CONTENTS

PART 1: SQL FOUNDATIONS · SESSIONS 1 TO 7

S01 **What Databases Are and Installing MySQL**

- 1.1 What Is a Database?
- 1.2 Installing MySQL

S02 **The MySQL Client and Your First Tables**

- 1.3 MySQL CLI & phpMyAdmin
- 2.1 Creating Databases & Tables

S03 **Data Types, Constraints and CRUD**

- 2.2 Data Types & Constraints
- 2.3 CRUD Operations

S04 **Filtering, Sorting and Limiting**

- 3.1 WHERE Clauses, Operators & Filtering
- 3.2 Sorting & Limiting Results

S05 **Aggregates and Subqueries**

- 3.3 Aggregate Functions & GROUP BY
- 3.4 Subqueries & Nested Queries

S06 **Relationships and Foreign Keys**

- 4.1 Table Relationships
- 4.2 Foreign Keys & Referential Integrity

S07 **Joins and the Part 1 Assessment**

- 4.3 Joins

PART 2: BEYOND THE BASICS · SESSIONS 8 TO 14

S08 CTEs and Window Functions

- 5.1 CTEs & Recursive Queries
- 5.2 Window Functions

S09 Indexes and Views

- 6.1 Indexes & Query Optimization
- 6.2 Views

S10 Normalization and Stored Programs

- 6.3 Database Design & Normalization
- 7.1 Stored Procedures & Functions

S11 Transactions and JSON

- 7.2 Transactions
- 7.3 JSON Columns

S12 Users, Permissions and Backups

- 7.4 User Management & Permissions
- 8.1 Importing & Exporting Data

S13 Final Project Kickoff and Studio I

- 8.2 Final Project: Design and Build a Database

S14 Final Project Studio II, Showcase and Next Steps

- 8.2 Final Project: Design and Build a Database
- 8.3 Next Steps & Best Practices

MYSQL FOUNDATIONS · ANSWER KEY

Part 1: SQL Foundations

Sessions 1 to 7. Modules 1 to 4: install MySQL, build and change tables, query, summarize and join. Runs on its own as a short course and ends with the Part 1 assessment.

INSTRUCTOR KIT · ANSWER KEY

Session 1: What Databases Are and Installing MySQL

Answers and reasoning for every Session 1 item: the diagnostic, both self-checks, the predict-then-run items, the lab reference outputs, the lesson pages' own quizzes and the exit ticket.

Session 1 | 12 self-check items | Confidential

START HERE · DIAGNOSTIC

Not graded. Each reason also says where the skill comes back in this course. The sheet is the one on the workbook's Start Here page; MySQL's own answers to D1 to D5 are in the diagnostic lab's output below.

D1

You filter the city column to show only Chicago. How many rows are left?

A) 1

B) 2

C) 3

D) 6

> **Why:** Orders 1002 and 1003. A filter keeps the rows that match a condition; in SQL that is WHERE city = 'Chicago'.
> **Skill:** filtering rows. **Where it comes back:** WHERE in Lesson 3.1 (and the lesson's first SELECT).

D2

You sort by price, largest first. Which item is on the top row?

A) Pride and Prejudice

B) The Hobbit

C) Dune

D) 1984

> **Why:** 14.99 is the highest price (Dune appears twice with it). Sorting in SQL is ORDER BY price DESC. **Skill:** sorting.
> **Where it comes back:** ORDER BY in Lesson 3.2.

D3

How many rows have the city New York AND a qty of 2 or more?

A) 1

B) 2

C) 4

D) 5

Why: Only 1005 (Frank Miller) meets both conditions. With OR the answer would be 4 rows (1001, 1002, 1004, 1005): the most common slip, and the reason for the lab's TODO 3. **Skill:** AND versus OR. **Where it comes back:** WHERE with AND/OR in Lesson 3.1.

D4

You add a column total = qty × price. What is the total for order 1004?

A) 14.99

B) 17.99

C) 29.98

D) 44.97

Why: 3 × 14.99 = 44.97. In SQL a formula column is an expression in the SELECT list: qty * price AS total. **Skill:** a formula column. **Where it comes back:** expressions in SELECT (Lesson 1.1's taste, then 3.3).

D5

=COUNTA(city) counts the filled-in cells of the city column. What does it give?

A) 4

B) 5

C) 6

D) 0

Why: Six rows, but Eve's city is empty, so 5. SQL has the same idea: COUNT(city) skips empty (NULL) values, COUNT(*) counts rows. **Skill:** empty cells. **Where it comes back:** NULL in Lessons 2.2 and 3.3.

D6

You saved bookstore.sql in your Downloads folder. Your terminal is open in your home folder. Which path names the file?

A) `bookstore.sql`

B) `Downloads/bookstore.sql`

C) `/bookstore.sql`

D) `home/Downloads.sql`

Why: A relative path starts from the folder the terminal is in: from home, the file is inside Downloads. (Windows writes Downloads\bookstore.sql.) From Lesson 1.3 on you run .sql files this way: SOURCE Downloads/bookstore.sql. **Skill:** files and folders. **Where it comes back:** the Setup Guide and Lesson 1.3 (running .sql files).

1.1 · WHAT IS A DATABASE?: SELF-CHECK

Q1

Which sentence describes MySQL correctly?

A) MySQL is a database: the file that holds your tables

B) MySQL is a database management system: server software that stores databases, enforces their rules and answers SQL

C) MySQL is a query language, a dialect of SQL

D) MySQL is a spreadsheet program that understands SQL

Why: A database is the organized data (for example the course's bookstore); the DBMS is the program that guards it and answers queries. MySQL, MariaDB and PostgreSQL are DBMSs.
A is wrong: The database is your data; MySQL is the software around it, and one MySQL server holds many databases.
C is wrong: SQL is the language; MySQL is a product that understands it (with its own small dialect).
D is wrong: A DBMS enforces types, keys and permissions that a spreadsheet does not.

Q2

A `books` table has the columns `id`, `title`, `author_id`, `price`. Which column is most likely a foreign key?

A) `id`

B) `title`

C) `author_id`

D) `price`

> **Why:** `author_id` holds another table's primary key (`authors.id`) to say which author wrote the book: that is what a foreign key is.
> **A** is wrong: `id` is this table's own primary key.
> **B** is wrong: A title is ordinary data; two books can even share one.
> **D** is wrong: A price is ordinary data, not a pointer to another row.

Q3

What does the last statement print?

```
CREATE TABLE pets (id INT PRIMARY KEY, name VARCHAR(20), kind VARCHAR(10));
INSERT INTO pets VALUES (1, 'Rex', 'dog'), (2, 'Tom', 'cat'), (3, 'Fido', 'dog');
UPDATE pets SET kind = 'cat' WHERE id = 3;
SELECT name FROM pets WHERE kind = 'cat' ORDER BY id;
```

Two rows: `Tom` and `Fido`

> **Why:** The UPDATE changes only the row whose id is 3 (Fido) from dog to cat, so two cats come back, in id order. A WHERE clause picks the rows an UPDATE touches, exactly as it picks the rows a SELECT shows.

Q4

The pets table from Q3 is still there. What happens when this runs?

```
-- pets now holds ids 1, 2 and 3
INSERT INTO pets VALUES (2, 'Kit', 'cat');
```

A) `ERROR 1062 (23000): Duplicate entry '2' for key 'pets.PRIMARY'`

B) A new row is added: Kit gets id 4 automatically

C) `ERROR 1366 (HY000): Incorrect integer value`

D) Tom's row is replaced by Kit's

Why: id is the primary key and 2 already belongs to Tom. A primary key must be unique, so MySQL refuses the whole row (the lesson's Common Mistake 1). Leaving the id to AUTO_INCREMENT avoids it.
B is wrong: The id was given explicitly (and this table has no AUTO_INCREMENT): MySQL does not change your values.
C is wrong: 2 is a perfectly good integer; 1366 is for text in a number column.
D is wrong: An INSERT never overwrites an existing row.

Q5

A music app must remember, for millions of logged-in users, which song each is playing right now, and look it up in under a millisecond. Which kind of database fits best?

A) A relational database with a table per song

B) A graph database

C) A document store

D) A key-value store such as Redis

Why: One value (the current song) looked up by one key (the session or user id), very fast, usually in memory: the job key-value stores are built for. The app's accounts and payments would still live in a relational database.
A is wrong: It would work, but the data is a single key and value, and speed is the whole requirement.
B is wrong: A graph database answers questions about chains of connections.
C is wrong: Documents suit records whose fields vary; here every record is one key and one value.

Q6

Carol closes her account. Which statement removes her row but keeps the table and every other customer?

A) `DROP TABLE customers;`

B) `DELETE FROM customers WHERE id = 3;`

C) `UPDATE customers SET email = NULL WHERE id = 3;`

D) `DELETE FROM customers;`

> **Why:** DELETE removes rows (the D in CRUD); its WHERE clause picks which ones.
> **A** is wrong: DROP TABLE removes the table itself, structure and all rows.
> **C** is wrong: UPDATE changes values in a row that stays (and here it would break the NOT NULL rule on email).
> **D** is wrong: Without WHERE, DELETE removes every customer.

1.1 · WHAT IS A DATABASE?: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
CREATE TABLE notes (id INT PRIMARY KEY, title VARCHAR(30) NOT NULL);
INSERT INTO notes VALUES (1, 'Buy milk'), (2, 'Call Bob'), (3, 'Pay rent');
DELETE FROM notes WHERE id = 2;
SELECT * FROM notes;
```

```
+----+----------+
| id | title    |
+----+----------+
|  1 | Buy milk |
|  3 | Pay rent |
+----+----------+
```

> **Why:** DELETE removes only the row whose id is 2; the other two stay with their ids. Ids are never renumbered after a delete.

P2

Predict the output, then run it.

```
CREATE TABLE members (id INT PRIMARY KEY, email VARCHAR(50) NOT NULL UNIQUE);
INSERT INTO members VALUES (1, 'ana@example.com');
INSERT INTO members VALUES (2, 'ana@example.com');
```

```
ERROR 1062 (23000): Duplicate entry 'ana@example.com' for key 'members.email'
```

Why: The first INSERT works. The second repeats an email in a UNIQUE column, so MySQL refuses it with 1062 and names the key (`members.email`). The table keeps one row.

1.2 · INSTALLING MYSQL: SELF-CHECK

Q7

`mysql --version` in a terminal prints 9.7.2. What does that prove?

A) The MySQL server is running

B) Your password is correct

C) The mysql client program is installed and on your PATH

D) Your server is MySQL 9.7, not MariaDB

Why: `mysql --version` describes the client on your computer and works with no server running at all. Only `SELECT VERSION();`, sent after logging in, describes the server (the lesson's Exercise 1).

A is wrong: No connection is made; a stopped server gives the same answer.

B is wrong: No password is asked for.

D is wrong: A 9.7 client can talk to an 8.4 or a MariaDB server; ask the server.

Q8

`mysql -u root -p` answers `ERROR 1045 (28000): Access denied for user ...`. What do you fix first?

A) The user name or the password

B) Start the server

C) Add mysql to your PATH

D) Reinstall MySQL with the native password plugin

> **Why:** 1045 means the server is running and answered, but refused the credentials. Retype the password (it is case-sensitive); remember -p; on Ubuntu's own package root may only work through sudo mysql.
> **B** is wrong: A stopped server cannot refuse you: it gives 2002 or 2003.
> **C** is wrong: Without the client on the PATH you would get "command not found", not a MySQL error.
> **D** is wrong: That plugin is removed in 9.x (error 1524), and the problem is the credentials.

Q9

On the course's MySQL 9.7 server, what does this print?

```
SELECT VERSION() LIKE '9.7.%' AS on_97;
```

`1` (in a column named `on_97`)

> **Why:** `LIKE` compares the version text with a pattern (% means anything), and a comparison gives 1 for true and 0 for false. `VERSION()` is `9.7.2` here, so the answer is 1; on 8.4 or MariaDB it would be 0.

Q10

You use Homebrew on a Mac. Which command installs the version this course uses?

A) `brew install mysql`

B) `brew install mariadb`

C) `brew install mysql-client`

D) `brew install mysql@9.7`

> **Why:** The versioned formula `mysql@9.7` stays on the 9.7 LTS series (it is keg-only: add its bin folder to the PATH).
> **A** is wrong: The plain formula follows the newest innovation release (calendar numbers such as 26.x).
> **B** is wrong: That is MariaDB, a different product.
> **C** is wrong: That installs only the client, with no server.

Q11

Your MySQL container was started with `-p 3307:3306`. Which command connects to it from a mysql client installed on your laptop?

A) `mysql -u root -p`

B) `mysql -h localhost -P 3306 -u root -p`

C) `mysql -h 127.0.0.1 -P 3307 -u root -p`

D) `mysql -h 127.0.0.1 -P 3306 -u root -p`

> **Why:** The container's port 3306 is published on the laptop's port 3307, and 127.0.0.1 forces a network connection (localhost would make the client look for a local socket file and ignore the port).
> **A** is wrong: That looks for a local server on the default socket: error 2002.
> **B** is wrong: localhost ignores -P on macOS and Linux, and 3306 is the wrong laptop port anyway.
> **D** is wrong: Right host, wrong port: 3306 on the laptop is not where the container listens.

Q12

On a fresh MySQL 9.7 install (not Docker), what does this print?

```
SELECT plugin
FROM mysql.user
WHERE user = 'root' AND host = 'localhost';
```

`caching_sha2_password`

> **Why:** caching_sha2_password is the default login method for every new account in 8.4 and 9.x.
> mysql_native_password no longer exists in 9.x, so you should never see it there.

1.2 · INSTALLING MYSQL: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
SELECT CURRENT_USER();
```

```
+----------------+
| CURRENT_USER() |
+----------------+
| root@localhost |
+----------------+
```

> **Why:** CURRENT_USER() is the account MySQL matched: root, connected from this computer. Through Docker's published port it would be root@%.

P4

Predict the output, then run it.

```
SELECT VERSION() LIKE '%MariaDB%' AS on_mariadb,
       @@character_set_server   AS charset;
```

```
+------------+---------+
| on_mariadb | charset |
+------------+---------+
|          0 | utf8mb4 |
+------------+---------+
```

> **Why:** 0 because the server is MySQL, not MariaDB; utf8mb4 is the default character set since 8.0.

LAB REFERENCE SOLUTIONS

- ENVIRONMENT CHECK: LABS/MODULE-01/L1_0-ORIENTATION/CHECK_ENVIRONMENT.SQL (+ .SH / .PS1)

A tool, not an exercise. On a ready MySQL 9.7 server in Session 1 it prints (the `.sh` and `.ps1` helpers first print a `client` line with `mysql --version` and an OK or CHECK line for the client):

```
+----------------+------------------------------+
| server_version | edition                      |
+----------------+------------------------------+
| 9.7.2          | MySQL Community Server - GPL |
+----------------+------------------------------+
+-------------------------------------------------+
| server_check                                    |
+-------------------------------------------------+
| OK     MySQL 9.7 LTS, the version the course uses |
+-------------------------------------------------+
+------------------------------+--------------------------------+
| charset_check                | sql_mode_check                 |
+------------------------------+--------------------------------+
| OK     utf8mb4 (full Unicode) | OK     default strict SQL mode |
+------------------------------+--------------------------------+
+----------------+-----------------------+
| logged_in_as   | login_plugin          |
+----------------+-----------------------+
| root@localhost | caching_sha2_password |
+----------------+-----------------------+
+-----------------------------------------------------------------+
| write_check                                                     |
+-----------------------------------------------------------------+
| OK     created a database and a table, and read back: it works |
+-----------------------------------------------------------------+
+---------------------------------------------------------+
| bookstore_check                                         |
+---------------------------------------------------------+
| INFO   no bookstore database yet: Lesson 2.1 creates it |
+---------------------------------------------------------+
+-----------------------------+
| verdict                     |
+-----------------------------+
| READY for MySQL Foundations |
+-----------------------------+
```

On MySQL 8.4 the server line says CHECK and the verdict `READY WITH NOTES`. On MariaDB (measured on 12.3): the server line, the SQL mode line (no `ONLY_FULL_GROUP_BY`) and the login plugin (`mysql_native_password`, normal there) differ, and the verdict is `READY WITH NOTES`. A client that cannot connect never reaches the tables: the helpers print the ERROR line and the triage hint (1045 credentials, 2002/2003 server or address).

- `DIAGNOSTIC PRACTICE: LABS/MODULE-01/L1_0-ORIENTATION/DIAGNOSTIC_STARTER.SQL → DIAGNOSTIC_SOLUTION.SQL`

The six Start Here questions, D1 to D5 done in SQL (D6 is about paths). The solution prints:

```
Database changed
+----------+---------------+----------+---------------------+-----+-------+
| order_no | customer      | city     | item                | qty | price |
+----------+---------------+----------+---------------------+-----+-------+
|     1001 | Alice Johnson | New York | Dune                |   1 | 14.99 |
|     1002 | Bob Smith     | Chicago  | 1984                |   2 |  9.99 |
|     1003 | Grace Wilson  | Chicago  | The Hobbit          |   1 | 10.99 |
|     1004 | Dave Brown    | Houston  | Dune                |   3 | 14.99 |
|     1005 | Frank Miller  | New York | Fahrenheit 451      |   2 | 10.49 |
|     1006 | Eve Davis     | NULL     | Pride and Prejudice |   1 |  7.99 |
+----------+---------------+----------+---------------------+-----+-------+
+----------+--------------+---------+
| order_no | customer     | city    |
+----------+--------------+---------+
|     1002 | Bob Smith    | Chicago |
|     1003 | Grace Wilson | Chicago |
+----------+--------------+---------+
+------+-------+
| item | price |
+------+-------+
| Dune | 14.99 |
+------+-------+
+----------+--------------+----------+-----+
| order_no | customer     | city     | qty |
+----------+--------------+----------+-----+
|     1005 | Frank Miller | New York |   2 |
+----------+--------------+----------+-----+
+----------+-----+-------+-------+
| order_no | qty | price | total |
+----------+-----+-------+-------+
|     1004 |   3 | 14.99 | 44.97 |
+----------+-----+-------+-------+
+----------+------------------+
| all_rows | cities_filled_in |
+----------+------------------+
|        6 |                5 |
+----------+------------------+
```

Common sticking points: deleting the quotes with the placeholder in D1 (`ERROR 1064`); putting DESC before `price` in D2; TODO 3 with OR left in place (four rows: exactly the diagnostic's most common wrong answer).

- LAB 1.1-A: LABS/MODULE-01/L1_1-WHAT-IS-A-DATABASE/ORDER_SHEET_STARTER.SQL → ORDER_SHEET_SOLUTION.SQL

The solution prints:

```
Database changed
+----------+---------------+-------------------+
| order_no | customer_name | customer_email    |
+----------+---------------+-------------------+
|        1 | Alice Johnson | alice@example.com |
|        2 | Alice Jonson  | alice@example.com |
|        6 | alice johnson | alice@example.com |
+----------+---------------+-------------------+
+----------+---------------+---------------+
| order_no | customer_name | customer_city |
+----------+---------------+---------------+
|        3 | Bob Smith     | Chicago       |
|        4 | Bob Smith     | Boston        |
+----------+---------------+---------------+
+----------+------------------+
| order_no | books            |
+----------+------------------+
|        1 | Dune, 1984, 1984 |
|        3 | Dune             |
|        4 | Dune Messiah     |
|        5 | The Hobbit, Dune |
+----------+------------------+
+----------+----------+
| order_no | total    |
+----------+----------+
|        5 | about 26 |
+----------+----------+
+----------+-------------+-------+
| order_no | customer_id | total |
+----------+-------------+-------+
|        1 |           1 | 34.97 |
|        2 |           1 |  7.99 |
|        6 |           1 |  9.49 |
+----------+-------------+-------+
+----+---------------+---------------------+
| id | name          | email               |
+----+---------------+---------------------+
|  1 | Alice Johnson | alice.j@example.com |
|  2 | Bob Smith     | bob@example.com     |
|  3 | Grace Wilson  | grace@example.com   |
+----+---------------+---------------------+
```

What each TODO teaches: (1) the same customer typed three ways (two by a typo, one by capitals), because the name is copied into every order; (2) two cities for one customer: one copy was updated, the other was not; (3) a list in a cell forces a text search, and `LIKE '%Dune%'` also matches Dune Messiah: a silent wrong answer, the most important of the four; (4) a text column accepts `about 26`, so the total cannot be trusted or added up. Step 5 rebuilds it as customers + orders: Alice exists once, her three orders point at id 1, and one UPDATE changes her

email everywhere. The relational design still lacks an order_items table for the books: that is the 1.1 project's Step 4.

- LAB 1.1-B: LABS/MODULE-01/L1_1-WHAT-IS-A-DATABASE/CRUD_TOUR_STARTER.SQL → CRUD_TOUR_SOLUTION.SQL

The solution prints:

```
Database changed
+----+------------------+-------+
| id | title            | price |
+----+------------------+-------+
|  2 | The Great Gatsby |  8.99 |
+----+------------------+-------+
+----+------------------+-------+----------+
| id | title            | price | in_stock |
+----+------------------+-------+----------+
|  2 | The Great Gatsby |  8.99 |        0 |
|  3 | The Hobbit       |  9.99 |        1 |
|  4 | Fahrenheit 451   | 10.49 |        1 |
+----+------------------+-------+----------+
```

Reading it: under $10 only The Great Gatsby qualifies before the sale; Fahrenheit 451 gets id 4 because ids keep counting; The Hobbit drops to 9.99; in_stock prints 0 and 1 (a BOOLEAN is a tiny integer in MySQL); Dune (id 1) is gone and id 1 is not reused. Ask what a DELETE without its WHERE line would have done: every book gone, no undo.

- HANDOUT: LABS/MODULE-01/L1_1-WHAT-IS-A-DATABASE/DEBUG_RULES.SQL (DELIBERATELY BROKEN)

As shipped, MySQL prints (the four bugs, then the table with the three rows that got in):

```
Database changed
ERROR 1062 (23000): Duplicate entry '2' for key 'books.PRIMARY'
ERROR 1364 (HY000): Field 'title' doesn't have a default value
ERROR 1366 (HY000): Incorrect decimal value: 'cheap' for column 'price' at row 1
ERROR 1406 (22001): Data too long for column 'isbn' at row 1
+----+--------------------+---------------+-------+
| id | title              | isbn          | price |
+----+--------------------+---------------+-------+
|  1 | Dune               | 9780441013593 | 14.99 |
|  2 | 1984               | 9780451524935 |  9.99 |
|  6 | Pride and Prejudice | 9780141439518 |  7.99 |
+----+--------------------+---------------+-------+
```

1. **Bug A, 1062:** id 2 already belongs to 1984. Fix: another id (3) or a table with AUTO_INCREMENT and no id in the INSERT.
2. **Bug B, 1364:** title is NOT NULL and the INSERT leaves it out. Fix: add the title (`'The Hobbit'`).

3. **Bug C, 1366:** `'cheap'` is not a decimal number. Fix: `9.49`.
4. **Bug D, 1406:** the ISBN with hyphens has 17 characters; the column holds 13. Fix: `'9781451673319'` (or a wider column).

Fixed, the final SELECT shows six books and no ERROR lines. On MariaDB 1062 names the key `'PRIMARY'` and 1366 the full column `kit_s1_rules`.`books`.`price`.

- LAB 1.2-A: LABS/MODULE-01/L1_2-INSTALLING-MYSQL/SERVER_FACTS_STARTER.SQL → SERVER_FACTS_SOLUTION.SQL

The solution prints (on the course's MySQL 9.7 test server, Linux):

```
+---------+------------------------------+
| version | edition                      |
+---------+------------------------------+
| 9.7.2   | MySQL Community Server - GPL |
+---------+------------------------------+
+-------------+------------+
| is_mysql_97 | is_mariadb |
+-------------+------------+
|           1 |          0 |
+-------------+------------+
+---------+--------------------+
| charset | collation          |
+---------+--------------------+
| utf8mb4 | utf8mb4_0900_ai_ci |
+---------+--------------------+
+--------+------------------+
| engine | lower_case_names |
+--------+------------------+
| InnoDB |                0 |
+--------+------------------+
+-----------------+
| max_connections |
+-----------------+
|             151 |
+-----------------+
+--------------------+
| SCHEMA_NAME        |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
```

What may differ on a learner's server: the patch version (any 9.7.x); `lower_case_names` 1 on Windows and 2 on macOS (names are compared without case there, with case on Linux: a reason to always type table names exactly); `max_connections` if someone changed it. On 8.4 `is_mysql_97` is 0; on MariaDB `is_mariadb` is 1, the edition is `mariadb.org binary distribution` and the collation `utf8mb4_uca1400_ai_ci`. The column header `SCHEMA_NAME` comes back in capitals: information_schema's own names, not a mistake.

- LAB 1.2-B: LABS/MODULE-01/L1_2-INSTALLING-MYSQL/ACCOUNTS_CHECK_STARTER.SQL → ACCOUNTS_CHECK_SOLUTION.SQL

The solution prints:

```
+----------------+-----------------+
| you_typed      | matched_account |
+----------------+-----------------+
| root@localhost | root@localhost  |
+----------------+-----------------+
+------------------+-----------+-----------------------+----------------+
| user             | host      | plugin                | account_locked |
+------------------+-----------+-----------------------+----------------+
| mysql.infoschema | localhost | caching_sha2_password | Y              |
| mysql.session    | localhost | caching_sha2_password | Y              |
| mysql.sys        | localhost | caching_sha2_password | Y              |
| root             | localhost | caching_sha2_password | N              |
+------------------+-----------+-----------------------+----------------+
+------------------+----------------+
| user             | account_locked |
+------------------+----------------+
| mysql.infoschema | Y              |
| mysql.session    | Y              |
| mysql.sys        | Y              |
+------------------+----------------+
Empty set
```

Reading it: `USER()` is the name the client sent, `CURRENT_USER()` the account that matched (they differ with wildcard hosts, Lesson 7.4). The three `mysql.*` accounts are the server's own and locked (Y): nobody can log in as them. The empty last result is the good news: no account uses the removed login method. Docker adds `root` with host `%`. On MariaDB `mysql.user` has no `account_locked` column (`ERROR 1054`): drop that column there.

- HANDOUT: LABS/MODULE-01/L1_2-INSTALLING-MYSQL/DEBUG_OLD_ADVICE.SQL (DELIBERATELY BROKEN)

As shipped, MySQL prints:

```
ERROR 1524 (HY000): Plugin 'mysql_native_password' is not loaded
ERROR 1193 (HY000): Unknown system variable 'default_authentication_plugin'
Empty set
```

1. **1524:** the `mysql_native_password` plugin is not loaded: removed in 9.x, disabled by default in 8.4. No account is created (the check query is empty). Modern replacement: leave out `IDENTIFIED WITH ...` (the default `caching_sha2_password`) and update the client or library.
2. **1193:** `default_authentication_plugin` no longer exists. Modern replacement: `SELECT @@authentication_policy;`.

On MariaDB the first statement fails with 1064 (MariaDB writes `IDENTIFIED VIA`); with corrected MariaDB syntax it would CREATE the account, so never "fix" it in class.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 1.1 · WHAT IS A DATABASE?

1. **B** The database is the data; the DBMS is the software that manages it: The database is the organized data itself. The DBMS, such as MySQL, is the software that stores it, enforces its rules and answers queries about it.
2. **C** A foreign key: A foreign key holds another table's primary key to record which row this one belongs to. Many orders can point at the same customer.
3. **A** SELECT: R is Read, and reading rows is what SELECT does. INSERT, UPDATE and DELETE cover Create, Update and Delete.
4. **C** id 1 is already used, and a primary key must be unique: id is the primary key and 1 already belongs to Alice. Primary key values must be unique, so MySQL refuses the duplicate entry.
5. **B** An order_items table with one row per book per order, holding order_id, book_id and quantity: An order_items table holds one row per book in an order, with order_id and book_id foreign keys and a quantity. Each fact sits in its own row and can be counted and joined.

LESSON 1.2 · INSTALLING MYSQL

1. **C** docker run ... mysql:9.7: The 9.7 tag pins the LTS series. latest, plain brew install mysql and distribution packages can each give you a different version.
2. **A** The server is not running: 2002 and 2003 mean nothing answered. A stopped server is the usual reason; start it and try again.
3. **B** It fails with error 1524, because the plugin is not loaded: mysql_native_password is removed in 9.x, so the statement fails with error 1524. Current phpMyAdmin and PHP support the default caching_sha2_password.
4. **A** mysql -h 127.0.0.1 -P 3306 -u root -p: 127.0.0.1 forces a TCP connection to the published port. The name localhost makes the client look for a local socket file, which the container does not provide.
5. **C** It is MariaDB: most SQL works, but some results and features differ from MySQL 9.7: XAMPP bundles MariaDB, a MySQL fork with its own version numbers. Basic SQL runs, but some features, messages and result orders differ from MySQL 9.7.

OPENING: READING THE ROOM

There is no retrieval warm-up in Session 1. The opening's two checks have their answers above: the environment check output (what READY looks like) and the diagnostic D1 to D6. Learners who end the clinic still NOT READY leave with a numbered plan in their workbook's Start Here box.

EXIT TICKET

1. **Server, client, database:** the server (`mysqld`) stores the data, enforces the rules and answers SQL; a client (`mysql`, phpMyAdmin, Workbench) is what you type into and it sends your SQL to the server; a database is a named set of tables inside the server (`preview_shop` today, `bookstore` from Session 2).
2. **2002 vs 1045:** 2002 means nothing answered: check that the server is running (start command for your platform) and that you are asking the right place (Docker: `-h 127.0.0.1 -P <port>`). 1045 means the server answered and refused: check the user name and password (and `-p`).
3. **Keys:** `id` is the primary key, `customer_id` the foreign key (it holds a customers.id). Storing the name in every order copies one fact many times; the copies drift apart (the order-sheet lab's three Alices).

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 A ("MySQL is a database").** The everyday usage. Draw the three boxes again: one server, many databases inside.
- **Q4 B ("Kit gets id 4").** Expects MySQL to repair the input. A database refuses rather than guesses; that is the point of rules.
- **Q6 A or D (DROP TABLE / DELETE without WHERE).** Mixes up removing rows with removing the table, or forgets that WHERE picks the rows. Both are destructive with no undo: worth a minute in the Session 2 warm-up.
- **Q7 D ("the client version proves the server version").** The most common misconception of the day. Exercise 1 of Lesson 1.2 is built on it.
- **Q8 B ("start the server" for 1045).** Has not separated "no answer" from "answered and refused". Repeat the triage line.
- **Q11 B or D.** Either localhost (socket, ignores the port) or the container's internal port. Draw the two ports: the laptop's 3307 leads to the container's 3306.
- **Exit ticket 1 with "client = phpMyAdmin only".** Fine, but ask what they type into in the terminal: the `mysql` command is a client too.

INSTRUCTOR KIT · ANSWER KEY

Session 2: The MySQL Client and Your First Tables

Answers and reasoning for every Session 2 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs, the lesson pages' own quizzes and the exit ticket.

Session 2 | 12 self-check items | Confidential

WARM-UP: SESSION 1

Not graded. Each reason says where the idea came from in Session 1.

W1

Is MySQL a database?

A) Yes: MySQL is the file that holds your tables

B) No: MySQL is the server (a DBMS); a database such as bookstore lives inside it

C) No: MySQL is a query language

D) Yes: every MySQL install is one database

Why: MySQL is the server software; one server holds many databases. Session 1's exit ticket question 1.

W2

Which error tells you the server is running and answered?

A) `ERROR 2002`

B) `ERROR 2003`

C) `ERROR 1045 (28000): Access denied`

D) None of them

Why: 1045 means the server answered and refused the credentials; 2002 and 2003 mean nothing answered.

W3

What does the SELECT print?

```
CREATE TABLE pets (id INT PRIMARY KEY, name VARCHAR(20));
INSERT INTO pets VALUES (1, 'Rex'), (2, 'Tom'), (3, 'Fido');
DELETE FROM pets WHERE id = 2;
SELECT * FROM pets;
```

A) Rows 1 Rex and 3 Fido

B) Rows 1 Rex and 2 Fido: the ids are renumbered

C) All three rows

D) `Empty set`

> **Why:** DELETE removes only the row whose id is 2; ids are never renumbered. MySQL prints:
>
> ```
> +----+------+
> | id | name |
> +----+------+
> |  1 | Rex  |
> |  3 | Fido |
> +----+------+
> ```

W4

Which statement removes Carol's row (id 3) and keeps every other customer?

A) `DROP TABLE customers;`

B) `DELETE FROM customers;`

C) `DELETE FROM customers WHERE id = 3;`

D) `UPDATE customers SET id = NULL WHERE id = 3;`

> **Why:** DELETE with a WHERE removes the rows the WHERE picks. Without WHERE every row goes; DROP TABLE removes the table.

W5

`mysql --version` **prints 9.7.2. What does that prove about your server?**

A) That it is 9.7.2

B) That it is running

C) That it is MySQL, not MariaDB

D) Nothing: it describes the client; ask the server with SELECT VERSION();

> **Why:** The client and the server are different programs. Only a query sent after logging in describes the server.

1.3 · MYSQL CLI & PHPMYADMIN: SELF-CHECK

Q1

Which of these does nothing in phpMyAdmin's SQL tab, because only the mysql command-line client understands it?

A) `SHOW DATABASES;`

B) `DESCRIBE user;`

C) `SOURCE ~/mysql-course/hello.sql`

D) `SELECT DATABASE();`

> **Why:** `SOURCE` (like `\G`, `\c`, help and exit) is a client command: the mysql program handles it itself and never sends it to the server. phpMyAdmin has its own Import tab for files.
>
> **A** is wrong: SHOW DATABASES is SQL (a MySQL addition to the standard): every client can send it.
>
> **B** is wrong: DESCRIBE is SQL too; phpMyAdmin also shows it as the Structure tab.
>
> **D** is wrong: A SELECT is plain SQL and works in any client.

Q2

You press Enter and the prompt changes from `mysql>` to `'>`. What does it mean?

A) The statement only needs a semicolon

B) A single quote is still open: close it, then type ;

C) The server is still running the query

D) You were logged out

> **Why:** The client shows `'>` while a text value in single quotes is unfinished. Close the quote, then end the statement with a semicolon (or `\c` after closing the quote to throw it away).
> **A** is wrong: That prompt is `->`; inside an open quote a semicolon is just part of the text.
> **C** is wrong: Nothing has been sent yet: the statement is not finished.
> **D** is wrong: Logging out prints Bye and returns you to the shell.

Q3

On MySQL 9.7, how many rows does this print?

```
SHOW TABLES FROM mysql LIKE 'time_zone%';
```

5 (`time_zone` , `time_zone_leap_second` , `time_zone_name` , `time_zone_transition` , `time_zone_transition_type`)

> **Why:** SHOW TABLES FROM lists another database's tables without switching to it, and LIKE filters the names: % means any characters. The mysql database keeps five time-zone tables.

Q4

You type `mysql -u root -p secret123` (with a space after -p). What happens?

A) It logs in with the password secret123

B) `ERROR 1045 (28000): Access denied`

C) It logs in without asking for a password

D) It asks for the password, then fails with `ERROR 1049 (42000): Unknown database 'secret123'`

> **Why:** -p on its own means "ask me"; the word after the space is read as the database to start in. Type the password at the prompt instead (the lesson's warning box).
> **A** is wrong: Only -psecret123 (no space) would pass it, and that leaves it in your shell history.
> **B** is wrong: The password typed at the prompt can be right; the failure is the unknown database.
> **C** is wrong: -p always makes the client ask.

Q5

report.sql ends its query with `\G`. Which way of running it stops with Unknown command on MySQL 9.7?

A) `mysql -u root -p < report.sql`

B) `SOURCE report.sql` at the mysql> prompt

C) `mysql --commands -u root -p < report.sql`

D) Typing the query at the prompt

> **Why:** Read from a redirected file, the 9.x client is in batch mode and ignores its own commands unless started with --commands. At the prompt, typed or SOURCEd, client commands always work.
> **B** is wrong: SOURCE runs the file as if you typed it: client commands work.
> **C** is wrong: --commands switches client commands back on in batch mode.
> **D** is wrong: Interactive use is not affected.

Q6

Which statement lists only the columns of mysql.user whose names contain lock?

A) `SHOW TABLES FROM mysql LIKE '%lock%';`

B) `SHOW COLUMNS FROM mysql.user LIKE '%lock%';`

C) `DESCRIBE mysql.user WHERE lock;`

D) `SELECT lock FROM mysql.user;`

> **Why:** SHOW COLUMNS FROM table LIKE 'pattern' is DESCRIBE with a filter. On 9.7 it prints two columns: Lock_tables_priv and account_locked (LIKE ignores case).
> **A** is wrong: That lists tables, not columns.
> **C** is wrong: DESCRIBE takes no WHERE: a syntax error.
> **D** is wrong: There is no column named lock: ERROR 1054.

1.3 · MYSQL CLI & PHPMYADMIN: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
SELECT DATABASE() AS before_use;
USE mysql;
SELECT DATABASE() AS after_use;
```

```
+------------+
| before_use |
+------------+
| NULL       |
+------------+
Database changed
+-----------+
| after_use |
+-----------+
| mysql     |
+-----------+
```

> **Why:** Before the first USE a session has no current database, so DATABASE() is NULL. `USE mysql;` prints `Database changed`, and from then on it is mysql.

P2

Predict the output, then run it.

```
SHOW COLUMNS FROM mysql.user LIKE 'account_locked';
```

```
+----------------+---------------+------+-----+---------+-------+
| Field          | Type          | Null | Key | Default | Extra |
+----------------+---------------+------+-----+---------+-------+
| account_locked | enum('N','Y') | NO   |     | N       |       |
+----------------+---------------+------+-----+---------+-------+
```

Why: account_locked is an ENUM of two letters, required, N by default: new accounts are unlocked unless you lock them.

2.1 · CREATING DATABASES & TABLES: SELF-CHECK

Q7

What happens when this runs?

```
CREATE DATABASE kit_q7 CHARACTER SET utf8;
SHOW WARNINGS;
```

A) `ERROR 1064`: utf8 is not a character set

B) The database stores full Unicode, emoji included

C) Nothing: CHARACTER SET is ignored

D) The database is created with utf8mb3 (no emoji), and warning 3719 says to use utf8mb4

Why: MySQL's utf8 is an alias for the three-byte utf8mb3, which cannot store emoji and some rarer characters. The statement works and warns; always write utf8mb4.
A is wrong: utf8 is accepted, with a warning.
B is wrong: Only utf8mb4 stores every character.
C is wrong: It is used: the database gets utf8mb3.

Q8

Which column does the last statement list first?

```
CREATE TABLE members (
    id   INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(40) NOT NULL
);
ALTER TABLE members ADD COLUMN email VARCHAR(100) FIRST;
SHOW COLUMNS FROM members;
```

`email` (then `id` and `name`)

> **Why:** ADD COLUMN puts a new column at the end unless you say AFTER column or FIRST. FIRST makes email the first column; existing rows would get NULL in it.

Q9

A script stops with `ERROR 1046 (3D000): No database selected`. What does it mean?

A) There is no current database: run USE first, or write database.table

B) The database name is misspelled

C) The table already exists

D) Your account may not create tables

> **Why:** A fresh session, or one whose current database was just dropped, has no current database, so a bare table name has nowhere to go.
>
> **B** is wrong: A misspelled database gives 1049 Unknown database.
>
> **C** is wrong: That is 1050.
>
> **D** is wrong: Permission errors are 1044 or 1142 (Lesson 7.4).

Q10

Which name follows the course's naming conventions for a table of book reviews?

A) `BookReviews`

B) `tbl_review`

C) `book_reviews`

D) `bookReview`

> **Why:** Lowercase, snake_case, a plural noun. Lowercase also keeps the name working when the database moves to a Linux server, where table names are case-sensitive.
> **A** is wrong: Capitals can break on Linux servers.
> **B** is wrong: No tbl_ prefix, and tables are plural.
> **D** is wrong: camelCase and singular.

Q11

In `shelf`, title was `VARCHAR(200) NOT NULL`. After the ALTER below, what is true?

```
CREATE TABLE shelf (
    id    INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(200) NOT NULL
);
ALTER TABLE shelf CHANGE COLUMN title book_title VARCHAR(250);
SHOW COLUMNS FROM shelf;
```

A) The column is now book_title and is still required

B) The column is now book_title, VARCHAR(250), and allows NULL

C) ERROR: CHANGE cannot rename a column

D) title is kept and a new column book_title is added

> **Why:** CHANGE takes the old name, the new name and the FULL new definition. Whatever is left out, here NOT NULL, is gone: SHOW COLUMNS prints YES under Null. Repeat every constraint, then check.
> **A** is wrong: Only a definition that says NOT NULL keeps it.
> **C** is wrong: Renaming plus redefining is exactly what CHANGE is for.
> **D** is wrong: CHANGE replaces the column; ADD COLUMN would add one.

Q12

Write the one statement that removes every row of the customers table, keeps the table and starts the ids at 1 again.

```
TRUNCATE TABLE customers;
```

> **Why:** TRUNCATE empties the table and resets AUTO_INCREMENT. DELETE FROM customers; also removes every row but keeps counting ids; DROP TABLE removes the table itself.

2.1 · CREATING DATABASES & TABLES: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
CREATE TABLE tickets (
    id    INT AUTO_INCREMENT PRIMARY KEY,
    label VARCHAR(20) NOT NULL
);
INSERT INTO tickets (label) VALUES ('first'), ('second');
TRUNCATE TABLE tickets;
INSERT INTO tickets (label) VALUES ('third');
SELECT * FROM tickets;
```

```
+----+-------+
| id | label |
+----+-------+
|  1 | third |
+----+-------+
```

> **Why:** TRUNCATE removed both rows AND reset the AUTO_INCREMENT counter, so the next row gets id 1. A DELETE FROM tickets would have given id 3.

P4

Predict the output, then run it.

```
CREATE TABLE notes (
    id    INT PRIMARY KEY,
    body  VARCHAR(100) NOT NULL
);
ALTER TABLE notes MODIFY COLUMN body TEXT;
SHOW COLUMNS FROM notes LIKE 'body';
```

```
+-------+------+------+-----+---------+-------+
| Field | Type | Null | Key | Default | Extra |
+-------+------+------+-----+---------+-------+
| body  | text | YES  |     | NULL    |       |
+-------+------+------+-----+---------+-------+
```

Why: MODIFY replaced the whole definition with `TEXT` alone, so NOT NULL is gone: Null says YES. Write `MODIFY COLUMN body TEXT NOT NULL` to keep the rule.

LAB REFERENCE SOLUTIONS

- LAB 1.3-A: LABS/MODULE-02/L1_3-MYSQL-CLI-PHPMYADMIN/FIND_YOUR_WAY_STARTER.SQL → FIND_YOUR_WAY_SOLUTION.SQL

The solution prints:

```
Database changed
+-------------+
| current_db  |
+-------------+
| kit_s2_cafe |
+-------------+
+-----------------------+
| Tables_in_kit_s2_cafe |
+-----------------------+
| cafe_orders           |
| menu_items            |
| staff_notes           |
+-----------------------+
+----------+--------------+------+-----+---------+----------------+
| Field    | Type         | Null | Key | Default | Extra          |
+----------+--------------+------+-----+---------+----------------+
| id       | int          | NO   | PRI | NULL    | auto_increment |
| name     | varchar(60)  | NO   | UNI | NULL    |                |
| price    | decimal(5,2) | NO   |     | NULL    |                |
| vegan    | tinyint(1)   | YES  |     | 0       |                |
| added_on | date         | YES  |     | NULL    |                |
+----------+--------------+------+-----+---------+----------------+
+------------+----------+------+-----+---------+-------+
| Field      | Type     | Null | Key | Default | Extra |
+------------+----------+------+-----+---------+-------+
| ordered_at | datetime | NO   |     | NULL    |       |
| paid_at    | datetime | YES  |     | NULL    |       |
+------------+----------+------+-----+---------+-------+
+------------------------------+
| Tables_in_mysql (time_zone%) |
+------------------------------+
| time_zone                    |
| time_zone_leap_second        |
| time_zone_name               |
| time_zone_transition         |
| time_zone_transition_type    |
+------------------------------+
*************************** 1. row ***************************
      id: 3
    name: Oat Latte
   price: 4.10
   vegan: 1
added_on: 2026-02-14
*************************** 1. row ***************************
       Table: staff_notes
Create Table: CREATE TABLE `staff_notes` (
  `id` int NOT NULL AUTO_INCREMENT,
  `note` varchar(200) COLLATE utf8mb4_unicode_ci NOT NULL,
```

```
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
```

Reading it: menu_items has `id` as the primary key (PRI) and `name` as a unique column (UNI); `name` and `price` say NO under Null, so they are required; `vegan` is a BOOLEAN shown as tinyint(1) with default 0. Of the `_at` columns only `paid_at` may be empty. TODO 4 reaches the mysql database with FROM, without leaving the café. TODO 5's vertical row and TODO 6's definition use `\G`. Learners who wrote `LIKE '%_at'` get the same two rows here (the `_` matches any one character, and the only candidates end in _at); `'%\_at'` is the exact form.

- LAB 1.3-B: LABS/MODULE-02/L1_3-MYSQL-CLI-PHPMYADMIN/SYSTEM_TOUR_STARTER.SQL → SYSTEM_TOUR_SOLUTION.SQL

The solution prints (MySQL 9.7):

```
+--------------+
| user_columns |
+--------------+
|           51 |
+--------------+
+--------------+
| priv_columns |
+--------------+
|           31 |
+--------------+
+-----------------------+-------------+-------------+
| COLUMN_NAME           | COLUMN_TYPE | IS_NULLABLE |
+-----------------------+-------------+-------------+
| Host                  | char(255)   | NO          |
| User                  | char(32)    | NO          |
| plugin                | char(64)    | NO          |
| password_last_changed | timestamp   | YES         |
+-----------------------+-------------+-------------+
+---------------+------------+
| TABLE_NAME    | TABLE_ROWS |
+---------------+------------+
| help_category |         53 |
| help_keyword  |        922 |
| help_relation |       2265 |
| help_topic    |        706 |
+---------------+------------+
+----------------------------------------+
| Tables_in_information_schema (COLUMN%) |
+----------------------------------------+
| COLUMNS                                |
| COLUMNS_EXTENSIONS                     |
| COLUMN_PRIVILEGES                      |
| COLUMN_STATISTICS                      |
+----------------------------------------+
```

Reading it: 51 columns, 31 of them privilege flags (one per kind of permission, Lesson 7.4). Host and User are fixed-length CHAR; `password_last_changed` is the only one of the four that may be NULL. TABLE_ROWS is an estimate for InnoDB tables. On 8.4 the help-table estimates differ; on MariaDB `mysql.user` is a view with 47 columns (see the README).

HANDOUT: LABS/MODULE-02/L1_3-MYSQL-CLI-PHPMYADMIN/DEBUG_LOST.SQL (DELIBERATELY BROKEN)

As shipped, MySQL prints (the four errors, then the accounts):

```
ERROR 1049 (42000): Unknown database 'mysq'
Database changed
ERROR 1146 (42S02): Table 'mysql.users' doesn't exist
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresp
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresp
+------------------+-----------+
| User             | Host      |
+------------------+-----------+
| mysql.infoschema | localhost |
| mysql.session    | localhost |
| mysql.sys        | localhost |
| root             | localhost |
+------------------+-----------+
```

1. **Bug A, 1049:** `mysq` is a typo. Fix: `USE mysql;` (SHOW DATABASES; and copy the exact name).
2. **Bug B, 1146:** the accounts table is `user`, singular; the message names the database it looked in (`mysql.users`). Fix: `FROM user`.
3. **Bug C, 1064:** `current_date` is a built-in function (like current_time in the lesson). Fix: `AS today` or backticks.
4. **Bug D, 1064:** no semicolon after the first SELECT, so MySQL reads both lines as one statement and gets lost at the second SELECT (`near 'SELECT CURRENT_USER() ...' at line 2`). Fix: end the first line with `;`.

Fixed, the file prints version, account, a date and the four accounts with no ERROR lines. Docker adds root with host `%`.

- LAB 2.1-A: LABS/MODULE-02/L2_1-CREATING-DATABASES-TABLES/EVENTS_BUILD_STARTER.SQL → EVENTS_BUILD_SOLUTION.SQL

The solution prints:

```
Database changed
+-------+------+------------------------------+
| Level | Code | Message                      |
+-------+------+------------------------------+
| Note  | 1050 | Table 'rooms' already exists |
+-------+------+------------------------------+
+------------+--------------------+
| TABLE_NAME | TABLE_COLLATION    |
+------------+--------------------+
| readings   | utf8mb4_unicode_ci |
| rooms      | utf8mb4_unicode_ci |
+------------+--------------------+
+------------+---------------+
| TABLE_NAME | columns_count |
+------------+---------------+
| readings   |             7 |
| rooms      |             3 |
+------------+---------------+
+--------------+--------------+------+-----+---------+----------------+
| Field        | Type         | Null | Key | Default | Extra          |
+--------------+--------------+------+-----+---------+----------------+
| id           | int          | NO   | PRI | NULL    | auto_increment |
| author       | varchar(100) | NO   |     | NULL    |                |
| book_title   | varchar(200) | NO   |     | NULL    |                |
| room_id      | int          | NO   |     | NULL    |                |
| starts_at    | datetime     | NO   |     | NULL    |                |
| ticket_price | decimal(6,2) | YES  |     | 0.00    |                |
| sold_out     | tinyint(1)   | YES  |     | 0       |                |
+--------------+--------------+------+-----+---------+----------------+
```

Reading it: IF NOT EXISTS turned the second CREATE TABLE rooms into a Note (1050), and rooms kept its three columns. Both tables inherited `utf8mb4_unicode_ci` from the database. In DESCRIBE, `ticket_price` shows default 0.00 and `sold_out` (BOOLEAN) shows tinyint(1) with default 0. Accept `INT UNSIGNED` for seats and a `UNIQUE` on readings' author plus starts_at as good extras; `DATE` for starts_at loses the time of day: ask for DATETIME.

- LAB 2.1-B: LABS/MODULE-02/L2_1-CREATING-DATABASES-TABLES/CHANGE_REQUEST_STARTER.SQL → CHANGE_REQUEST_SOLUTION.SQL

The solution prints:

```
Database changed
+------------+--------------+------+-----+---------+----------------+
| Field      | Type         | Null | Key | Default | Extra          |
+------------+--------------+------+-----+---------+----------------+
| id         | int          | NO   | PRI | NULL    | auto_increment |
| email      | varchar(100) | NO   | UNI | NULL    |                |
| first_name | varchar(60)  | NO   |     | NULL    |                |
| city       | varchar(50)  | YES  |     | NULL    |                |
| joined_on  | date         | YES  |     | NULL    |                |
| source     | varchar(20)  | YES  |     | website |                |
+------------+--------------+------+-----+---------+----------------+
+----+------------------+------------+--------+------------+---------+
| id | email            | first_name | city   | joined_on  | source  |
+----+------------------+------------+--------+------------+---------+
|  1 | maya@example.com | Maya       | Denver | 2026-03-01 | website |
|  2 | omar@example.com | Omar       | NULL   | 2026-03-04 | website |
+----+------------------+------------+--------+------------+---------+
+----+-------------------+---------+
| id | email             | source  |
+----+-------------------+---------+
|  1 | again@example.com | website |
+----+-------------------+---------+
+--------------------------+
| Tables_in_kit_s2_signups |
+--------------------------+
| newsletter_signups       |
+--------------------------+
```

Reading it: after the four ALTERs both rows survived with their values, and the new `source` column shows `website` for both (a DEFAULT fills existing rows when the column is added). `first_name` is varchar(60) and still NO under Null because TODO 3 repeated NOT NULL; without it the column would silently accept NULL. TRUNCATE reset the counter, so Again got id 1. SHOW TABLES at the end lists only newsletter_signups: the copy was renamed and dropped.

- HANDOUT: LABS/MODULE-02/L2_1-CREATING-DATABASES-TABLES/DEBUG_BUILD_SCRIPT.SQL (DELIBERATELY BROKEN)

As shipped, MySQL prints:

```
ERROR 1007 (HY000): Can't create database 'kit_s2_shelf'; database exists
Database changed
ERROR 1046 (3D000): No database selected
Database changed
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresp
ERROR 1050 (42S01): Table 'shelves' already exists
ERROR 1146 (42S02): Table 'kit_s2_shelf.shelf' doesn't exist
```

1. **Bug A, 1007:** the database already exists. Fix: `CREATE DATABASE IF NOT EXISTS kit_s2_shelf;` (or delete the line: the database was just created).
2. **Bug B, 1046:** the session's current database (`kit_s2_scratch`) was dropped one line earlier, so there is no database selected. Fix: delete the stray CREATE TABLE (it belonged to the scratch database), or `USE kit_s2_shelf;` before it and drop the later duplicate.
3. **Bug C, 1064:** a comma after the last column; MySQL gets lost at the closing `)`. Fix: remove the comma after `capacity INT NOT NULL`.
4. **Bug D, 1050:** the table already exists, as it will whenever a script is rerun. Fix: `CREATE TABLE IF NOT EXISTS shelves`, or a `DROP TABLE IF EXISTS shelves;` before it.
5. **Bug E, 1146:** the table is `shelves`, plural (the message names `kit_s2_shelf.shelf`). Fix: `DESCRIBE shelves;`.

Fixed, the file prints DESCRIBE shelves with four columns (id, label UNI, genre, capacity) and no ERROR lines.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 1.3 · MYSQL CLI & PHPMYADMIN

1. **A** The statement has no semicolon yet, so the client is waiting for more: The client sends a statement only when it sees the terminator. The arrow prompt means it is waiting for the rest; type a semicolon.
2. **B** ERROR 1146: Table 'mysql.customers' doesn't exist: MySQL looks for the table in the current database, mysql, and reports the full name it tried: Table 'mysql.customers' doesn't exist.
3. **C** \G at the end of a query: Backslash commands such as \G are handled by the mysql client itself and never reach the server, so other tools do not understand them.
4. **B** ERROR 1064, because current_time is a reserved word: CURRENT_TIME is a reserved word (a built-in function), so it cannot be a plain alias. Rename it (now_time) or wrap it in backticks.
5. **A** End the statements with ; instead of \G (or run the client with --commands): In batch mode the 9.x client ignores its own commands. Ending statements with semicolons makes the file work everywhere; --commands is the alternative.

LESSON 2.1 · CREATING DATABASES & TABLES

1. **C** Nothing changes; MySQL reports a note and keeps the existing table: IF NOT EXISTS turns the error into a note (1050) and leaves the existing table and its rows exactly as they were. The new definition is ignored.
2. **A** SHOW CREATE TABLE books;: SHOW CREATE TABLE prints the exact CREATE TABLE statement that would rebuild the table, with every constraint, the engine and the character set and collation.

3. **B** title is VARCHAR(250) and now allows NULL: MODIFY replaces the whole column definition with exactly what you typed. NOT NULL was not repeated, so title now accepts NULL, without any warning.

4. **A** A table with one row, id 1: TRUNCATE removes every row and resets the AUTO_INCREMENT counter, so the next row gets id 1. The table itself, with its columns and keys, stays.

5. **C** MySQL's utf8 is a three-byte subset that cannot store every character: In MySQL, utf8 means utf8mb3, a three-byte subset that cannot store emoji and some rarer characters, and MySQL warns about it (warning 3719). utf8mb4 is full Unicode.

EXIT TICKET

1. Name two statements that work in phpMyAdmin's SQL tab and one command that only the mysql client understands. **Answer:** SQL (any client): `SHOW TABLES;`, `DESCRIBE books;`, `USE bookstore;`, `SELECT DATABASE();`, `CREATE TABLE ...`. Client only: `\G`, `SOURCE file.sql`, `\c`, `help`, `exit`, `status`.

2. A script stops with `ERROR 1046 (3D000): No database selected`. Give two ways to fix it. **Answer:** Put `USE bookstore;` (the right database) before the statement, or write the full name `bookstore.books`. It happens in a fresh session and after dropping the database you were in.

3. title is `VARCHAR(200) NOT NULL`. You run `ALTER TABLE books MODIFY COLUMN title VARCHAR(250);`. What did you lose, and which statement shows it? **Answer:** The NOT NULL rule: MODIFY replaces the whole definition. `DESCRIBE books;` (or `SHOW COLUMNS FROM books LIKE 'title';`) shows YES under Null. Fix: repeat it, `VARCHAR(250) NOT NULL`.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 B ("DESCRIBE does nothing in phpMyAdmin").** Thinks only SELECT is SQL. Show the Structure tab and the SQL tab running DESCRIBE.
- **Q2 A ("it only needs a semicolon").** Has not seen the quote prompt. Type `SELECT 'abc;` live: the semicolon becomes part of the text.
- **Q4 B (1045).** Reads every login failure as a password problem. The message says Unknown database: read the message.
- **Q7 B ("utf8 stores emoji").** The most expensive misconception of the day: a database created with utf8 loses emoji years later. Repeat: always utf8mb4.
- **Q9 B ("misspelled database").** Mixes 1046 with 1049. Write the pair on the board: 1049 = no database by that name; 1046 = no database chosen.
- **Q11 A ("still required").** Expects ALTER to keep what it is not told. The MODIFY trap from the demo; the exit ticket's question 3.

- **Exit ticket 2 answered with "create the database".** Needs the cabinet picture: the database exists, the session is just not inside it.

INSTRUCTOR KIT · ANSWER KEY

Session 3: Data Types, Constraints and CRUD

Answers and reasoning for every Session 3 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs, the lesson pages' own quizzes and the exit ticket.

Session 3 | 12 self-check items | Confidential

WARM-UP: SESSION 2

Not graded.

W1

The prompt shows `->`. What do you type?

A) `exit`

B) `;` and Enter

C) Ctrl+C

D) `USE bookstore;`

Why: -> means the statement is not finished: the client waits for the semicolon (Lesson 1.3).

W2

MySQL says `ERROR 1146 (42S02): Table 'mysql.books' doesn't exist`. What do you check first?

A) Reinstall MySQL

B) Whether books was dropped

C) Which database you are in: the message says mysql, not bookstore

D) The password

Why: The message names the database it looked in. `USE bookstore;` (or bookstore.books) fixes it.

W3

Which of these works in phpMyAdmin's SQL tab?

A) `DESCRIBE books;`

B) `SOURCE create_bookstore.sql`

C) `\G`

D) `exit`

> **Why:** DESCRIBE is SQL; the other three are mysql client commands.

W4

What does SHOW COLUMNS print under Null?

```
CREATE TABLE w4 (id INT PRIMARY KEY, title VARCHAR(200) NOT NULL);
ALTER TABLE w4 MODIFY COLUMN title VARCHAR(250);
SHOW COLUMNS FROM w4 LIKE 'title';
```

A) `NO`: NOT NULL is kept

B) `YES`**: MODIFY replaced the whole definition**

C) Nothing: ERROR 1064

D) `NO`, with a warning

> **Why:** MODIFY uses exactly the definition you typed: repeat NOT NULL. Session 2's exit ticket question 3. MySQL prints:
>
> ```
> +-------+--------------+------+-----+---------+-------+
> | Field | Type         | Null | Key | Default | Extra |
> +-------+--------------+------+-----+---------+-------+
> | title | varchar(250) | YES  |     | NULL    |       |
> +-------+--------------+------+-----+---------+-------+
> ```

W5

Right after `DROP DATABASE scratch;` (you were inside scratch), `CREATE TABLE t (id INT);` fails. Why?

A) The table exists (1050)

B) A syntax error (1064)

C) Permission denied

D) No database selected (1046): USE one, or write database.table

> **Why:** Dropping the current database leaves the session with none.

2.2 · DATA TYPES & CONSTRAINTS: SELF-CHECK

Q1

A column holds a discount as a whole percentage from 0 to 100. Which type is the smallest sensible choice?

A) `TINYINT UNSIGNED`

B) `INT`

C) `DECIMAL(5, 2)`

D) `VARCHAR(3)`

> **Why:** TINYINT UNSIGNED holds 0 to 255 in one byte: enough for 0 to 100, and it refuses negatives. Add a CHECK (percent BETWEEN 0 AND 100) to enforce the exact range.
> **B** is wrong: It works, but takes four bytes for a value that fits in one.
> **C** is wrong: There are no decimals to store.
> **D** is wrong: A number stored as text sorts and adds up wrongly.

Q2

What does the SELECT print?

```
CREATE TABLE q2 (amount DECIMAL(5, 2));
INSERT INTO q2 VALUES (7.005);
SELECT amount FROM q2;
```

`7.01`

> **Why:** A DECIMAL(5, 2) keeps two decimals, so 7.005 is rounded (half up) to 7.01, with a Note 1265. Rounding to the column's precision is not an error; a value too big for the column is (1264).

Q3

What happens when this runs?

```
CREATE TABLE q3 (due DATE);
INSERT INTO q3 VALUES ('2026-04-31');
```

A) 2026-05-01 is stored

B) 2026-04-30 is stored

C) `ERROR 1292 (22007): Incorrect date value: '2026-04-31' for column 'due' at row 1`

D) NULL is stored with a warning

> **Why:** April has 30 days. In strict mode (the default) MySQL checks that a date really exists and refuses the row.
> **A** is wrong: MySQL does not roll dates over.
> **B** is wrong: MySQL does not correct your value.
> **D** is wrong: That is what happens with strict mode switched off: a reason to keep it on.

Q4

A library stores the moment each long-term loan ends, some of them in 2045. Which type fits best?

A) `TIMESTAMP`

B) `DATE`

C) `YEAR`

D) `DATETIME`

> **Why:** A moment needs a date AND a time, and DATETIME runs to the year 9999. This course uses DATETIME everywhere.
> **A** is wrong: TIMESTAMP stops in January 2038 on MySQL (error 1292 for 2040 in the lesson).
> **B** is wrong: A DATE has no time of day.
> **C** is wrong: YEAR holds only a year (1901 to 2155).

Q5

email is `NOT NULL`. An INSERT gives it the value NULL explicitly. Which error?

A) `ERROR 1364`: Field 'email' doesn't have a default value

B) `ERROR 1048`: Column 'email' cannot be null

C) `ERROR 1062`: Duplicate entry

D) `ERROR 3819`: Check constraint is violated

> **Why:** Two flavors of NOT NULL: an explicit NULL gives 1048; leaving the column out of the INSERT when it has no default gives 1364.
> **A** is wrong: 1364 is the column left out entirely.
> **C** is wrong: 1062 is a duplicate in a UNIQUE column or primary key.
> **D** is wrong: 3819 is a CHECK constraint.

Q6

In which order does the SELECT list the three sizes?

```
CREATE TABLE q6 (size ENUM('small', 'medium', 'large'));
INSERT INTO q6 VALUES ('large'), ('small'), ('medium');
SELECT size FROM q6 ORDER BY size;
```

`small`, `medium`, `large`

> **Why:** ORDER BY on an ENUM sorts by the position in the list the column was declared with, not alphabetically (that would be large, medium, small). Handy here; surprising elsewhere.

2.2 · DATA TYPES & CONSTRAINTS: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
CREATE TABLE codes (code CHAR(6), label VARCHAR(6));
INSERT INTO codes VALUES ('ab  ', 'ab  ');
SELECT CHAR_LENGTH(code)  AS code_len,
       CHAR_LENGTH(label) AS label_len
FROM codes;
```

```
+----------+-----------+
| code_len | label_len |
+----------+-----------+
|        2 |         4 |
+----------+-----------+
```

> **Why:** Both received 'ab' plus two spaces. CHAR pads when storing and strips trailing spaces when reading (2); VARCHAR keeps them (4).

P2

Predict the output, then run it.

```
CREATE TABLE counts (n TINYINT UNSIGNED);
INSERT INTO counts VALUES (255);
INSERT INTO counts VALUES (256);
SELECT n FROM counts;
```

```
ERROR 1264 (22003): Out of range value for column 'n' at row 1
+------+
| n    |
+------+
|  255 |
+------+
```

Why: TINYINT UNSIGNED stops at 255: the first INSERT works, the second is refused with 1264, and the table holds one row.

2.3 · CRUD OPERATIONS: SELF-CHECK

Q7

Why should you avoid `INSERT INTO books VALUES (NULL, 'Emma', ...);` without a column list?

A) It is slower than listing the columns

B) NULL is not allowed in an id

C) It inserts the row twice

D) It depends on the table's exact column order: an added column breaks it (1136) or shifts values silently

Why: With a column list, the INSERT keeps working whatever columns are added later.

A is wrong: Speed is the same.

B is wrong: NULL in an AUTO_INCREMENT column means "give it the next number".

C is wrong: One VALUES list is one row.

Q8

What does the last SELECT print?

```
CREATE TABLE q8 (id INT AUTO_INCREMENT PRIMARY KEY, t VARCHAR(10));
INSERT INTO q8 (t) VALUES ('a');
INSERT INTO q8 (t) VALUES ('b'), ('c'), ('d');
SELECT LAST_INSERT_ID();
```

`2`

> **Why:** After a multi-row INSERT, LAST_INSERT_ID() returns the id of the FIRST row of that statement (b got 2, c 3, d 4).

Q9

An UPDATE reports `Rows matched: 0 Changed: 0 Warnings: 0`. What is most likely?

A) The WHERE found no rows: often a typo in the value or the wrong column

B) It worked: every row was changed

C) The rows already had those values

D) Safe-updates mode refused it

> **Why:** Rows matched counts what the WHERE found; Changed counts the rows that got a new value. Matched 0 means nothing was found.
>
> **B** is wrong: Changed 0 says nothing was changed.
>
> **C** is wrong: That is Rows matched 1 (or more), Changed 0.
>
> **D** is wrong: A refused statement prints ERROR 1175, not a row count.

Q10

After `SET SQL_SAFE_UPDATES = 1;`, which statement still runs? (id is the primary key.)

A) `UPDATE books SET price = 0;`

B) `DELETE FROM books WHERE price > 10;`

C) `UPDATE books SET price = 9.99 WHERE id = 2;`

D) `DELETE FROM books;`

> **Why:** Safe-updates refuses UPDATE and DELETE unless the WHERE uses a key column (or a LIMIT is given). The other three end with ERROR 1175.
> **A** is wrong: No WHERE at all: 1175.
> **B** is wrong: price is not a key column: 1175, even with a WHERE.
> **D** is wrong: No WHERE: 1175.

Q11

A category name already exists. Which statement updates its description and keeps its id and every other column?

A) `REPLACE INTO categories (name, description) VALUES (...);`

B) `INSERT ... AS new ON DUPLICATE KEY UPDATE description = new.description;`

C) `INSERT IGNORE INTO categories (name, description) VALUES (...);`

D) `DELETE` the old row, then `INSERT` the new one

> **Why:** The upsert changes only the columns you name in the UPDATE part and keeps the row (and its id).
> **A** is wrong: REPLACE deletes the old row and inserts a new one: a new id, and unlisted columns get their defaults.
> **C** is wrong: INSERT IGNORE skips the row: nothing is updated.
> **D** is wrong: That is what REPLACE does, with the same problems.

Q12

What does ROW_COUNT() print?

```
CREATE TABLE q12 (name VARCHAR(20) PRIMARY KEY, note VARCHAR(30));
INSERT INTO q12 VALUES ('a', 'x'), ('b', 'y');
INSERT INTO q12 VALUES ('a', 'x'), ('b', 'z'), ('c', 'w') AS new
ON DUPLICATE KEY UPDATE note = new.note;
SELECT ROW_COUNT() AS affected;
```

3

> **Why:** An upsert counts 1 per inserted row, 2 per updated row, 0 per row that already had those values: a unchanged (0) + b updated (2) + c inserted (1) = 3.

2.3 · CRUD OPERATIONS: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
CREATE TABLE notes (id INT AUTO_INCREMENT PRIMARY KEY, t VARCHAR(10));
INSERT INTO notes (t) VALUES ('a'), ('b'), ('c');
DELETE FROM notes WHERE id = 3;
INSERT INTO notes (t) VALUES ('d');
SELECT * FROM notes;
```

```
+----+------+
| id | t    |
+----+------+
|  1 | a    |
|  2 | b    |
|  4 | d    |
+----+------+
```

> **Why:** Id 3 is gone and is never handed out again, so the new row gets 4.

P4

Predict the output, then run it.

```
CREATE TABLE tags (name VARCHAR(20) PRIMARY KEY, uses INT NOT NULL);
INSERT INTO tags VALUES ('sale', 1);
INSERT INTO tags VALUES ('sale', 1) AS new
ON DUPLICATE KEY UPDATE uses = tags.uses + new.uses;
SELECT * FROM tags;
```

```
+------+------+
| name | uses |
+------+------+
| sale |    2 |
+------+------+
```

Why: The second INSERT finds 'sale' and adds the new uses to the old: 1 + 1 = 2. The column is written `tags.uses` on purpose: an unqualified `uses = uses + new.uses` is ERROR 1052 (Column 'uses' in field list is ambiguous) on MySQL 9.7, because the row alias `new` has a column named uses too.

LAB REFERENCE SOLUTIONS

- LAB 2.2-A: LABS/MODULE-03/L2_2-DATA-TYPES-CONSTRAINTS/TYPES_MATTER_STARTER.SQL → TYPES_MATTER_SOLUTION.SQL

As shipped, the starter prints (no error anywhere):

```
+-------+--------+
| title | copies |
+-------+--------+
| 1984  | 9      |
| Emma  | 15     |
| Dune  | 120    |
+-------+--------+
+--------------------+
| total_cost         |
+--------------------+
| 556.9499866962433 |
+--------------------+
+-------+------------+
| title | arrives    |
+-------+------------+
| 1984  | 01/15/2027 |
| Dune  | 03/10/2026 |
| Emma  | 12/01/2026 |
+-------+------------+
+---------------+
| last_delivery |
+---------------+
| 12/01/2026    |
+---------------+
```

The solution prints:

```
+-------+--------+
| title | copies |
+-------+--------+
| Dune  |    120 |
| Emma  |     15 |
| 1984  |      9 |
+-------+--------+
+------------+
| total_cost |
+------------+
|     556.95 |
+------------+
+-------+------------+
| title | arrives    |
+-------+------------+
| Dune  | 2026-03-10 |
| Emma  | 2026-12-01 |
| 1984  | 2027-01-15 |
+-------+------------+
+---------------+
| last_delivery |
+---------------+
| 2027-01-15    |
+---------------+
```

What each TODO teaches: (1) text sorts character by character, so '9' comes after '15' and '120'; (2) FLOAT stores binary fractions: 556.9499866962433 instead of 556.95, a crumb that grows with every order; (3) dates as text sort by the month digits, so `12/01/2026` looks like the latest; a DATE column refuses the US format (1292), and `STR_TO_DATE` converts it. Accept `SMALLINT UNSIGNED` for copies and any DECIMAL with 2 decimals for the cost.

- LAB 2.2-B: LABS/MODULE-03/L2_2-DATA-TYPES-CONSTRAINTS/MEMBERSHIP_RULES_STARTER.SQL → MEMBERSHIP_RULES_SOLUTION.SQL

The solution prints:

```
+----+------------------+------------+--------+--------+------------+
| id | email            | first_name | tier   | points | birthday   |
+----+------------------+------------+--------+--------+------------+
|  1 | ana@example.com  | Ana        | reader |      0 | NULL       |
|  2 | ben@example.com  | Ben        | reader |      0 | NULL       |
|  3 | cleo@example.com | Cleo       | patron |    250 | 1990-07-14 |
+----+------------------+------------+--------+--------+------------+
+-----------------+-----------------+
| CONSTRAINT_NAME | CONSTRAINT_TYPE |
+-----------------+-----------------+
| chk_points      | CHECK           |
| PRIMARY         | PRIMARY KEY     |
| email           | UNIQUE          |
+-----------------+-----------------+
+------------+-------------------------------------+------+-----+---------+-----------
| Field      | Type                                | Null | Key | Default | Extra
+------------+-------------------------------------+------+-----+---------+-----------
| id         | int                                 | NO   | PRI | NULL    | auto_incre
| email      | varchar(100)                        | NO   | UNI | NULL    |
| first_name | varchar(30)                         | NO   |     | NULL    |
| tier       | enum('reader','collector','patron') | NO   |     | reader  |
| points     | int                                 | NO   |     | 0       |
| birthday   | date                                | YES  |     | NULL    |
+------------+-------------------------------------+------+-----+---------+-----------
```

As shipped, Ana's and Ben's tier and points are NULL and the only constraint is PRIMARY. With the rules, the defaults fill them (reader, 0), SHOW COLUMNS says NO under Null for the four required columns, and information_schema lists the CHECK by the name chk_points. `points INT UNSIGNED NOT NULL DEFAULT 0` also keeps negatives out (with 1264 instead of 3819): accept it, and ask which error message is easier to read.

- HANDOUT: LABS/MODULE-03/L2_2-DATA-TYPES-CONSTRAINTS/DEBUG_BAD_ROWS.SQL (DELIBERATELY BROKEN)

As shipped, MySQL prints:

```
ERROR 1048 (23000): Column 'email' cannot be null
ERROR 1062 (23000): Duplicate entry 'ana@example.com' for key 'kit_members.email'
ERROR 1265 (01000): Data truncated for column 'tier' at row 1
ERROR 1292 (22007): Incorrect date value: '1995-02-29' for column 'birthday' at row 1
ERROR 1364 (HY000): Field 'first_name' doesn't have a default value
ERROR 3819 (HY000): Check constraint 'chk_points' is violated.
+----+-----------------+------------+-----------+--------+----------+
| id | email           | first_name | tier      | points | birthday |
+----+-----------------+------------+-----------+--------+----------+
|  1 | ana@example.com | Ana        | reader    |      0 | NULL     |
|  2 | ben@example.com | Ben        | collector |      0 | NULL     |
|  4 | ivy@example.com | Ivy        | reader    |     75 | NULL     |
+----+-----------------+------------+-----------+--------+----------+
```

1. **Bug A, 1048 (R1):** an explicit NULL email. Fix: a real email, `'dev@example.com'`.
2. **Bug B, 1062 (R1):** ana@example.com is taken. Fix: Anna's own email.
3. **Bug C, 1265 (R3):** gold is not one of the tiers. Fix: `'patron'` (or add gold to the ENUM, repeating the whole definition).
4. **Bug D, 1292 (R5's type):** 1995 was not a leap year. Fix: `'1995-02-28'` (or `'1996-02-29'`).
5. **Bug E, 1364 (R2):** first_name is required and has no default. Fix: add it.
6. **Bug F, 3819 (R4):** points may not be negative. Fix: 0 or a positive number.

Notice Ivy's id: 4, not 3. The refused duplicate (Bug B) had already reserved id 3. Fixed, the file lists nine members.

- LAB 2.3-A: LABS/MODULE-03/L2_3-CRUD-OPERATIONS/NEW_ARRIVALS_STARTER.SQL → NEW_ARRIVALS_SOLUTION.SQL

The solution prints:

```
+--------------+
| first_new_id |
+--------------+
|            1 |
+--------------+
+----+--------------+-------+
| id | title        | price |
+----+--------------+-------+
|  4 | Moby Dick    | 12.40 |
|  5 | Little Women | 10.20 |
+----+--------------+-------+
+--------------+
| rows_changed |
+--------------+
|            2 |
+--------------+
+--------------+-------+----------+
| title        | price | in_stock |
+--------------+-------+----------+
| Frankenstein |  6.99 |        0 |
+--------------+-------+----------+
+----+--------------+------------+
| id | title        | published  |
+----+--------------+------------+
|  1 | Emma         | 1815-12-23 |
|  2 | Frankenstein | 1818-01-01 |
+----+--------------+------------+
+--------------+
| rows_deleted |
+--------------+
|            2 |
+--------------+
+----+------------------+-------+----------+
| id | title            | price | in_stock |
+----+------------------+-------+----------+
|  3 | Dracula          |  9.75 |        1 |
|  4 | Moby Dick        | 13.02 |        1 |
|  5 | Little Women     | 10.71 |        1 |
|  6 | The Time Machine |  6.95 |        1 |
|  7 | Middlemarch      | 11.10 |        1 |
+----+------------------+-------+----------+
```

Reading it: LAST_INSERT_ID() is the FIRST id of a multi-row INSERT (1); the preview and the UPDATE use the same WHERE, and both find 2 rows; 12.40 × 1.05 = 13.02 and 10.20 × 1.05 = 10.71; Frankenstein is changed once more (6.99, out of stock) and then deleted with Emma; Middlemarch gets 7 because ids 1 and 2 are never reused.

- LAB 2.3-B: LABS/MODULE-03/L2_3-CRUD-OPERATIONS/CATEGORY_FEED_STARTER.SQL → CATEGORY_FEED_SOLUTION.SQL

The solution prints:

```
+----------+
| affected |
+----------+
|        3 |
+----------+
+----+---------+-----------------------------+
| id | name    | description                 |
+----+---------+-----------------------------+
|  1 | Fiction | Novels and stories          |
|  2 | Poetry  | Poems and verse collections |
|  3 | Travel  | Guides and journeys         |
|  4 | History | The past, told well         |
|  5 | Cooking | Recipes and kitchen craft   |
+----+---------+-----------------------------+
+---------+------+-------------------------------------------------------+
| Level   | Code | Message                                               |
+---------+------+-------------------------------------------------------+
| Warning | 1062 | Duplicate entry 'History' for key 'categories.name' |
+---------+------+-------------------------------------------------------+
+----+---------+-----------------------------+
| id | name    | description                 |
+----+---------+-----------------------------+
|  2 | Poetry  | Poems and verse collections |
|  3 | Travel  | Guides and journeys         |
|  4 | History | The past, told well         |
|  5 | Cooking | Recipes and kitchen craft   |
|  9 | Fiction | NULL                        |
+----+---------+-----------------------------+
```

Reading it: one statement handled three rows: Poetry updated (2), Travel unchanged (0), Cooking inserted (1) = 3. INSERT IGNORE stored nothing and turned the 1062 into a warning. REPLACE deleted Fiction (id 1) and inserted a new row: id 9 and a NULL description. Ids 6 to 8 were reserved by the upsert's two non-inserting rows and by INSERT IGNORE, and never used. On MariaDB, `AS new` is a syntax error: `VALUES(description)` there.

- HANDOUT: LABS/MODULE-03/L2_3-CRUD-OPERATIONS/DEBUG_CRUD_SLIPS.SQL (DELIBERATELY BROKEN)

As shipped, MySQL prints:

```
ERROR 1136 (21S01): Column count doesn't match value count at row 1
ERROR 1054 (42S22): Unknown column 'Rosa' in 'field list'
ERROR 1062 (23000): Duplicate entry 'nina@example.com' for key 'customers.email'
ERROR 1175 (HY000): You are using safe update mode and you tried to update a table witl
ERROR 1292 (22007): Truncated incorrect DOUBLE value: 'Chicago'
+----+------------+-----------+--------+-------------------+
| id | first_name | last_name | city   | email             |
+----+------------+-----------+--------+-------------------+
|  1 | Nina       | Park      | Denver | nina@example.com  |
|  2 | Omar       | Haddad    | Boston | omar@example.com  |
|  3 | Paula      | Reyes     | Denver | paula@example.com |
+----+------------+-----------+--------+-------------------+
```

1. **Bug A, 1136:** three columns, two values. Fix: add Quinn's email.
2. **Bug B, 1054:** `Rosa` without quotes is read as a column name. Fix: `'Rosa'`.
3. **Bug C, 1062:** nina@example.com is taken (email is UNIQUE). Fix: Nora's own email.
4. **Bug D, 1175:** with safe-updates on, `WHERE city = 'Denver'` is refused because city is not a key column. Fix: turn safe-updates off for a reviewed change, or name the rows by key: `WHERE id IN (1, 3)` (after a preview).
5. **Bug E, 1292:** AND instead of a comma in SET. With text values strict mode refuses it (`'Chicago'` cannot be read as a number); with numbers the same slip is silent (Lesson 2.3, Common Mistake 5). Fix: `SET city = 'Chicago', email = 'omar.h@example.com'`.

Fixed, the final SELECT shows five customers: Nina and Paula in Aurora, Omar in Chicago with omar.h@example.com, Quinn and Rosa.

NOTE ON LESSON 2.3'S BONUS CHALLENGE

The challenge's hint qualifies the old value: `copies = stock_counts.copies + new.copies`. The unqualified `copies = copies + new.copies` is ERROR 1052 on MySQL 9.7 (and 8.4), Column 'copies' in field list is ambiguous, because the row alias `new` has a copies column too. Verified: 5 copies of Dune twice gives 10. Learners who type it without the table name will hit 1052; that is the teaching moment.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 2.2 · DATA TYPES & CONSTRAINTS

1. **B** DECIMAL(8, 2): DECIMAL stores exact decimal values, so prices and totals are correct to the cent. DECIMAL(8, 2) holds up to 999999.99.
2. **C** Fails with error 1264, Out of range value: A signed TINYINT holds -128 to 127. In strict mode an out-of-range value is an error (1264) and the row is not stored.

3. **A** One with an email that another row already has: UNIQUE refuses a second row with the same value, with error 1062 Duplicate entry. A NULL would fail too, but with 1048, because of NOT NULL.

4. **D** A two-letter country code: CHAR suits values that always have the same length, such as a two-letter country code or a 13-digit ISBN.

5. **B** WHERE in_stock: BOOLEAN is a TINYINT, and MySQL treats any non-zero number as true. WHERE in_stock keeps 1, 5 and every other non-zero value.

LESSON 2.3 · CRUD OPERATIONS

1. **A** Every book's price becomes 0.00: Without a WHERE clause, UPDATE applies to every row. Only safe-updates mode (error 1175) would stop it, and it is off by default.

2. **C** The row was found but already had those values: The WHERE found one row, but it already had the new values, so there was nothing to change. Matched 0 would point at a WHERE that found nothing.

3. **B** ... AS new ON DUPLICATE KEY UPDATE description = new.description: The row alias form (AS new, then new.column) is the current MySQL syntax; VALUES() is deprecated. On MariaDB, which lacks the alias, VALUES() is still the way.

4. **C** 11: AUTO_INCREMENT only moves forward. The gap at 10 stays, and the new row gets the next number, 11. Only TRUNCATE resets the counter.

5. **A** Sets its price to 0.00 and leaves it in stock: SET reads price = (15.99 AND (in_stock = FALSE)). The book is in stock, so the comparison is false (0), and the price becomes 0.00. Separate assignments with commas.

EXIT TICKET

1. Why is money stored as DECIMAL and never as FLOAT? One sentence and one example. **Answer:** DECIMAL is exact; FLOAT stores binary fractions, so most decimal values are slightly off and the crumbs add up: 0.1 stored as FLOAT times 3 printed 0.30000000447034836 in the demo, and the lab's FLOAT total was 556.9499866962433 instead of 556.95.

2. An UPDATE says `Rows matched: 3 Changed: 2`. What happened to the third row? **Answer:** The WHERE found it, but it already had the new value, so nothing changed in it. (Matched 0 would mean the WHERE found nothing.)

3. What do you run before every UPDATE or DELETE that touches more than one row, and why? **Answer:** A SELECT with exactly the same WHERE: it shows which rows will be touched, and its row count must match the UPDATE's Rows matched. Safe-updates mode (`SET SQL_SAFE_UPDATES = 1;` or `mysql --safe-updates`) is the second net.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 B (INT).** Correct but not thoughtful: ask for the smallest type that comfortably fits.
- **Q3 A or B.** Expects MySQL to repair a date. Strict mode refuses; that is the point.
- **Q4 A (TIMESTAMP).** Has not met the 2038 limit: show the lesson's 2040 error.
- **Q5 A (1364).** Mixes the two NOT NULL errors: explicit NULL 1048, left out 1364.
- **Q9 C.** Mixes Matched 0 with Changed 0. Write both lines on the board.
- **Q10 B.** Thinks any WHERE satisfies safe-updates; it wants a key column.
- **Q11 A (REPLACE).** The word sounds right. REPLACE is delete plus insert: a new id.

INSTRUCTOR KIT · ANSWER KEY

Session 4: Filtering, Sorting and Limiting

Answers and reasoning for every Session 4 item: the warm-up, both self-checks, the predict-then-run items, Checkpoint 1 (rubric, model answer, Part D), the lab reference outputs, the lesson pages' own quizzes and the exit ticket.

Session 4 | 12 self-check items | Checkpoint 1 | Confidential

WARM-UP: SESSION 3

Not graded.

W1

Which type stores a book's price?

A) `FLOAT`

B) `DECIMAL(8, 2)`

C) `VARCHAR(10)`

D) `INT`

Why: Money is exact: DECIMAL. FLOAT's binary fractions add up to crumbs (the types lab: 556.9499866962433).

W2

An INSERT gives the NOT NULL column email the value NULL. Which error?

A) `1364`

B) `1062`

C) `1048`

D) `3819`

Why: 1048 for an explicit NULL; 1364 when the column is left out.

W3

What does the second line say after this UPDATE?

```
CREATE TABLE w3 (id INT PRIMARY KEY, price DECIMAL(6, 2));
INSERT INTO w3 VALUES (1, 9.99);
UPDATE w3 SET price = 9.99 WHERE id = 1;
```

A) `Rows matched: 0 Changed: 0`

B) `Rows matched: 1 Changed: 1`

C) `ERROR 1062`

D) `Rows matched: 1 Changed: 0`

Why: The row was found, but already had that value. MySQL prints:

```
Query OK, 0 rows affected
Rows matched: 1  Changed: 0  Warnings: 0
```

W4

What do you run right before an UPDATE that touches several rows?

A) A SELECT with exactly the same WHERE

B) `SHOW TABLES;`

C) `SET SQL_SAFE_UPDATES = 0;`

D) Nothing: UPDATE can be undone

Why: The preview: its row count must match Rows matched.

W5

Which statement gives an existing category a NEW id?

A) `INSERT ... AS new ON DUPLICATE KEY UPDATE ...`

B) `INSERT IGNORE ...`

C) `REPLACE INTO ...`

D) `UPDATE ... WHERE id = 3`

> **Why:** REPLACE deletes and re-inserts; the upsert keeps the row.

3.1 · WHERE CLAUSES, OPERATORS & FILTERING: SELF-CHECK

Q1

Two of the 8 customers live in Chicago and two have no city. How many rows does this return?

```
SELECT * FROM customers WHERE city != 'Chicago';
```

A) 6

B) 4

C) 8

D) 0

> **Why:** != drops the two Chicago rows AND the two NULL cities: "is an unknown city different from Chicago?" is unknown, not true. Add `OR city IS NULL` to keep them (6).
>
> **A** is wrong: 6 is what you get once you add OR city IS NULL.
>
> **C** is wrong: Chicago is excluded on purpose.
>
> **D** is wrong: Only city = NULL (or a NULL inside NOT IN) finds nothing.

Q2

On the Module 3 data, what does this print?

```
SELECT COUNT(*) AS n
FROM books
WHERE price BETWEEN 8.99 AND 9.99;
```

`4`

Why: BETWEEN includes both ends: The Great Gatsby and The Catcher in the Rye (8.99), Brave New World (9.49) and 1984 (9.99).

Q3

Which WHERE keeps every book NOT written by Tolkien or Orwell?

A) `WHERE author NOT IN ('J.R.R. Tolkien', 'George Orwell')`

B) `WHERE author != 'J.R.R. Tolkien' OR author != 'George Orwell'`

C) `WHERE NOT author = 'J.R.R. Tolkien' OR 'George Orwell'`

D) `WHERE author NOT LIKE 'Tolkien, Orwell'`

Why: NOT IN removes both names: 7 of the 10 books remain (Tolkien and Orwell have three between them).
B is wrong: Every author differs from at least one of the two names, so the condition is always true: all 10 books.
C is wrong: OR 'George Orwell' is a separate test on a piece of text, not on author.
D is wrong: That compares with one literal string, which no author matches.

Q4

created_at is a DATETIME. Which WHERE finds the customers who signed up on January 27, 2026 (Frank, at 13:30)?

A) `WHERE created_at = '2026-01-27'`

B) `WHERE created_at BETWEEN '2026-01-27' AND '2026-01-27'`

C) `WHERE created_at >= '2026-01-27' AND created_at < '2026-01-28'`

D) `WHERE created_at LIKE 2026-01-27`

> **Why:** A half-open range: on or after midnight of the day, and before midnight of the next. It never misses an evening row.
> **A** is wrong: '2026-01-27' means midnight exactly: 0 rows (measured).
> **B** is wrong: The same single instant, midnight: 0 rows.
> **D** is wrong: Without quotes 2026-01-27 is arithmetic (1998), and LIKE on a date is the wrong tool.

Q5

Which first names does this return, in order?

```
SELECT first_name
FROM customers
WHERE city = 'New York' OR city = 'Chicago' AND first_name LIKE 'G%'
ORDER BY first_name;
```

`Alice`, `Frank`, `Grace`

> **Why:** AND runs before OR: city = 'New York' OR (city = 'Chicago' AND first_name LIKE 'G%'). Both New Yorkers pass, and of the Chicago customers only Grace. Written with parentheses, (New York OR Chicago) AND LIKE 'G%', it would return only Grace.

Q6

Which of these returns no rows at all?

A) `WHERE city IS NULL`

B) `WHERE city <=> NULL`

C) `WHERE city LIKE '%o%'`

D) `WHERE city NOT IN ('Reno', NULL)`

> **Why:** NOT IN ('Reno', NULL) means city != 'Reno' AND city != NULL; the second part is unknown for every row, so nothing passes (COUNT 0 on the course data).
> **A** is wrong: IS NULL finds Eve and Hank.
> **B** is wrong: <=> is NULL-safe: it works like IS NULL here.
> **C** is wrong: New York, Chicago, Los Angeles and Houston all contain an o.

3.1 · WHERE CLAUSES, OPERATORS & FILTERING: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
SELECT COUNT(*) AS n
FROM customers
WHERE city NOT IN ('Chicago', 'Houston');
```

```
+---+
| n |
+---+
| 3 |
+---+
```

> **Why:** Chicago (2) and Houston (1) are left out, and so are Eve and Hank: NULL NOT IN (...) is unknown. Alice, Carol and Frank remain: 3.

P2

Predict the output, then run it.

```
SELECT title
FROM books
WHERE title LIKE '%o%o%'
ORDER BY title;
```

```
+-----------------------+
| title                 |
+-----------------------+
| Lord of the Rings     |
| To Kill a Mockingbird |
+-----------------------+
```

Why: `%o%o%` needs two o's anywhere: Lord of the Rings (Lord, of) and To Kill a Mockingbird (To, Mockingbird).

3.2 · SORTING & LIMITING RESULTS: SELF-CHECK

Q7

What does `ORDER BY in_stock, price DESC` do?

A) in_stock ascending (0 first), then price descending within each group

B) Both columns descending

C) `ERROR 1064`: DESC must come after every column

D) price first, then in_stock

Why: ASC and DESC belong to the one column right before them; ASC is the default. For both descending write `in_stock DESC, price DESC`.

B is wrong: DESC applies only to price.

C is wrong: It is valid SQL.

D is wrong: The first column listed is the first sort key.

Q8

What does this print?

```
SELECT title
FROM books
ORDER BY pages DESC
LIMIT 1 OFFSET 2;
```

`Dune` (412 pages: the third longest)

> **Why:** Longest first: Lord of the Rings 1178, Pride and Prejudice 432, Dune 412. OFFSET 2 skips two rows, LIMIT 1 keeps the next.

Q9

What happens with `'price'` in quotes: `ORDER BY 'price' DESC LIMIT 3`?

A) The three most expensive books, as intended

B) `ERROR 1054`: Unknown column

C) No error, but the rows come in no particular order: 'price' is a fixed piece of text

D) The rows are sorted alphabetically by title

> **Why:** In single quotes, price is text, the same for every row, so there is nothing to sort by. Column names are written bare (or in backticks).
> **A** is wrong: On the test server it returned Dune, 1984, To Kill a Mockingbird: not the top three.
> **B** is wrong: A text value is valid, so there is no error: the danger.
> **D** is wrong: Nothing sorts by title here.

Q10

Where do Eve and Hank (no city) appear with `ORDER BY city DESC, first_name`?

A) First

B) They are left out

C) `ERROR 1064`

D) Last

> **Why:** MySQL treats NULL as smaller than any value: first in an ascending sort, last in a descending one. To force them last in an ascending sort: `ORDER BY city IS NULL, city`.
> **A** is wrong: That is the ascending sort.
> **B** is wrong: ORDER BY never removes rows.
> **C** is wrong: Valid SQL.

Q11

A list sorted by id shows 20 rows per page, and page 1 ended at id 57. Which query gets the next page fast, even on a huge table?

A) `... ORDER BY id LIMIT 20 OFFSET 57`

B) `... WHERE id > 57 ORDER BY id LIMIT 20`

C) `... WHERE id >= 57 LIMIT 20`

D) `... ORDER BY id LIMIT 57, 20`

> **Why:** Keyset pagination: remember the last row seen and ask for the rows after it. With the primary key's index MySQL jumps straight to id 58.
> **A** is wrong: OFFSET counts rows, not ids, and reads and discards them all: slower with every page.
> **C** is wrong: It repeats id 57 and has no ORDER BY.
> **D** is wrong: The comma form is OFFSET 57: the same row-counting problem.

Q12

With 3 books per page, how many pages? What does this print?

```
SELECT CEILING(COUNT(*) / 3) AS pages FROM books;
```

`4`

> **Why:** 10 books / 3 = 3.33, and CEILING rounds up: the last page holds one book.

3.2 · SORTING & LIMITING RESULTS: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
SELECT first_name, city
FROM customers
ORDER BY city DESC, first_name
LIMIT 3;
```

```
+------------+-------------+
| first_name | city        |
+------------+-------------+
| Alice      | New York    |
| Frank      | New York    |
| Carol      | Los Angeles |
+------------+-------------+
```

Why: Descending puts New York first and NULLs last; first_name breaks the New York tie: Alice, Frank, then Carol (Los Angeles).

P4

Predict the output, then run it.

```
SELECT title, pages
FROM books
ORDER BY pages
LIMIT 2 OFFSET 8;
```

```
+---------------------+-------+
| title               | pages |
+---------------------+-------+
| Pride and Prejudice |   432 |
| Lord of the Rings   |  1178 |
+---------------------+-------+
```

Why: Shortest first, skip 8 of the 10: the two longest are left, Pride and Prejudice (432) and Lord of the Rings (1178).

CHECKPOINT 1: RUBRIC AND MODEL ANSWER

Five rows, 0 to 2 points each (10 in all). Score from the learner's file (run it once) and the Part D page. 8 to 10: on track. 5 to 7: review the weakest row before Session 6. Under 5: a one-to-one before Lesson 4.1.

CRITERION	STRONG (2)	PARTIAL (1)	MISSING (0)
Script hygiene (Modules 1 and 2)	Runs twice in a row: DROP DATABASE IF EXISTS, utf8mb4 + a named collation, USE; a comment per table; every INSERT has a column list	Runs once but not twice, or no collation, or one INSERT without a column list	Does not run, or tables created in the wrong database (1046 or bookstore)
Types (2.1, 2.2)	DECIMAL for the offer, DATE for received, ENUM for the condition, BOOLEAN for paid, sensible VARCHAR sizes; book_condition (not the reserved word condition)	One wrong type (FLOAT money, a date as text, INT for the condition)	Several wrong types, or everything VARCHAR
Constraints (2.2)	AUTO_INCREMENT primary keys; NOT NULL where the spec says required; UNIQUE email; CHECK chk_offer for 0.50 to 20.00; DEFAULT 'good' and DEFAULT FALSE	Two or three rules missing (often the CHECK or a DEFAULT), or the CHECK unnamed	Primary keys only, or no rules
Changing data safely (2.3)	B3 and B5 by id; B4 preview with the same WHERE and rows_paid 2; B6 one upsert (AS new ON DUPLICATE KEY UPDATE) that leaves Ivy as id 2 in Henderson	Results right but no preview, or B6 done with UPDATE (works, but not the asked statement) or REPLACE (new id)	A WHERE that changes the wrong rows, or an UPDATE/DELETE without WHERE
Reading MySQL's answers (Part C, Part D)	C1 to C3 match the model answer; Part D names 3819, 1062 and 1265 with the rule each one protects and a fix	Part C right, two of the three error numbers	Part C wrong, or Part D blank

MODEL ANSWER: LABS/MODULE-04/CHECKPOINT-1/CHECKPOINT_SOLUTION.SQL

```sql
-- ---------- Part A · build (rerunnable) ----------
DROP DATABASE IF EXISTS kit_cp1_tradein;
CREATE DATABASE kit_cp1_tradein
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;
USE kit_cp1_tradein;

-- People who bring books in. email is how the desk finds them again.
CREATE TABLE sellers (
    id     INT AUTO_INCREMENT PRIMARY KEY,
    name   VARCHAR(80)  NOT NULL,
    email  VARCHAR(100) NOT NULL UNIQUE,
    city   VARCHAR(50)
);

-- One row per book brought in. seller_id says whose it is (Module 4 makes it a foreign
CREATE TABLE trade_ins (
    id              INT AUTO_INCREMENT PRIMARY KEY,
    seller_id       INT NOT NULL,
    title           VARCHAR(200) NOT NULL,
    book_condition  ENUM('like new', 'good', 'worn') NOT NULL DEFAULT 'good',
    offer           DECIMAL(5, 2) NOT NULL,
    received        DATE NOT NULL,
    paid            BOOLEAN NOT NULL DEFAULT FALSE,
    CONSTRAINT chk_offer CHECK (offer BETWEEN 0.50 AND 20.00)
);

-- ---------- Part B · fill and change ----------
-- B1 · three sellers
INSERT INTO sellers (name, email, city) VALUES
    ('Mo Rahman', 'mo@example.com', 'Reno'),
    ('Ivy Chen', 'ivy@example.com', NULL),
    ('Sam Okoro', 'sam@example.com', 'Sparks');

-- B2 · five trade-ins (paid is left to its default)
INSERT INTO trade_ins (seller_id, title, book_condition, offer, received) VALUES
    (1, 'Dune', 'good', 4.00, '2026-05-01'),
    (1, 'Emma', 'worn', 1.50, '2026-05-01'),
    (2, 'The Hobbit', 'like new', 6.00, '2026-05-03'),
    (3, '1984', 'good', 3.50, '2026-05-04'),
    (3, 'Dracula', 'worn', 1.00, '2026-05-04');

-- B3 · Dune's offer goes up by one dollar (it is id 1)
UPDATE trade_ins SET offer = offer + 1.00 WHERE id = 1;

-- B4 · Mo is paid for all his books: preview, then the same WHERE
SELECT id, title, paid FROM trade_ins WHERE seller_id = 1 ORDER BY id;
```

```
UPDATE trade_ins SET paid = TRUE WHERE seller_id = 1;
SELECT ROW_COUNT() AS rows_paid;

-- B5 · Dracula (id 5) was turned down
DELETE FROM trade_ins WHERE id = 5;

-- B6 · Ivy sends her details again, now with a city: insert her, or update her
INSERT INTO sellers (name, email, city)
VALUES ('Ivy Chen', 'ivy@example.com', 'Henderson') AS new
ON DUPLICATE KEY UPDATE city = new.city;

-- ---------- Part C · read ----------
-- C1 · every trade-in, with 10% extra if taken as store credit
SELECT id, title, book_condition, offer,
       ROUND(offer * 1.10, 2) AS store_credit, paid
FROM trade_ins
ORDER BY id;

-- C2 · the conditions that occur, each once
SELECT DISTINCT book_condition FROM trade_ins ORDER BY book_condition;

-- C3 · the sellers now
SELECT id, name, city FROM sellers ORDER BY id;

DROP DATABASE kit_cp1_tradein;
```

It prints:

```
Database changed
+----+-------+------+
| id | title | paid |
+----+-------+------+
|  1 | Dune  |    0 |
|  2 | Emma  |    0 |
+----+-------+------+
+-----------+
| rows_paid |
+-----------+
|         2 |
+-----------+
+----+------------+----------------+-------+--------------+------+
| id | title      | book_condition | offer | store_credit | paid |
+----+------------+----------------+-------+--------------+------+
|  1 | Dune       | good           |  5.00 |         5.50 |    1 |
|  2 | Emma       | worn           |  1.50 |         1.65 |    1 |
|  3 | The Hobbit | like new       |  6.00 |         6.60 |    0 |
|  4 | 1984       | good           |  3.50 |         3.85 |    0 |
+----+------------+----------------+-------+--------------+------+
+----------------+
| book_condition |
+----------------+
| like new       |
| good           |
| worn           |
+----------------+
+----+-----------+-----------+
| id | name      | city      |
+----+-----------+-----------+
|  1 | Mo Rahman | Reno      |
|  2 | Ivy Chen  | Henderson |
|  3 | Sam Okoro | Sparks    |
+----+-----------+-----------+
```

PART D ANSWERS (EACH RUN AFTER THE MODEL ANSWER)

D1

The desk offers 25.00 for a rare copy of Emma.

```
INSERT INTO trade_ins (seller_id, title, offer, received)
VALUES (1, 'Emma', 25.00, '2026-05-02');
```

```
ERROR 3819 (HY000): Check constraint 'chk_offer' is violated.
```

Why: The CHECK chk_offer keeps offers from 0.50 to 20.00 (the desk's rule). Fix: an offer within the range, or a manager changes the rule with ALTER TABLE.

D2

Mo comes back with a new name on his card, same email.

```
INSERT INTO sellers (name, email)
VALUES ('Mo R.', 'mo@example.com');
```

```
ERROR 1062 (23000): Duplicate entry 'mo@example.com' for key 'sellers.email'
```

Why: email is UNIQUE: one seller per email, so the desk finds people again. Fix: UPDATE Mo's name by id, or the B6 upsert.

D3

Ivy's Emma is in "mint" condition.

```
INSERT INTO trade_ins (seller_id, title, book_condition, offer, received)
VALUES (2, 'Emma', 'mint', 3.00, '2026-05-02');
```

```
ERROR 1265 (01000): Data truncated for column 'book_condition' at row 1
```

Why: mint is not on the ENUM list; 1265 "Data truncated" is how MySQL says so. Fix: 'like new', or add the value by redefining the whole column.

SCORING NOTES

- **Accept** `DECIMAL(4, 2)` or wider for the offer, `TINYINT(1)` for paid, `VARCHAR` sizes close to the spec, and B4 without the ROW_COUNT line if the preview is there.

- **B6 by UPDATE** gives the right table but not the asked statement: 1 on row 4. **B6 by REPLACE** gives Ivy a new id (4) and is a 1 too, with a note on why.
- **A file that only runs once** (no DROP DATABASE IF EXISTS, or CREATE without IF NOT EXISTS after an edit) loses the hygiene point it most needs.
- **Column named `condition`:** a 1064 they should have fixed with the hint; if the rest runs, score the rest.

LAB REFERENCE SOLUTIONS

- LAB 3.1-A: LABS/MODULE-04/L3_1-WHERE-CLAUSES-OPERATORS-FILTERING/STAFF_REQUESTS_STARTER.SQL → STAFF_REQUESTS_SOLUTION.SQL

The solution prints:

```
+----------------------+---------------+
| title                | author        |
+----------------------+---------------+
| 1984                 | George Orwell |
| Fahrenheit 451       | Ray Bradbury  |
| To Kill a Mockingbird | Harper Lee    |
+----------------------+---------------+
+-----------------+-------+------------+
| title           | price | published  |
+-----------------+-------+------------+
| Brave New World |  9.49 | 1932-01-01 |
| The Hobbit      | 10.99 | 1937-09-21 |
| 1984            |  9.99 | 1949-06-08 |
+-----------------+-------+------------+
+------------+-----------+
| first_name | last_name |
+------------+-----------+
| Alice      | Johnson   |
| Grace      | Wilson    |
+------------+-----------+
+-------+
| title |
+-------+
| 1984  |
| Dune  |
+-------+
+------------+-------------+
| first_name | city        |
+------------+-------------+
| Bob        | Chicago     |
| Carol      | Los Angeles |
| Dave       | Houston     |
| Eve        | NULL        |
| Grace      | Chicago     |
| Hank       | NULL        |
+------------+-------------+
+-----------------+---------------+-------+
| title           | author        | price |
+-----------------+---------------+-------+
| Brave New World | Aldous Huxley |  9.49 |
| 1984            | George Orwell |  9.99 |
| Fahrenheit 451  | Ray Bradbury  | 10.49 |
+-----------------+---------------+-------+
+------------+---------------------+
| first_name | created_at          |
+------------+---------------------+
| Grace      | 2026-02-02 08:55:00 |
```

```
| Hank        | 2026-02-10 19:12:00 |
+-------------+---------------------+
```

Request 5 is the one most learners get wrong first: `city != 'New York'` alone returns 4 rows; Eve and Hank need `OR city IS NULL`. Request 7 uses a half-open range; `BETWEEN '2026-02-01' AND '2026-02-14'` would miss anyone who signed up on the 14th after midnight.

- LAB 3.1-B: LABS/MODULE-04/L3_1-WHERE-CLAUSES-OPERATORS-FILTERING/NULL_DETECTIVE_STARTER.SQL → NULL_DETECTIVE_SOLUTION.SQL

The solution prints:

```
+------------+-------------+------------+-------------+---------+
| first_name | city        | is_chicago | not_chicago | no_city |
+------------+-------------+------------+-------------+---------+
| Alice      | New York    |          0 |           1 |       0 |
| Bob        | Chicago     |          1 |           0 |       0 |
| Carol      | Los Angeles |          0 |           1 |       0 |
| Dave       | Houston     |          0 |           1 |       0 |
| Eve        | NULL        |       NULL |        NULL |       1 |
| Frank      | New York    |          0 |           1 |       0 |
| Grace      | Chicago     |          1 |           0 |       0 |
| Hank       | NULL        |       NULL |        NULL |       1 |
+------------+-------------+------------+-------------+---------+
+------------+
| in_chicago |
+------------+
|          2 |
+------------+
+------------+
| elsewhere  |
+------------+
|          4 |
+------------+
+---------+
| no_city |
+---------+
|       2 |
+---------+
+------------+-------------+
| first_name | city        |
+------------+-------------+
| Alice      | New York    |
| Bob        | Chicago     |
| Carol      | Los Angeles |
| Eve        | NULL        |
| Frank      | New York    |
| Grace      | Chicago     |
| Hank       | NULL        |
+------------+-------------+
+------------+------+
| first_name | city |
+------------+------+
| Eve        | NULL |
| Hank       | NULL |
+------------+------+
```

As shipped, `no_city` is 0 and queries 3 and 4 return nothing: the NULL inside NOT IN makes every row unknown, and `= NULL` is never true. With IS NULL the counts add to 8.

- HANDOUT: LABS/MODULE-04/L3_1-WHERE-CLAUSES-OPERATORS-FILTERING/DEBUG_FILTERS.SQL (DELIBERATELY BROKEN)

As shipped, MySQL prints:

```
ERROR 1054 (42S22): Unknown column 'Chicago' in 'where clause'
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresp
Empty set
Empty set
+------------------------+---------------+----------+
| title                  | author        | in_stock |
+------------------------+---------------+----------+
| Pride and Prejudice    | Jane Austen   |        1 |
| The Catcher in the Rye | J.D. Salinger |        0 |
+------------------------+---------------+----------+
```

1. **Bug A, 1054:** `Chicago` without quotes is read as a column. Fix: `'Chicago'`.
2. **Bug B, 1064:** each comparison needs its column: `pages > 300 AND pages < 500` (or `BETWEEN 301 AND 499`).
3. **Bug C, silent:** `= NULL` finds nothing. Fix: `IS NULL` (Eve, Hank).
4. **Bug D, silent:** `=` takes the % literally. Fix: `LIKE 'The %'` (three titles).
5. **Bug E, silent:** AND before OR lets the out-of-stock Catcher in. Fix: `in_stock = TRUE AND (author = 'Jane Austen' OR author = 'J.D. Salinger')` (only Pride and Prejudice).

- LAB 3.2-A: LABS/MODULE-04/L3_2-SORTING-LIMITING-RESULTS/BESTSELLER_BOARD_STARTER.SQL → BESTSELLER_BOARD_SOLUTION.SQL

The solution prints:

```
+------------+-----------+---------------------+
| first_name | last_name | created_at          |
+------------+-----------+---------------------+
| Hank       | Taylor    | 2026-02-10 19:12:00 |
| Grace      | Wilson    | 2026-02-02 08:55:00 |
| Frank      | Miller    | 2026-01-27 13:30:00 |
| Eve        | Davis     | 2026-01-20 11:08:00 |
| Dave       | Brown     | 2026-01-12 16:25:00 |
| Carol      | Williams  | 2026-01-09 09:40:00 |
| Bob        | Smith     | 2026-01-06 14:02:00 |
| Alice      | Johnson   | 2026-01-04 10:15:00 |
+------------+-----------+---------------------+
+------------------------+--------------+
| title                  | title_length |
+------------------------+--------------+
| The Catcher in the Rye |           22 |
| To Kill a Mockingbird  |           21 |
| Pride and Prejudice    |           19 |
| Lord of the Rings      |           17 |
| The Great Gatsby       |           16 |
| Brave New World        |           15 |
| Fahrenheit 451         |           14 |
| The Hobbit             |           10 |
| 1984                   |            4 |
| Dune                   |            4 |
+------------------------+--------------+
+---------------------+------------------------+------------+
| author              | title                  | published  |
+---------------------+------------------------+------------+
| Aldous Huxley       | Brave New World        | 1932-01-01 |
| F. Scott Fitzgerald | The Great Gatsby       | 1925-04-10 |
| Frank Herbert       | Dune                   | 1965-08-01 |
| George Orwell       | 1984                   | 1949-06-08 |
| Harper Lee          | To Kill a Mockingbird  | 1960-07-11 |
| J.D. Salinger       | The Catcher in the Rye | 1951-07-16 |
| J.R.R. Tolkien      | Lord of the Rings      | 1954-07-29 |
| J.R.R. Tolkien      | The Hobbit             | 1937-09-21 |
| Jane Austen         | Pride and Prejudice    | 1813-01-28 |
| Ray Bradbury        | Fahrenheit 451         | 1953-10-19 |
+---------------------+------------------------+------------+
+------------------------+-------+
| title                  | pages |
+------------------------+-------+
| Fahrenheit 451         |   158 |
| To Kill a Mockingbird  |   281 |
| The Hobbit             |   310 |
+------------------------+-------+
+-----------------+
```

```
| name            |
+-----------------+
| Classic         |
| Fiction         |
| Dystopian       |
| Fantasy         |
| Science Fiction |
+-----------------+
+------------+-----------+-------------+
| first_name | last_name | city        |
+------------+-----------+-------------+
| Eve        | Davis     | NULL        |
| Hank       | Taylor    | NULL        |
| Alice      | Johnson   | New York    |
| Frank      | Miller    | New York    |
| Carol      | Williams  | Los Angeles |
| Dave       | Brown     | Houston     |
| Bob        | Smith     | Chicago     |
| Grace      | Wilson    | Chicago     |
+------------+-----------+-------------+
```

List 2 needs the tie-breaker: 1984 and Dune both have 4 characters. List 6: `city IS NOT NULL` is 0 for the missing cities, so ascending puts them first; then `city DESC`.

- LAB 3.2-B: LABS/MODULE-04/L3_2-SORTING-LIMITING-RESULTS/PAGING_STARTER.SQL → PAGING_SOLUTION.SQL

The solution prints:

```
+----+------------------------+-------+
| id | title                  | price |
+----+------------------------+-------+
|  5 | Pride and Prejudice    |  7.99 |
|  4 | The Great Gatsby       |  8.99 |
|  8 | The Catcher in the Rye |  8.99 |
|  7 | Brave New World        |  9.49 |
+----+------------------------+-------+
+----+-----------------------+-------+
| id | title                 | price |
+----+-----------------------+-------+
|  2 | 1984                  |  9.99 |
| 10 | Fahrenheit 451        | 10.49 |
|  6 | The Hobbit            | 10.99 |
|  3 | To Kill a Mockingbird | 11.99 |
+----+-----------------------+-------+
+----+-------------------+-------+
| id | title             | price |
+----+-------------------+-------+
|  1 | Dune              | 14.99 |
|  9 | Lord of the Rings | 15.99 |
+----+-------------------+-------+
+-------+-------+
| books | pages |
+-------+-------+
|    10 |     3 |
+-------+-------+
Empty set
+------------------------+
| note                   |
+------------------------+
| page 4 printed nothing |
+------------------------+
+----+-----------------------+-------+
| id | title                 | price |
+----+-----------------------+-------+
|  2 | 1984                  |  9.99 |
| 10 | Fahrenheit 451        | 10.49 |
|  6 | The Hobbit            | 10.99 |
|  3 | To Kill a Mockingbird | 11.99 |
+----+-----------------------+-------+
```

Two books cost 8.99, so id breaks the tie and the pages never repeat or skip a book. Page 4 (OFFSET 12) is not an error: it prints nothing (in the interactive client, `Empty set`). The keyset query returns exactly page 2.

- HANDOUT: LABS/MODULE-04/L3_2-SORTING-LIMITING-RESULTS/DEBUG_SORTING.SQL (DELIBERATELY BROKEN)

As shipped, MySQL prints:

```
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresp
ERROR 1054 (42S22): Unknown column '3' in 'order clause'
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresp
+----------------------+-------+
| title                | pages |
+----------------------+-------+
| Dune                 |   412 |
| 1984                 |   328 |
| To Kill a Mockingbird |   281 |
+----------------------+-------+
+------------------------+----------+-------+
| title                  | in_stock | pages |
+------------------------+----------+-------+
| The Catcher in the Rye |        0 |   234 |
| The Great Gatsby       |        0 |   180 |
| Lord of the Rings      |        1 |  1178 |
| Pride and Prejudice    |        1 |   432 |
| Dune                   |        1 |   412 |
| 1984                   |        1 |   328 |
| Brave New World        |        1 |   311 |
| The Hobbit             |        1 |   310 |
| To Kill a Mockingbird  |        1 |   281 |
| Fahrenheit 451         |        1 |   158 |
+------------------------+----------+-------+
```

1. **Bug A, 1064:** WHERE must come before ORDER BY.
2. **Bug B, 1054:** there is no third column; sort by name: `ORDER BY price` (a column you did not select is allowed).
3. **Bug C, 1064:** no formulas in LIMIT/OFFSET: `OFFSET 8`.
4. **Bug D, silent:** `'pages'` in quotes is a constant; the three rows are arbitrary (they can differ). Fix: `ORDER BY pages DESC, title`.
5. **Bug E, silent:** in_stock is ascending. Fix: `ORDER BY in_stock DESC, pages DESC, title`.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 3.1 · WHERE CLAUSES, OPERATORS & FILTERING

1. **B** 0: city = NULL is unknown for every row, even for Eve and Hank, and WHERE only keeps rows where the test is true. The query returns an empty set without any error. Use IS NULL instead.
2. **C** a = 1 OR (b = 2 AND c = 3): AND is evaluated before OR, just as multiplication comes before addition. Add parentheses whenever you mix the two so the meaning is clear.

3. **A** 9.49, 10.99 and everything in between: BETWEEN includes both ends. It is shorthand for: price is at least 9.49 AND price is at most 10.99.

4. **C** LIKE '___': Each underscore matches exactly one character, so three underscores match exactly three: Bob and Eve.

5. **A** The text value Austen is missing its quotes: Without quotes, Austen is read as a column name. Text values belong in single quotes: WHERE author = 'Jane Austen'.

LESSON 3.2 · SORTING & LIMITING RESULTS

1. **A** At the top: MySQL sorts NULL as smaller than every value, so NULLs come first in ascending order and last in descending order.

2. **C** Rows 7 to 9: OFFSET 6 skips rows 1 to 6, then LIMIT 3 keeps the next three: rows 7, 8 and 9.

3. **B** The first row is the 7.99 book; the second can be either 8.99 book: The 7.99 book is certain to come first, but the 8.99 books tie, so which one fills the second slot is up to the server. Add a unique tie-breaker such as title.

4. **A** LIMIT 4 OFFSET (3 - 1) * 4: LIMIT and OFFSET accept plain numbers or placeholders, not expressions. Compute 8 first and write OFFSET 8.

5. **C** LIMIT 20 OFFSET 80: Page N starts after (N - 1) x page size rows: (5 - 1) x 20 = 80 rows skipped, then 20 returned.

EXIT TICKET

1. Why does `WHERE city != 'Chicago'` skip Eve and Hank, and how do you include them? **Answer:** Their city is NULL, so the comparison is unknown, not true, and only true rows are kept. Add `OR city IS NULL`.

2. Write the WHERE clause for: books from $9 to $11, both ends included, that are not by Tolkien. **Answer:** `WHERE price BETWEEN 9 AND 11 AND author != 'J.R.R. Tolkien'` (on the course data: Brave New World, 1984, Fahrenheit 451).

3. Why should every ORDER BY that feeds a LIMIT end with a unique column? **Answer:** If rows tie at the cut-off, the server may return either one, and the choice can change between runs: page 1 and page 2 can repeat or skip a row. A unique column (id, title here) makes the order repeatable.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 A (6).** Counts the NULL cities as "not Chicago". The day's main idea: unknown is dropped.
- **Q3 B.** The classic != OR != that is always true. Translate it into English with them.
- **Q4 A.** Treats a DATETIME like a DATE: show the 0-row result.

- **Q7 B.** Thinks DESC applies to the whole ORDER BY.
- **Q9 A.** Does not see the quotes: the most dangerous silent sort mistake.
- **Q11 A or D.** Confuses rows skipped with ids: an OFFSET counts rows.

INSTRUCTOR KIT · ANSWER KEY

Session 5: Aggregates and Subqueries

Answers and reasoning for every Session 5 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs and the fixed debug handouts, the lesson pages' own quizzes and the exit ticket.

Session 5 | 12 self-check items | Confidential

WARM-UP

W1

Who comes back, and why?

```
SELECT first_name, city, id
FROM customers
WHERE city = 'Chicago' OR city = 'Houston' AND id > 5
ORDER BY id;
```

Bob and Grace. AND binds before OR: `city = 'Chicago' OR (city = 'Houston' AND id > 5)`, and Dave (Houston) has id 4.

```
+------------+---------+----+
| first_name | city    | id |
+------------+---------+----+
| Bob        | Chicago |  2 |
| Grace      | Chicago |  7 |
+------------+---------+----+
```

W2

Two customers have no city. Why does this find nobody? What would you write instead?

```
SELECT first_name FROM customers WHERE city = NULL;
```

Nobody: nothing equals NULL, not even NULL. Write `WHERE city IS NULL` (Eve and Hank).

```
Empty set
```

W3

In which order must you write `WHERE`, `ORDER BY` and `LIMIT`?

WHERE, then ORDER BY, then LIMIT. Lesson 3.3 adds GROUP BY and HAVING between WHERE and ORDER BY.

W4

Which three books does this show?

```
SELECT title, price
FROM books
ORDER BY price DESC, title
LIMIT 3 OFFSET 2;
```

The 3rd, 4th and 5th most expensive: OFFSET 2 skips Lord of the Rings and Dune.

```
+------------------------+-------+
| title                  | price |
+------------------------+-------+
| To Kill a Mockingbird  | 11.99 |
| The Hobbit             | 10.99 |
| Fahrenheit 451         | 10.49 |
+------------------------+-------+
```

W5

Predict: how many customers live outside Chicago, according to this query?

```
SELECT first_name
FROM customers
WHERE city NOT IN ('Chicago', NULL);
```

None: `city != 'Chicago' AND city != NULL` is unknown for every row. A NULL inside a NOT IN list empties the result (Lesson 3.1), and it comes back inside subqueries in Lesson 3.4.

```
Empty set
```

3.3 · AGGREGATE FUNCTIONS & GROUP BY: SELF-CHECK

Q1

Grace (Chicago) and Hank (no city) are the only customers who signed up in February. What does this print?

```
SELECT COUNT(*), COUNT(city)
FROM customers
WHERE created_at >= '2026-02-01';
```

A) `2` and `2`

B) `2` and `1`

C) `1` and `1`

D) `8` and `6`

Why: WHERE keeps the two February rows first. `COUNT(*)` counts both rows; `COUNT(city)` counts only the filled-in cities, and Hank's is NULL.

A is wrong: `COUNT(column)` skips NULL; only `COUNT(*)` counts every row.

C is wrong: Hank's row is still a row: COUNT(*) counts rows, not values.

D is wrong: Those are the counts for the whole table; the WHERE line runs before the counting.

Q2

You want only the authors whose books average more than $10. Where does that condition go?

A) In `WHERE`, before `GROUP BY`

B) In `ORDER BY`

C) In `HAVING`, after `GROUP BY`

D) In the `SELECT` list, as `AVG(price) > 10`

Why: The average belongs to a whole group (an author's pile), so it can only be tested after the piles exist: HAVING. A condition on single rows (price > 10) goes in WHERE.

A is wrong: WHERE runs before the piles exist; an aggregate there is error 1111.

B is wrong: ORDER BY sorts; it never removes rows.

D is wrong: That adds a 0/1 column; every author still appears.

Q3

The flat bookstore has no books by Isaac Asimov. What does this print?

```
SELECT COUNT(*) AS n, MAX(price) AS top
FROM books
WHERE author = 'Isaac Asimov';
```

One row: `n` = 0 and `top` = NULL

> **Why:** An aggregate over no rows still returns one row. COUNT answers 0; every other aggregate has no values to work with and returns NULL (use COALESCE(MAX(price), 0) when a report needs 0).

Q4

What happens when this runs on MySQL 9.7?

```
SELECT city, first_name
FROM customers
GROUP BY city;
```

A) `ERROR 1055 (42000)`: first_name is not in GROUP BY and not aggregated

B) One row per city, each with one customer's first name

C) `ERROR 1140`: an aggregated query without GROUP BY

D) One row per customer, sorted by city

> **Why:** Chicago's pile holds Bob and Grace: which first name belongs in Chicago's one row? MySQL's default ONLY_FULL_GROUP_BY mode refuses to guess (error 1055). Fix: aggregate it (MIN(first_name)), group by it, or drop it.
>
> **B** is wrong: That is what MariaDB (no ONLY_FULL_GROUP_BY) does silently: an arbitrary name per city.
>
> **C** is wrong: 1140 is for an aggregate with no GROUP BY at all; this query has one.
>
> **D** is wrong: GROUP BY makes one row per group, never one per input row.

Q5

A colleague says: "I don't need `ORDER BY` after `GROUP BY city`: the cities always come out sorted." On MySQL 9.7, is that right?

A) Yes: GROUP BY sorts the groups by the grouped column

B) Yes, but only when the column has an index

C) Only on Windows servers

D) No: since MySQL 8.0 the order of groups is not guaranteed; add ORDER BY

> **Why:** MySQL 8.0 and later return groups in whatever order is convenient (the lesson's Common Mistake 5 shows them in first-seen order). MariaDB still sorts, so the same query can come back in a different order on another server. A report always says ORDER BY.
> **A** is wrong: That was true of MySQL 5.7 and is still true of MariaDB, not of MySQL 9.7.
> **B** is wrong: An index may make a sorted order likely, never guaranteed.
> **C** is wrong: The operating system has nothing to do with it.

Q6

What does this print? (Under $10: three books in stock and two out of stock.)

```
SELECT in_stock, COUNT(*) AS books
FROM books
WHERE price < 10
GROUP BY in_stock
ORDER BY in_stock;
```

Two rows: `0 | 2` and `1 | 3`

> **Why:** WHERE keeps the five books under $10; GROUP BY makes one pile per in_stock value (0 = out of stock, 1 = in stock); ORDER BY in_stock puts 0 first.

3.3 · AGGREGATE FUNCTIONS & GROUP BY: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
SELECT COUNT(*), COUNT(city), COUNT(DISTINCT city)
FROM customers
WHERE id > 4;
```

```
+----------+-------------+----------------------+
| COUNT(*) | COUNT(city) | COUNT(DISTINCT city) |
+----------+-------------+----------------------+
|        4 |           2 |                    2 |
+----------+-------------+----------------------+
```

Why: Customers 5 to 8 are Eve (NULL), Frank (New York), Grace (Chicago) and Hank (NULL): four rows, two filled-in cities, two different cities.

P2

Predict the output, then run it.

```
SELECT in_stock, MAX(pages) AS longest
FROM books
GROUP BY in_stock
HAVING MAX(pages) > 300
ORDER BY in_stock;
```

```
+----------+---------+
| in_stock | longest |
+----------+---------+
|        1 |    1178 |
+----------+---------+
```

Why: Two piles: in stock (longest 1178, Lord of the Rings) and out of stock (longest 234). HAVING drops the out-of-stock pile; one row remains.

3.4 · SUBQUERIES & NESTED QUERIES: SELF-CHECK

Q7

The average price is 10.99. What does this print?

```
SELECT COUNT(*)
FROM books
WHERE price < (SELECT AVG(price) FROM books);
```

A) `4`

B) `6`

C) `7`

D) `10`

Why: The subquery returns 10.99; six books cost less (7.99 to 10.49). The Hobbit costs exactly 10.99 and is not less than itself.

A is wrong: Four books cost MORE than or equal to the average (three above, The Hobbit equal).

C is wrong: Seven would include The Hobbit, which costs exactly 10.99.

D is wrong: Ten is every book: the subquery is a real filter, not a decoration.

Q8

What happens when this runs?

```
SELECT title
FROM books
WHERE price = (SELECT price FROM books
               WHERE author = 'J.R.R. Tolkien');
```

A) `ERROR 1242 (21000): Subquery returns more than 1 row`

B) It returns Lord of the Rings

C) `ERROR 1241 (21000): Operand should contain 1 column(s)`

D) `ERROR 1248 (42000): Every derived table must have its own alias`

> **Why:** Tolkien has two books, so the subquery returns two prices, and `=` needs one value. Fix: `IN` (any of his prices) or an aggregate such as MAX(price). The query would have worked while he had one book: the lesson's warning about queries that break when the data grows.
> **B** is wrong: MySQL does not pick one of the values for you.
> **C** is wrong: 1241 is too many COLUMNS; here there is one column and too many rows.
> **D** is wrong: 1248 is for a subquery in FROM without a name; this one is in WHERE.

Q9

Customer 5 (Eve) has no city; customer 6 (Frank) lives in New York. What does this print?

```
SELECT COUNT(*)
FROM customers
WHERE city NOT IN (SELECT city FROM customers WHERE id IN (5, 6));
```

`0` **(no customer passes the test)**

> **Why:** The list is (NULL, 'New York'). `city NOT IN (...)` means city != NULL AND city != 'New York', and anything compared with NULL is unknown, so no row is ever kept. Add `AND city IS NOT NULL` inside the subquery, or use NOT EXISTS.

Q10

You need the customers whom no other customer shares a city with, and city can be NULL. Which approach gives a predictable answer with no NULL trap?

A) `city NOT IN (SELECT city FROM customers ...)`

B) `city != ALL (SELECT city ...)`

C) `NOT EXISTS (SELECT 1 FROM customers AS other WHERE other.city = c.city AND other.id != c.id)`

D) `city NOT LIKE (SELECT city ...)`

> **Why:** NOT EXISTS asks "is there a matching row?": a NULL city never matches, so it cannot empty the result, and the customers with no city are kept (nobody is known to share their city). That is why many developers write NOT EXISTS by default.
> **A** is wrong: One NULL in the list empties a NOT IN.
> **B** is wrong: != ALL is the same as NOT IN, with the same trap.
> **D** is wrong: LIKE compares with one pattern; with several rows it fails with error 1242.

Q11

What does this print? (Two customers have no city.)

```sql
SELECT MAX(n) AS biggest_group
FROM (
    SELECT city, COUNT(*) AS n
    FROM customers
    GROUP BY city
) AS per_city;
```

`2`

> **Why:** The derived table holds one row per city, NULL included: NULL 2, Chicago 2, New York 2, Houston 1, Los Angeles 1. The outer MAX reads that small table: 2. The alias per_city is required (error 1248 without it).

Q12

This was meant to list the books priced above THEIR OWN author's average. Why does it return three books instead of one?

```
SELECT title, author, price
FROM books
WHERE price > (
    SELECT AVG(price)
    FROM books
    WHERE author = author
)
ORDER BY price;
```

A) The subquery runs once per author, but AVG ignores the out-of-stock books

B) MySQL treats the second author as a string literal

C) HAVING is missing from the subquery

D) Both author names refer to the inner table, so author = author is always true and the subquery returns the store average

Why: Inside the subquery the nearest table wins: both sides of author = author are the inner books row. The subquery is not correlated at all and returns 10.99, so you get "above the store average" by accident. Fix: alias both copies (FROM books AS b ... FROM books AS b2 WHERE b2.author = b.author).

A is wrong: AVG uses every book; in_stock plays no part here.

B is wrong: An unquoted name is a column, not a string.

C is wrong: HAVING filters groups; there are no groups in this subquery.

3.4 · SUBQUERIES & NESTED QUERIES: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
SELECT COUNT(*) AS long_books
FROM books
WHERE pages > (SELECT AVG(pages) FROM books);
```

```
+------------+
| long_books |
+------------+
|          3 |
+------------+
```

> **Why:** The inner query returns 382.4; only Lord of the Rings (1178), Pride and Prejudice (432) and Dune (412) are longer.

P4

Predict the output, then run it.

```
SELECT first_name
FROM customers
WHERE city NOT IN (SELECT city FROM customers WHERE id = 5);
```

```
Empty set
```

> **Why:** The list is just (NULL): Eve's city. `city NOT IN (NULL)` is unknown for every row, so nobody comes back, not even the customers with a city.

WORKBOOK WORKED EXAMPLES (OUTPUTS FOR REFERENCE)

The worked examples print exactly what the workbook shows; they are new examples, not the lessons' own. 3.3: Tolkien 1937-09-21 to 1954-07-29; WHERE leaves one pile (in stock, 5 books over $10); HAVING drops the out-of-stock pile (2 books). 3.4: the latest signup per city is Grace (Chicago), Dave (Houston), Carol (Los Angeles), Frank (New York).

LAB REFERENCE SOLUTIONS

- LAB 3.3-A: LABS/MODULE-05/L3_3-AGGREGATE-FUNCTIONS/CUSTOMER_SUMMARY_STARTER.SQL → CUSTOMER_SUMMARY_SOLUTION.SQL

The solution prints:

```
+-----------+-----------+--------------+--------+---------------------+---------------
| customers | with_city | missing_city | cities | first_signup        | latest_signup
+-----------+-----------+--------------+--------+---------------------+---------------
|         8 |         6 |            2 |      4 | 2026-01-04 10:15:00 | 2026-02-10 19:1
+-----------+-----------+--------------+--------+---------------------+---------------
+---------+---------+
| month   | signups |
+---------+---------+
| 2026-01 |       6 |
| 2026-02 |       2 |
+---------+---------+
+----------+-----------+---------------------+
| city     | customers | first_joined        |
+----------+-----------+---------------------+
| New York |         2 | 2026-01-04 10:15:00 |
| Chicago  |         2 | 2026-01-06 14:02:00 |
+----------+-----------+---------------------+
+-------------+-----------------+
| city        | january_signups |
+-------------+-----------------+
| New York    |               2 |
| (no city)   |               1 |
| Chicago     |               1 |
| Houston     |               1 |
| Los Angeles |               1 |
+-------------+-----------------+
```

What each TODO teaches: (1) the three COUNTs and the NULL rule in one row (8 rows, 6 filled-in cities, 2 missing, 4 different cities); (2) grouping by an expression label (`DATE_FORMAT`), which sorts correctly as text; (3) HAVING on a pile size, plus MIN as "when did the pile start"; (4) WHERE and HAVING-free grouping with COALESCE: the NULL cities form one pile, labeled `(no city)`. Sticking points: COALESCE in SELECT but `GROUP BY city` (works, but the label and the pile differ in the learner's head: ask them to group by the same expression), and TODO 3's HAVING typed before GROUP BY (1064).

- LAB 3.3-B: LABS/MODULE-05/L3_3-AGGREGATE-FUNCTIONS/PAGE_REPORT_STARTER.SQL → PAGE_REPORT_SOLUTION.SQL

The solution prints:

```
+---------------------+-------+-----------+
| length_band         | books | avg_price |
+---------------------+-------+-----------+
| Short (under 250)   |     3 |      9.49 |
| Medium (250 to 399) |     4 |     10.62 |
| Long (400 and up)   |     3 |     12.99 |
+---------------------+-------+-----------+
+-----------------+-------+-------------+
| author          | books | total_pages |
+-----------------+-------+-------------+
| J.R.R. Tolkien |     2 |        1488 |
| Jane Austen     |     1 |         432 |
| Frank Herbert   |     1 |         412 |
| George Orwell   |     1 |         328 |
| Aldous Huxley   |     1 |         311 |
+-----------------+-------+-------------+
+---------+-------+------------+
| century | books | earliest   |
+---------+-------+------------+
|      19 |     1 | 1813-01-28 |
|      20 |     9 | 1925-04-10 |
+---------+-------+------------+
+-----------------+----------------+
| author          | pages_in_stock |
+-----------------+----------------+
| J.R.R. Tolkien |           1488 |
| Jane Austen     |            432 |
| Frank Herbert   |            412 |
| George Orwell   |            328 |
| Aldous Huxley   |            311 |
| Harper Lee      |            281 |
+-----------------+----------------+
```

Reading it: the length bands are sorted by `MIN(pages)`, not by name (alphabetical would put Long first); the century formula `FLOOR((YEAR(published) - 1) / 100) + 1` puts 1813 in the 19th century and 1925 in the 20th; in TODO 4, removing the WHERE line lets J.D. Salinger in (his out-of-stock book has 234 pages), which is the WHERE-before-piles idea in one row.

- HANDOUT: LABS/MODULE-05/L3_3-AGGREGATE-FUNCTIONS/DEBUG_GROUP_BY.SQL (DELIBERATELY BROKEN)

As shipped, MySQL prints:

```
ERROR 1140 (42000): In aggregated query without GROUP BY, expression #1 of SELECT list
ERROR 1055 (42000): Expression #2 of SELECT list is not in GROUP BY clause and contains
ERROR 1111 (HY000): Invalid use of group function
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresp
```

1. **Bug A, 1140:** a whole-table COUNT next to a plain column. Fix: drop first_name (`SELECT COUNT(*) AS customers FROM customers;`).
2. **Bug B, 1055:** last_name is neither grouped nor aggregated. Fix: `MIN(last_name) AS a_last_name` (or drop it) and add ORDER BY.
3. **Bug C, 1111:** an aggregate in WHERE. Fix: `HAVING COUNT(*) > 1` after GROUP BY.
4. **Bug D, 1064:** HAVING written before GROUP BY. Fix: GROUP BY city, then HAVING.

Fixed (blocks in `blocks_S05.py`), the four queries print, in order:

```
+-----------+
| customers |
+-----------+
|         8 |
+-----------+
+-------------+-------------+-----------+
| city        | a_last_name | customers |
+-------------+-------------+-----------+
| NULL        | Davis       |         2 |
| Chicago     | Smith       |         2 |
| Houston     | Brown       |         1 |
| Los Angeles | Williams    |         1 |
| New York    | Johnson     |         2 |
+-------------+-------------+-----------+
+-----------------+-------+
| author          | books |
+-----------------+-------+
| J.R.R. Tolkien |     2 |
+-----------------+-------+
+----------+-----------+
| city     | customers |
+----------+-----------+
| NULL     |         2 |
| Chicago  |         2 |
| New York |         2 |
+----------+-----------+
```

Bug D's fixed result includes the NULL group (Eve and Hank): add `WHERE city IS NOT NULL` if the question is about real cities. Learners on MariaDB see no error for Bug B (it silently picks a last name): a good moment to show why the MySQL check exists.

- LAB 3.4-A: LABS/MODULE-05/L3_4-SUBQUERIES/PAGE_CHECKS_STARTER.SQL → PAGE_CHECKS_SOLUTION.SQL

The solution prints:

```
+---------------------+-------------+------------+
| title               | author      | published  |
+---------------------+-------------+------------+
| Pride and Prejudice | Jane Austen | 1813-01-28 |
+---------------------+-------------+------------+
+-------------------------+-------+-------+
| title                   | pages | price |
+-------------------------+-------+-------+
| Fahrenheit 451          |   158 | 10.49 |
| The Great Gatsby        |   180 |  8.99 |
| The Catcher in the Rye  |   234 |  8.99 |
| Brave New World         |   311 |  9.49 |
| 1984                    |   328 |  9.99 |
+-------------------------+-------+-------+
+------------------------+---------------------+----------+
| title                  | author              | in_stock |
+------------------------+---------------------+----------+
| The Great Gatsby       | F. Scott Fitzgerald |        0 |
| The Catcher in the Rye | J.D. Salinger       |        0 |
+------------------------+---------------------+----------+
+------------------------+------------+
| title                  | published  |
+------------------------+------------+
| Fahrenheit 451         | 1953-10-19 |
| Lord of the Rings      | 1954-07-29 |
| To Kill a Mockingbird  | 1960-07-11 |
| Dune                   | 1965-08-01 |
+------------------------+------------+
+---------------+
| same_with_max |
+---------------+
|             4 |
+---------------+
```

Reading it: (1) the row behind MIN(published); (2) two scalar subqueries in one WHERE (382.4 pages and 10.99 dollars); (3) an IN list of two authors, whose only books are the out-of-stock ones; (4) `> ALL` equals `> (SELECT MAX(...))` here (4 rows each). If a learner asks: with an EMPTY list, `> ALL` is true for every row while `> MAX(...)` keeps nothing (the lesson's edge case).

- LAB 3.4-B: LABS/MODULE-05/L3_4-SUBQUERIES/CITY_NEIGHBORS_STARTER.SQL → CITY_NEIGHBORS_SOLUTION.SQL

The solution prints:

```
+------------+-------------+-----------+
| first_name | city        | neighbors |
+------------+-------------+-----------+
| Alice      | New York    |         1 |
| Bob        | Chicago     |         1 |
| Carol      | Los Angeles |         0 |
| Dave       | Houston     |         0 |
| Eve        | NULL        |         0 |
| Frank      | New York    |         1 |
| Grace      | Chicago     |         1 |
| Hank       | NULL        |         0 |
+------------+-------------+-----------+
+------------+-------------+---------------------+
| first_name | city        | created_at          |
+------------+-------------+---------------------+
| Alice      | New York    | 2026-01-04 10:15:00 |
| Bob        | Chicago     | 2026-01-06 14:02:00 |
| Carol      | Los Angeles | 2026-01-09 09:40:00 |
| Dave       | Houston     | 2026-01-12 16:25:00 |
+------------+-------------+---------------------+
+------------------+---------------+
| authors_over_300 | longest_total |
+------------------+---------------+
|                5 |          1488 |
+------------------+---------------+
+------------+-------------+
| first_name | city        |
+------------+-------------+
| Carol      | Los Angeles |
| Dave       | Houston     |
+------------+-------------+
+------------+-------------+
| first_name | city        |
+------------+-------------+
| Carol      | Los Angeles |
| Dave       | Houston     |
+------------+-------------+
```

What each TODO teaches: (1) a correlated subquery in the SELECT list; Eve and Hank get 0 because NULL matches nothing; (2) a correlated subquery in WHERE (the row behind each city's MIN); (3) a derived table filtered by its own column (5 authors over 300 pages; Tolkien's 1488 the largest); (4) the NOT IN trap: as shipped 4a prints nothing because the later customers include Eve and Hank (NULL); with `IS NOT NULL` it returns Carol and Dave, and NOT EXISTS agrees.

HANDOUT: LABS/MODULE-05/L3_4-SUBQUERIES/DEBUG_SUBQUERIES.SQL (DELIBERATELY BROKEN)

As shipped, MySQL prints:

```
ERROR 1242 (21000): Subquery returns more than 1 row
ERROR 1241 (21000): Operand should contain 1 column(s)
ERROR 1248 (42000): Every derived table must have its own alias
ERROR 1235 (42000): This version of MySQL doesn't yet support 'LIMIT & IN/ALL/ANY/SOME
```

1. **Bug A, 1242:** two February customers, so two dates for `=`. Fix: `IN`.
2. **Bug B, 1241:** the IN list has two columns. Fix: select only `author`; `SUM(pages)` stays in HAVING.
3. **Bug C, 1248:** the derived table has no name. Fix: `AS city_counts`.
4. **Bug D, 1235:** LIMIT inside an IN subquery. Fix: wrap the LIMIT query in a derived table and select from it.

Fixed, the four queries print, in order:

```
+------------+---------------------+
| first_name | created_at          |
+------------+---------------------+
| Grace      | 2026-02-02 08:55:00 |
| Hank       | 2026-02-10 19:12:00 |
+------------+---------------------+
+----------------------+----------------+
| title                | author         |
+----------------------+----------------+
| Dune                 | Frank Herbert  |
| Lord of the Rings    | J.R.R. Tolkien |
| The Hobbit           | J.R.R. Tolkien |
| Pride and Prejudice | Jane Austen    |
+----------------------+----------------+
+--------+
| AVG(n) |
+--------+
| 1.5000 |
+--------+
+----------------------+-------+
| title                | pages |
+----------------------+-------+
| Lord of the Rings    |  1178 |
| Pride and Prejudice |   432 |
+----------------------+-------+
```

On MariaDB Bug C is a 1064 syntax error near `''` instead of 1248 (the lesson's version note).

THE LESSON PAGES' OWN QUICK QUIZZES

1. **B** 6: COUNT(column) counts only the values that are not NULL: 8 rows minus the 2 missing cities is 6. COUNT(*) would return 8.

2. **C** In a HAVING clause after GROUP BY: HAVING runs after GROUP BY, when each author's pile has been counted, so it can keep or drop whole groups by their aggregate.

3. **A** 4: AVG skips the NULL completely, so it divides 5 + 3 + 4 = 12 by 3 values, not by 4 rows.

4. **B** title is neither in GROUP BY nor inside an aggregate: With ONLY_FULL_GROUP_BY on, every selected column must be grouped or aggregated. An author with two books has two titles, so title is ambiguous.

5. **A** Add an ORDER BY clause: Since MySQL 8.0, GROUP BY makes no promise about order (MariaDB still sorts). ORDER BY is the only way to get the same order everywhere.

LESSON 3.4 · SUBQUERIES & NESTED QUERIES

1. **C** 3: Only To Kill a Mockingbird, Dune and Lord of the Rings cost more than 10.99. The Hobbit equals the average, and greater-than does not include equal.

2. **B** Use IN instead of =, or make the subquery return a single value: The subquery sits next to = or another single-value operator but returned a list. Use IN for a list, or reduce the subquery to one value with MIN, MAX or a unique condition.

3. **A** No rows at all: NOT IN means not equal to every item. Not equal to NULL is unknown for every row, so no row can pass. Filter NULLs out of the subquery or use NOT EXISTS.

4. **C** It refers to a column of the outer query's current row: A correlated subquery uses a column of the outer query's current row, such as b2.author = b.author, so its answer can differ for every outer row.

5. **B** Adding an alias such as AS author_counts after the closing parenthesis: Every derived table must have its own alias. Adding AS author_counts after the closing parenthesis makes the query run.

EXIT TICKET

1. **WHERE and HAVING:** any single-row condition in WHERE (`in_stock = TRUE`, `price > 10`, `published < '1950-01-01'`) and any aggregate condition in HAVING (`COUNT(*) > 1`, `AVG(price) > 10`, `SUM(pages) > 400`). A learner who puts `price > 10` in HAVING has the clauses as interchangeable filters: reteach the run order.

2. **NOT IN with NULL:** no rows. Fixes: `AND city IS NOT NULL` inside the subquery, or NOT EXISTS (which also keeps the NULL-city customers, a different but predictable answer).

3. **Correlated:** it refers to the outer query's current row, so it is evaluated again for every outer row. With one table used twice, aliases (b for the outer row, b2 for the inner scan) are how you say which copy a column belongs to; without them `author = author` compares a row with itself.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 A (2 and 2).** Thinks COUNT(column) counts rows. Show `COUNT(*)` and `COUNT(city)` side by side again in the Session 6 warm-up.
- **Q2 A (WHERE).** Treats WHERE and HAVING as the same filter. The run-order diagram is the cure; ask "when does WHERE run?"
- **Q4 B (one name per city).** Has used MariaDB or old MySQL, or imagines MySQL picks "the first". The 1055 check exists so answers are honest.
- **Q7 C (7).** Counted The Hobbit (exactly 10.99): `<` and `<=` are different questions. Worth ten seconds.
- **Q9 any number other than 0.** Has not met the NOT IN trap yet: run the lesson's two queries side by side.
- **Q12 A or C.** Looks for a missing clause instead of reading what `author` refers to. Ask them to run the subquery alone: it returns one number, the store average.
- **Exit ticket 3 with "it is inside another query".** That is any subquery; correlated means it uses the OUTER row. Ask: "can you run it on its own?"

INSTRUCTOR KIT · ANSWER KEY

Session 6: Relationships and Foreign Keys

Answers and reasoning for every Session 6 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs and the debug handouts, the lesson pages' own quizzes and the exit ticket.

Session 6 | 12 self-check items | Confidential

WARM-UP

W1

Three numbers: predict them. (Customers 1 to 5 signed up before 25 January; Eve has no city.)

```
SELECT COUNT(*), COUNT(city), COUNT(DISTINCT city)
FROM customers
WHERE created_at < '2026-01-25';
```

5, 4, 4: five rows, four filled-in cities (Eve's is NULL), four different cities.

```
+----------+-------------+----------------------+
| COUNT(*) | COUNT(city) | COUNT(DISTINCT city) |
+----------+-------------+----------------------+
|        5 |           4 |                    4 |
+----------+-------------+----------------------+
```

W2

`price > 10` and `COUNT(*) > 1`: which goes in WHERE, and which in HAVING?

`price > 10` in WHERE (one row's column); `COUNT(*) > 1` in HAVING (a group's aggregate).

W3

Which book is it?

```
SELECT title
FROM books
WHERE pages = (SELECT MAX(pages) FROM books);
```

Lord of the Rings: the row behind MAX(pages).

```
+-------------------+
| title             |
+-------------------+
| Lord of the Rings |
+-------------------+
```

W4

How many customers? (Customer 7 is Grace in Chicago, 8 is Hank with no city.)

```
SELECT COUNT(*)
FROM customers
WHERE city NOT IN (SELECT city FROM customers WHERE id > 6);
```

0: the list is ('Chicago', NULL), and one NULL empties a NOT IN. Add `AND city IS NOT NULL` inside, or use NOT EXISTS.

```
+----------+
| COUNT(*) |
+----------+
|        0 |
+----------+
```

W5

What is wrong, and what is the error number?

```
SELECT author, title, MAX(price)
FROM books
GROUP BY author;
```

Error 1055: title is neither grouped nor aggregated (Tolkien's pile has two titles). Drop it or use `MIN(title)`.

```
ERROR 1055 (42000): Expression #2 of SELECT list is not in GROUP BY clause and c
```

4.1 · TABLE RELATIONSHIPS: SELF-CHECK

Q1

A customer can have many delivery addresses; each address belongs to exactly one customer. Where does the foreign key go?

A) `customers.address_id`

B) `addresses.customer_id`

C) A junction table `customer_addresses`

D) In both tables, one pointing at the other

Why: Ask "can one customer have many addresses?" Yes: the key goes on the many side, in addresses (like books.author_id and orders.customer_id).

A is wrong: A column holds one value: each customer could then have only one address (the lesson's Mistake 5).

C is wrong: A junction table is for many-to-many; an address does not belong to several customers.

D is wrong: Two keys pointing at each other duplicate one fact and make inserting either row first impossible.

Q2

Which of these needs a junction table?

A) One author writes many books

B) Each employee has at most one manager

C) Students and the courses they take

D) Each order belongs to one customer

> **Why:** A student takes many courses and a course has many students: two yeses make many-to-many, and that needs a third table with two foreign keys (enrollments), which can also hold facts about the link such as a grade.
> **A** is wrong: One-to-many: books.author_id.
> **B** is wrong: A self-referencing one-to-many: employees.manager_id.
> **D** is wrong: One-to-many: orders.customer_id.

Q3

In the finished bookstore, what does this print?

```
SELECT book_id, COUNT(*) AS categories
FROM book_categories
GROUP BY book_id
HAVING COUNT(*) = 3
ORDER BY book_id;
```

Three rows: books `2`, `7` and `10`, each with `3`

> **Why:** Each row of the junction table is one link. Grouping the links per book and keeping the piles of three finds 1984 (2), Brave New World (7) and Fahrenheit 451 (10): Fiction, plus Dystopian and one more.

Q4

The flat books table types "J.R.R. Tolkien" in two rows. A staff member edits the spelling in one of them only. What is this called?

A) An update anomaly: one fact stored twice now disagrees with itself

B) An insert anomaly

C) A delete anomaly

D) Error 1062, a duplicate entry

> **Why:** One fact (the author's name) is stored in several rows; changing one copy leaves the table believing in two Tolkiens, with no error. Storing the name once in authors removes the cause.
> **B** is wrong: An insert anomaly is having nowhere to store something new (an author with no books).
> **C** is wrong: A delete anomaly is losing a fact by deleting another (the last book removes the author).
> **D** is wrong: No rule is broken: that is exactly why the anomaly is dangerous.

Q5

A colleague builds `customer_profiles` with `customer_id INT NOT NULL` and a foreign key, but leaves out `UNIQUE`. What has the relationship become?

A) Many-to-many

B) Still one-to-one: the foreign key enforces it

C) Nothing: MySQL refuses the CREATE TABLE

D) One-to-many: a customer can now have several profiles

> **Why:** A one-to-one relationship in MySQL is a one-to-many whose foreign key column is also UNIQUE. Without UNIQUE, a second profile for Alice is accepted silently (the lesson's demo shows the 1062 that UNIQUE gives).
> **A** is wrong: Many-to-many needs a junction table with two foreign keys.
> **B** is wrong: A foreign key checks that the parent exists, not that it is referenced only once.
> **C** is wrong: The statement is valid; the design is what changed.

Q6

The bookstore's table is called books. What happens?

```
CREATE TABLE kit_reviews (
    id      INT PRIMARY KEY,
    book_id INT NOT NULL,
    FOREIGN KEY (book_id) REFERENCES book(id)
);
```

```
ERROR 1824 (HY000): Failed to open the referenced table 'book'
```

> **Why:** The foreign key names a table that does not exist (book). The same error appears when the name is right but the child is created before its parent: create parents first, children second (and drop in the opposite order).

4.1 · TABLE RELATIONSHIPS: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
INSERT INTO book_categories (book_id, category_id) VALUES (11, 3);
```

```
ERROR 1062 (23000): Duplicate entry '11-3' for key 'book_categories.PRIMARY'
```

> **Why:** Animal Farm (11) is already in Classic (3): the composite primary key allows each pair once, and the message shows the pair as '11-3'.

P2

Predict the output, then run it.

```
SELECT COUNT(*) AS authors_without_books
FROM authors
WHERE id NOT IN (SELECT author_id FROM books);
```

```
+-----------------------+
| authors_without_books |
+-----------------------+
|                     1 |
+-----------------------+
```

Why: Every book's author_id is NOT NULL, so the NOT IN list holds no NULL and the query is safe: only Isaac Asimov (10) has no book.

4.2 · FOREIGN KEYS & REFERENTIAL INTEGRITY: SELF-CHECK

Q7

The bookstore has customers 1 to 8. What happens?

```
INSERT INTO orders (customer_id, order_date)
VALUES (9, '2026-07-01');
```

A) `ERROR 1451 (23000): Cannot delete or update a parent row`

B) `ERROR 1452 (23000): Cannot add or update a child row`

C) `ERROR 1062 (23000): Duplicate entry`

D) Order 13 is created for customer 9

Why: orders is the CHILD of customers; inserting a child whose parent (customer 9) does not exist is refused by fk_orders_customer: "add or update a child row", 1452. Nothing is inserted.

A is wrong: 1451 is about a PARENT that still has children (a delete or a key change).

C is wrong: Nothing is duplicated; the id is new.

D is wrong: That is exactly what a declared foreign key prevents: an orphan order.

Q8

The bookstore has 18 order lines; order 5 has two of them. What does the SELECT print?

```
START TRANSACTION;
DELETE FROM orders WHERE id = 5;
SELECT COUNT(*) FROM order_items;
ROLLBACK;
```

A) `16`

B) `18`

C) `ERROR 1451 (23000): Cannot delete or update a parent row`

D) `0`

> **Why:** fk_items_order is ON DELETE CASCADE: deleting the order deletes its two lines too (16 left). ROLLBACK then puts all of it back.
>
> **B** is wrong: The lines are not left behind as orphans: CASCADE removes them.
>
> **C** is wrong: The delete is refused only when a RESTRICT rule protects the row; order lines cascade.
>
> **D** is wrong: Only order 5's lines go, not every line.

Q9

James Okafor (8) manages Nina Kowalski (9). What does the SELECT print?

```
START TRANSACTION;
DELETE FROM employees WHERE id = 8;
SELECT manager_id FROM employees WHERE id = 9;
ROLLBACK;
```

`NULL` **(Nina stays, with no manager)**

> **Why:** fk_employees_manager is ON DELETE SET NULL: when the manager row is deleted, the reports keep their rows and their manager_id becomes NULL. The column allows NULL, which SET NULL requires (error 1830 otherwise).

Q10

A music app deletes a playlist. Its rows in playlist_songs (the links between the playlist and its songs) should...

A) block the delete: ON DELETE RESTRICT

B) keep the links with playlist_id NULL: ON DELETE SET NULL

C) go with it: ON DELETE CASCADE

D) stay unchanged: leave the rule out (NO ACTION)

> **Why:** A junction row is only the link itself: without the playlist it means nothing. That is the lesson's first question: can the child exist without its parent? No, so CASCADE (like book_categories and order_items).
> **A** is wrong: Users would have to empty a playlist before deleting it: possible, but it protects nothing worth keeping.
> **B** is wrong: The playlist_id is part of the composite primary key, which cannot be NULL, and a link to nothing is meaningless.
> **D** is wrong: NO ACTION behaves like RESTRICT in MySQL: the delete would be refused.

Q11

The accountant wants `fk_items_order` changed from CASCADE to RESTRICT. How do you do it in MySQL?

A) `ALTER TABLE order_items ALTER FOREIGN KEY fk_items_order ON DELETE RESTRICT;`

B) `ALTER TABLE order_items MODIFY CONSTRAINT fk_items_order ON DELETE RESTRICT;`

C) `SET FOREIGN_KEY_CHECKS = 0;` then delete as usual

D) `DROP FOREIGN KEY fk_items_order`, then `ADD CONSTRAINT fk_items_order ... ON DELETE RESTRICT`

> **Why:** There is no ALTER FOREIGN KEY: drop the key by name and add it again with the new rule (the index MySQL created stays and is reused). The lesson's Exercise 4 does exactly this and then restores CASCADE.
> **A** is wrong: MySQL has no such statement (syntax error 1064).
> **B** is wrong: MODIFY changes a column, not a constraint.
> **C** is wrong: That switches checking off for the session; it changes no rule and lets orphans in.

Q12

What does this print?

```
SELECT DELETE_RULE
FROM information_schema.REFERENTIAL_CONSTRAINTS
WHERE CONSTRAINT_SCHEMA = 'bookstore'
  AND CONSTRAINT_NAME = 'fk_employees_manager';
```

`SET NULL`

> **Why:** information_schema.REFERENTIAL_CONSTRAINTS lists every foreign key with its rules. The employees self-reference is ON DELETE SET NULL (and, with no ON UPDATE written, UPDATE_RULE says NO ACTION).

4.2 · FOREIGN KEYS & REFERENTIAL INTEGRITY: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
START TRANSACTION;
DELETE FROM categories WHERE id = 4;
SELECT COUNT(*) AS links FROM book_categories;
ROLLBACK;
```

```
+-------+
| links |
+-------+
|    23 |
+-------+
```

> **Why:** fk_bc_category cascades: Fantasy's two links (The Hobbit, Lord of the Rings) go with it, 25 - 2 = 23. ROLLBACK restores both.

P4

Predict the output, then run it.

```
DELETE FROM customers WHERE id = 2;
```

```
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constra:
```

> **Why:** Bob has orders, and fk_orders_customer is RESTRICT: 1451, "delete or update a parent row". Nothing changes.

LAB REFERENCE SOLUTIONS

- LAB 4.1-A: LABS/MODULE-06/L4_1-TABLE-RELATIONSHIPS/PUBLISHER_SPLIT_STARTER.SQL → PUBLISHER_SPLIT_SOLUTION.SQL

The solution prints (the two Notes come from `DROP TABLE IF EXISTS` on a fresh database and are harmless):

```
+----------------------+----------------+----------+
| publisher_name       | publisher_city | editions |
+----------------------+----------------+----------+
| Blue Door Publishing | Portland       |        2 |
| Harbor Lane Press    | Boston         |        1 |
| Harbor Lane Press    | Cambridge      |        1 |
| North Star Book      | Denver         |        1 |
| North Star Books     | Denver         |        1 |
+----------------------+----------------+----------+
+----+----------------+-----------------+
| id | title          | publisher_name  |
+----+----------------+-----------------+
|  5 | Fahrenheit 451 | North Star Book |
+----+----------------+-----------------+
+---------------------------+
| editions_without_publisher |
+---------------------------+
|                         0 |
+---------------------------+
+---------------------+----------------------+
| title               | publisher            |
+---------------------+----------------------+
| Dune                | Harbor Lane Press    |
| The Hobbit          | North Star Books     |
| Brave New World     | Harbor Lane Press    |
| 1984                | Blue Door Publishing |
| Fahrenheit 451      | North Star Books     |
| Pride and Prejudice | Blue Door Publishing |
+---------------------+----------------------+
+----------------------+----------+----------+
| name                 | city     | editions |
+----------------------+----------+----------+
| Harbor Lane Press    | Boston   |        2 |
| North Star Books     | Denver   |        2 |
| Blue Door Publishing | Portland |        2 |
+----------------------+----------+----------+
```

What it teaches: the anomaly table shows both kinds of damage (Harbor Lane Press in two cities: a copy that drifted; North Star split by a typo: a copy that never matched). Step 4 is the point: the UPDATE ... JOIN leaves edition 5 unmatched, and without the IS NULL check Step 5 would fail with 1138 (or, with a nullable column, silently lose the publisher). Learners must also DECIDE Harbor Lane Press's city (the parent table holds one): the cost of an update anomaly is a judgment call during the migration.

- `LAB 4.1-B: LABS/MODULE-06/L4_1-TABLE-RELATIONSHIPS/NAVIGATE_LINKS_STARTER.SQL → NAVIGATE_LINKS_SOLUTION.SQL`

The solution prints:

```
+-----------------+-------------+------------+
| TABLE_NAME      | COLUMN_NAME | points_to  |
+-----------------+-------------+------------+
| book_categories | book_id     | books      |
| book_categories | category_id | categories |
| books           | author_id   | authors    |
| employees       | manager_id  | employees  |
| order_items     | book_id     | books      |
| order_items     | order_id    | orders     |
| orders          | customer_id | customers  |
+-----------------+-------------+------------+
+-----------------+------------+
| title           | categories |
+-----------------+------------+
| 1984            |          3 |
| Brave New World |          3 |
| Fahrenheit 451  |          3 |
| Animal Farm     |          2 |
| Dune            |          2 |
+-----------------+------------+
+------------+--------+
| first_name | orders |
+------------+--------+
| Alice      |      3 |
| Bob        |      2 |
| Dave       |      2 |
| Frank      |      2 |
| Grace      |      2 |
| Eve        |      1 |
| Carol      |      0 |
| Hank       |      0 |
+------------+--------+
+------------+------------+
| first_name | department |
+------------+------------+
| Tom        | sales      |
| Lena       | sales      |
| Sofia      | inventory  |
| Nina       | support    |
| Ben        | sales      |
+------------+------------+
+-------------------+
| title             |
+-------------------+
| Lord of the Rings |
| The Hobbit        |
+-------------------+
```

Reading it: (1) seven declared keys, one self-reference (employees.manager_id); (2) the junction table counted per book; (3) the zero-or-many end of customers → orders (Carol and Hank 0); (4) NOT EXISTS on a self-reference: the five employees nobody reports to; (5) two links followed with nested IN. Every one of these becomes a join in Lesson 4.3; this lab is the "before" picture.

- HANDOUT: LABS/MODULE-06/L4_1-TABLE-RELATIONSHIPS/DEBUG_RELATIONSHIPS.SQL (DELIBERATELY BROKEN)

As shipped, MySQL prints:

```
ERROR 1824 (HY000): Failed to open the referenced table 'customer'
ERROR 1062 (23000): Duplicate entry '9-4' for key 'book_categories.PRIMARY'
ERROR 1138 (22004): Invalid use of NULL value
```

1. **Bug A, 1824:** the key names `customer`; the table is `customers`. Fix the name.
2. **Bug B, 1062:** Lord of the Rings is already Fantasy ('9-4'): the link exists. Nothing to fix unless another pair was meant.
3. **Bug C, 1138:** signing 2 has no author yet, so the column cannot become NOT NULL. Fill it first (look the author up), check for NULLs, then ALTER.

Fixed (blocks in `blocks_S06.py`): the wishlist gets both keys, the existing links of book 9 are 1 and 4, and the signings table accepts NOT NULL once row 2 has an author:

```
+----------------------+
| CONSTRAINT_NAME      |
+----------------------+
| fk_wishlist_book     |
| fk_wishlist_customer |
+----------------------+
+---------+-------------+
| book_id | category_id |
+---------+-------------+
|       9 |           1 |
|       9 |           4 |
+---------+-------------+
+----+-----------+---------+
| id | author_id | city    |
+----+-----------+---------+
|  1 |         6 | Chicago |
|  2 |         9 | Houston |
+----+-----------+---------+
```

On MariaDB, Bug A is `ERROR 1005 ... errno: 150` and Bug C is `ERROR 1265 Data truncated` (the lesson's version note).

LAB 4.2-A: LABS/MODULE-06/L4_2-FOREIGN-KEYS/DELETE_RULES_STARTER.SQL → DELETE_RULES_SOLUTION.SQL

The solution (all five TODO lines on) prints:

```
+--------------+
| links_before |
+--------------+
|           25 |
+--------------+
+-------------+
| links_after |
+-------------+
|          22 |
+-------------+
+----+------------+------------+
| id | first_name | manager_id |
+----+------------+------------+
|  7 | Sofia      |       NULL |
+----+------------+------------+
+-----------------+
| customers_left |
+-----------------+
|               7 |
+-----------------+
+----+-------------+------------+
| id | customer_id | order_date |
+----+-------------+------------+
|  6 |          70 | 2026-02-14 |
| 11 |          70 | 2026-05-19 |
+----+-------------+------------+
+--------------+
| items_before |
+--------------+
|           18 |
+--------------+
+-------------+
| items_after |
+-------------+
|          16 |
+-------------+
+------------+-------+-----------+--------+-------+-----------------+
| categories | links | customers | orders | items | sofias_manager |
+------------+-------+-----------+--------+-------+-----------------+
|          5 |    25 |         8 |     12 |    18 |               6 |
+------------+-------+-----------+--------+-------+-----------------+
```

What each test shows: (1) CASCADE from a category to its links; (2) SET NULL on the self-reference; (3) RESTRICT only blocks a parent that HAS children: Hank has none; (4) ON UPDATE CASCADE: Grace's two orders follow her

new id; (5) CASCADE from an order to its lines. The last row proves every ROLLBACK worked.

- LAB 4.2-B: LABS/MODULE-06/L4_2-FOREIGN-KEYS/ORPHAN_AUDIT_STARTER.SQL → ORPHAN_AUDIT_SOLUTION.SQL

The solution prints:

```
+----+-------------+--------+
| id | customer_id | amount |
+----+-------------+--------+
|  3 |          12 |  50.00 |
|  5 |          15 |  20.00 |
+----+-------------+--------+
+--------------------+-----------------------+-------------+-------------+
| CONSTRAINT_NAME    | REFERENCED_TABLE_NAME | DELETE_RULE | UPDATE_RULE |
+--------------------+-----------------------+-------------+-------------+
| fk_cards_customer | customers             | SET NULL    | CASCADE     |
+--------------------+-----------------------+-------------+-------------+
+----+-------------+--------+
| id | customer_id | amount |
+----+-------------+--------+
|  1 |           1 |  25.00 |
|  2 |        NULL |  10.00 |
|  3 |        NULL |  50.00 |
|  4 |           6 |  15.00 |
|  5 |        NULL |  20.00 |
+----+-------------+--------+
+-------+--------+----------+
| cards | total  | assigned |
+-------+--------+----------+
|     5 | 120.00 |        3 |
+-------+--------+----------+
```

Sticking points: running the ALTER before the fix (1452 with a `#sql-...` table name, as in the lesson); filling the gap with `0` instead of NULL in TODO 2 (0 is not a customer either: the ALTER still fails); choosing CASCADE in TODO 3 (deleting a customer would destroy prepaid money). The final row (5 cards, 120.00, 3 assigned) shows the ROLLBACK gave card 2 back to Carol.

- HANDOUT: LABS/MODULE-06/L4_2-FOREIGN-KEYS/DEBUG_FK_ERRORS.SQL (DELIBERATELY BROKEN)

As shipped, MySQL prints:

```
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fail
ERROR 3780 (HY000): Referencing column 'book_id' and referenced column 'id' in foreign
ERROR 1830 (HY000): Column 'customer_id' cannot be NOT NULL: needed in a foreign key c
ERROR 6125 (HY000): Failed to add the foreign key constraint. Missing unique key for c
+--------+-------+
| orders | books |
+--------+-------+
|     12 |    11 |
+--------+-------+
```

1. **Bug A, 1452 (child insert):** customer 42 does not exist. Use a real customer id, or create the customer first.
2. **Bug B, 1451 (parent delete):** 1984 has sold order lines (fk_items_book RESTRICT). Mark it `in_stock = FALSE` instead.
3. **Bug C, 3780 (badly formed key):** `INT UNSIGNED` against `books.id INT`. Copy the parent's exact type: `INT`.
4. **Bug D, 1830 (badly formed key):** SET NULL on a NOT NULL column. Make `customer_id INT` (nullable), or choose RESTRICT.
5. **Bug E, 6125 (badly formed key):** customers.city is not unique, so "the parent" would be ambiguous. Point at a primary key (here a cities table would be the real design).

The final SELECT shows 12 orders and 11 books: every refused statement changed nothing. On MariaDB, Bugs C, D and E all come back as `ERROR 1005 ... errno: 150 "Foreign key constraint is incorrectly formed"`.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 4.1 · TABLE RELATIONSHIPS

1. **B** books.author_id: The foreign key goes on the many side. Each book holds one author_id, and any number of books can hold the same one.
2. **C** A NULL would mean a name did not match, and dropping the text would lose that author: A NULL after the UPDATE means a name found no match in authors. Dropping the text column would then lose that book's author for good, so you check first.
3. **A** The foreign key column is also UNIQUE: The foreign key column is also UNIQUE (or is the primary key), so no parent row can be referenced by more than one child row.
4. **C** Error 1062: duplicate entry for the primary key: PRIMARY KEY (book_id, category_id) allows each pair once, so MySQL reports error 1062, a duplicate entry 2-1 for the primary key.

5 **B** It records what the customer paid then, which must not change when the price does: books.price is today's price; price_at_time records what was paid on the order date. They are two different facts, so old orders keep their real amounts when prices change.

LESSON 4.2 · FOREIGN KEYS & REFERENTIAL INTEGRITY

1 **C** INSERT INTO order_items (order_id, book_id, price_at_time) VALUES (50, 3, 11.99);: Error 1452 is about a child row: order_items.order_id = 50 points at an order that does not exist. Deleting a referenced parent is error 1451.

2 **A** They are deleted too: fk_items_order says ON DELETE CASCADE, so the order's rows in order_items are deleted with it.

3 **B** SET NULL cannot work on a NOT NULL column: SET NULL would write NULL into book_id, which is declared NOT NULL. MySQL reports error 1830 when the key is created.

4 **A** It refuses, like RESTRICT: The default is NO ACTION, which InnoDB checks immediately, exactly like RESTRICT: the delete is refused with error 1451.

5 **C** Nothing: they may contain orphans until you check with a query: Rows inserted while the checks were off were never checked, and turning them back on does not re-check. Run an orphan query per key to be sure.

EXIT TICKET

1 **Where the key goes:** one-to-many: in the many side's table (books.author_id); one-to-one: a foreign key that is also UNIQUE, usually in the optional table (customer_profiles.customer_id); many-to-many: a junction table with two foreign keys and a composite primary key (book_categories).

2 **1452 vs 1451:** 1452 = you were inserting or updating a CHILD and its parent does not exist. 1451 = you were deleting or changing the key of a PARENT that children still point at, under RESTRICT or NO ACTION.

3 **order_items.book_id → books:** RESTRICT (the bookstore's choice): sold books stay on file for the sales history; mark them out of stock. Accept SET NULL only with a reason that the history does not need the book (and note the column is NOT NULL today); CASCADE is a red flag (it deletes sales).

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 A (customers.address_id).** Puts the key on the one side: the most common design error. The question "can one A have many Bs?" fixes it; ask it every time.
- **Q4 D (error 1062).** Expects the database to catch a spelling difference. It cannot: that is exactly why anomalies are dangerous.
- **Q5 B (still one-to-one).** Thinks the foreign key itself limits references. A key checks existence; UNIQUE limits count.

- **Q7 A (1451).** Mixes up parent and child. Reteach with the first words of each message.
- **Q8 B or C.** B: expects the lines to stay as orphans; C: expects every delete with children to be refused. Look up the rule before predicting.
- **Q10 A or B.** Choosing a rule without asking whether the child can exist alone. The two-question diagram is the remedy.
- **Exit ticket 3 with CASCADE.** "It makes deleting easy." Ask what the sales report says next year about a book that was deleted.

INSTRUCTOR KIT · ANSWER KEY

Session 7: Joins and the Part 1 Assessment

Answers and reasoning for every Session 7 item: the warm-up, the 4.3 and Part 1 review self-checks, the predict-then-run items, the lab reference outputs and the debug handout, the Part 1 assessment's reference run, the lesson page's own quiz and the exit ticket.

Session 7 | 12 self-check items | Confidential

WARM-UP

W1

One order has many lines. Where does the foreign key go?

`order_items.order_id` : the key goes on the many side.

W2

Predict the count. (Order 3 has one line; fk_items_order is ON DELETE CASCADE.)

```
START TRANSACTION;
DELETE FROM orders WHERE id = 3;
SELECT COUNT(*) AS items FROM order_items;
ROLLBACK;
```

17: order 3's one line was deleted with it (CASCADE); ROLLBACK put both back.

```
+-------+
| items |
+-------+
|    17 |
+-------+
```

W3

What happens, and why?

```
DELETE FROM authors WHERE id = 6;
```

Error 1451: Tolkien has books, and fk_books_author is ON DELETE RESTRICT ("delete or update a parent row").

```
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constra
```

W4

A key is declared with no ON DELETE. What does MySQL do when a parent with children is deleted?

It refuses: with no rule written the key is NO ACTION, which MySQL enforces exactly like RESTRICT.

W5

How many links?

```
SELECT COUNT(*) AS links
FROM book_categories
WHERE book_id IN (SELECT id FROM books WHERE author_id = 6);
```

4: The Hobbit and Lord of the Rings are each Fiction and Fantasy.

```
+-------+
| links |
+-------+
|     4 |
+-------+
```

4.3 · JOINS: SELF-CHECK

Q1

Eight customers placed 12 orders; Carol and Hank placed none. How many rows does this return?

```
SELECT c.first_name, o.id
FROM customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.id;
```

A) `8`

B) `12`

C) `14`

D) `96`

Why: A LEFT JOIN returns every matching pair (12 orders, each with its customer) plus one row for each customer with no partner (Carol, Hank), padded with NULL: 14.

A is wrong: One row per customer would need GROUP BY; Alice alone has three orders.

B is wrong: 12 is the inner join: it drops Carol and Hank.

D is wrong: 96 (8 × 12) is a cross join: a join without its ON condition.

Q2

Which query lists the books that have never been sold?

A) `FROM books b LEFT JOIN order_items oi ON oi.book_id = b.id WHERE oi.id IS NULL`

B) `FROM books b JOIN order_items oi ON oi.book_id = b.id WHERE oi.id IS NULL`

C) `FROM books b LEFT JOIN order_items oi ON oi.book_id = b.id WHERE b.id IS NULL`

D) `FROM order_items oi LEFT JOIN books b ON oi.book_id = b.id WHERE b.id IS NULL`

Why: The anti-join: keep every book (LEFT JOIN from books), then keep only the rows where the right side found nothing (its primary key is NULL). The answer is The Catcher in the Rye and The Great Gatsby.

B is wrong: An inner join has no unmatched rows, so the IS NULL test keeps nothing.

C is wrong: b.id is never NULL: every row starts from a real book.

D is wrong: This keeps order lines whose book is missing; the foreign key makes that impossible, so it returns nothing.

Q3

Alice (customer 1) placed three orders. What does this print?

```
SELECT COUNT(*) AS rows_returned
FROM orders AS o
JOIN order_items AS oi ON oi.order_id = o.id
WHERE o.customer_id = 1;
```

`4` (one row per order line, not per order)

> **Why:** Joining order_items makes one row per order LINE: order 1 has two lines, orders 2 and 8 one each. Ask "what does one row of this join stand for?" before counting (COUNT(DISTINCT o.id) gives 3).

Q4

What happens when this runs?

```
SELECT id, order_date, first_name
FROM orders
JOIN customers ON orders.customer_id = customers.id;
```

A) It returns 12 rows, using the order's id

B) `ERROR 1052 (23000): Column 'id' in field list is ambiguous`

C) `ERROR 1054 (42S22): Unknown column 'id' in 'field list'`

D) It returns 96 rows: a cross join

> **Why:** Both orders and customers have an id, and MySQL will not guess which one you mean. Qualify it: orders.id (or o.id with an alias). first_name and order_date exist in only one table each, so they are fine.
> **A** is wrong: MySQL never guesses between two tables' columns.
> **C** is wrong: 1054 is a column that exists nowhere (or a table name after an alias); here it exists twice.
> **D** is wrong: The ON clause is there: this is an inner join.

Q5

Carol (3) has no orders; Dave (4) has two. What does this print?

```
SELECT c.first_name, COUNT(o.id) AS orders
FROM customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.id
WHERE c.id IN (3, 4)
GROUP BY c.id, c.first_name
ORDER BY c.id;
```

`Carol | 0` and `Dave | 2`

> **Why:** The LEFT JOIN gives Carol one row padded with NULL. COUNT(o.id) counts only the non-NULL order ids, so Carol gets 0. COUNT(*) would have counted her padded row and said 1 (the lesson's Mistake 4).

Q6

This was meant to list every author with their in-stock books. Isaac Asimov (no books) and the authors of the two out-of-stock books are missing. Why?

```
SELECT a.name, b.title
FROM authors AS a
LEFT JOIN books AS b ON b.author_id = a.id
WHERE b.in_stock = TRUE
ORDER BY a.id, b.title;
```

A) LEFT JOIN cannot be combined with WHERE

B) in_stock is a BOOLEAN, so TRUE must be written as 1

C) The ORDER BY removes the NULL rows

D) The WHERE runs after the join: for the padded rows b.in_stock is NULL, so the LEFT JOIN behaves like an inner join

> **Why:** A condition on the right table belongs in ON when every left row must stay: `LEFT JOIN books b ON b.author_id = a.id AND b.in_stock = TRUE`. Then Asimov, Fitzgerald and Salinger appear with NULL titles (the lesson's Mistake 3).
>
> **A** is wrong: WHERE is fine after a LEFT JOIN; it just filters the joined rows, NULL-padded ones included.
>
> **B** is wrong: TRUE and 1 are the same in MySQL.
>
> **C** is wrong: ORDER BY only sorts.

4.3 · JOINS: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
SELECT COUNT(*) AS rows_returned
FROM customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.id;
```

```
+---------------+
| rows_returned |
+---------------+
|            14 |
+---------------+
```

Why: Twelve order rows (each with its customer) plus Carol and Hank with NULLs: 14.

P2

Predict the output, then run it.

```
SELECT c.first_name, COUNT(*) AS n
FROM customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.id
WHERE c.id IN (3, 6)
GROUP BY c.id, c.first_name
ORDER BY c.id;
```

```
+------------+---+
| first_name | n |
+------------+---+
| Carol      | 1 |
| Frank      | 2 |
+------------+---+
```

Why: Carol has no orders, but the LEFT JOIN gives her one padded row, and COUNT(*) counts rows: 1 (COUNT(o.id) would say 0). Frank has two orders: 2.

PART 1 REVIEW: SELF-CHECK

Q7

Review (Lesson 2.2): which column type fits a book's price?

A) `DECIMAL(8, 2)`

B) `FLOAT`

C) `VARCHAR(10)`

D) `INT`

> **Why:** DECIMAL stores exact decimal values: right for money. The course stores every price as DECIMAL(8, 2).
> **B** is wrong: FLOAT is approximate: sums of prices can be off by fractions of a cent.
> **C** is wrong: Text accepts "about 26" and cannot be summed or compared as a number (Session 1's order sheet).
> **D** is wrong: INT has no cents (unless you store cents, a different design).

Q8

Review (Lessons 2.2 and 2.3): what happens?

```
INSERT INTO customers (first_name, last_name, email)
VALUES ('Ivy', 'Lane', 'alice@example.com');
```

`ERROR 1062 (23000): Duplicate entry 'alice@example.com' for key 'customers.email'`

> **Why:** customers.email is UNIQUE and alice@example.com belongs to Alice: the INSERT is refused and nothing is added. UNIQUE is a constraint on a column, like the PRIMARY KEY on id.

Q9

Review (Lesson 3.1): how many books does this count?

```
SELECT COUNT(*) FROM books WHERE title LIKE 'The %';
```

A) `2`

B) `3`

C) `4`

D) `11`

Why: 'The %' means "starts with The and a space": The Great Gatsby, The Hobbit, The Catcher in the Rye. LIKE ignores case with the course's collation.

A is wrong: Count again: three titles start with "The ".

C is wrong: To Kill a Mockingbird starts with "To", not "The".

D is wrong: 11 is every book; the pattern is a real filter.

Q10

Review (Lessons 3.2 and 3.3): in which order must these clauses be written?

A) SELECT, FROM, GROUP BY, WHERE, HAVING, ORDER BY, LIMIT

B) SELECT, FROM, WHERE, ORDER BY, GROUP BY, HAVING, LIMIT

C) SELECT, FROM, WHERE, HAVING, GROUP BY, ORDER BY, LIMIT

D) SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT

Why: The written order (Some Frogs Will Gladly Hop Over Logs). The run order differs: FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY, LIMIT; with a join, the JOINs belong to FROM.

A is wrong: WHERE comes before GROUP BY: it picks rows before the piles are made.

B is wrong: ORDER BY comes after grouping (it sorts the result).

C is wrong: HAVING after GROUP BY: written the other way it is error 1064.

Q11

Review (Lesson 3.4): what does this print?

```sql
SELECT title
FROM books
WHERE price = (SELECT MIN(price) FROM books);
```

`Pride and Prejudice` **(one row)**

> **Why:** The inner query finds the lowest price (7.99); the outer query returns the row(s) with it. If two books tied, both would come back, unlike ORDER BY price LIMIT 1.

Q12

Review (Lessons 4.1 and 4.2): what is the bookstore's order_items table?

A) A one-to-one extension of orders

B) A copy of books with a quantity column

C) A junction table linking orders and books (many-to-many), carrying facts about each link: quantity and price_at_time

D) A self-referencing table

> **Why:** An order holds many books and a book appears in many orders: two foreign keys in a third table. Its extra columns describe the link (this book in this order). Deleting an order cascades to its lines; deleting a sold book is RESTRICTed.
>
> **A** is wrong: An order has MANY lines.
>
> **B** is wrong: It stores book_id, not a copy of the book; price_at_time is history, not duplication.
>
> **D** is wrong: That is employees (manager_id points at employees).

LAB REFERENCE SOLUTIONS

- LAB 4.3-A: LABS/MODULE-07/L4_3-JOINS/CATALOG_JOINS_STARTER.SQL → CATALOG_JOINS_SOLUTION.SQL

The solution prints:

```
+----------+----------------+---------------+----------+
| order_id | title          | author        | quantity |
+----------+----------------+---------------+----------+
|        7 | 1984           | George Orwell |        1 |
|        7 | Animal Farm    | George Orwell |        1 |
|       12 | Animal Farm    | George Orwell |        1 |
|       12 | Fahrenheit 451 | Ray Bradbury  |        1 |
+----------+----------------+---------------+----------+
+---------------------+-------+----------+
| name                | books | cheapest |
+---------------------+-------+----------+
| George Orwell       |     2 |     8.49 |
| J.R.R. Tolkien      |     2 |    10.99 |
| Aldous Huxley       |     1 |     9.49 |
| F. Scott Fitzgerald |     1 |     8.99 |
+---------------------+-------+----------+
+--------------+-------+----------+
| name         | books | cheapest |
+--------------+-------+----------+
| Isaac Asimov |     0 |     NULL |
+--------------+-------+----------+
+------------+------------+
| first_name | department |
+------------+------------+
| Tom        | sales      |
| Lena       | sales      |
| Sofia      | inventory  |
| Nina       | support    |
| Ben        | sales      |
+------------+------------+
+----------+---------+-------------+
| employee | manager | years_after |
+----------+---------+-------------+
| Sofia    | Marcus  |           5 |
| Tom      | Priya   |           3 |
| Lena     | Priya   |           2 |
| Priya    | Daniel  |           2 |
| Ben      | Priya   |           1 |
| Marcus   | Daniel  |           1 |
+----------+---------+-------------+
```

What each TODO teaches: (1) an INNER JOIN through three tables, one row per order line; (2) LEFT JOIN with GROUP BY and COUNT of the right table's id (Asimov 0, his cheapest NULL); (3) the anti-join on a self-reference, the same five employees Session 6 found with NOT EXISTS; (4) a self-join with a computed column (YEAR differences, not exact years of service). Sticking points: COUNT(*) in TODO 2, testing the left table's id in TODO 3, and 1066 when a self-join has no aliases.

LAB 4.3-B: LABS/MODULE-07/L4_3-JOINS/CITY_SALES_STARTER.SQL → CITY_SALES_SOLUTION.SQL

The solution prints:

```
+-------------+-----------+--------+---------+
| city        | customers | orders | revenue |
+-------------+-----------+--------+---------+
| New York    |         2 |      5 |  100.91 |
| Chicago     |         2 |      4 |   67.44 |
| Houston     |         1 |      2 |   45.96 |
| (no city)   |         2 |      1 |   22.98 |
| Los Angeles |         1 |      0 |    0.00 |
+-------------+-----------+--------+---------+
+-----------------------+
| title                 |
+-----------------------+
| 1984                  |
| Animal Farm           |
| Dune                  |
| To Kill a Mockingbird |
+-----------------------+
+------------+--------------------+
| first_name | orders_since_march |
+------------+--------------------+
| Bob        |                  2 |
| Alice      |                  1 |
| Dave       |                  1 |
| Eve        |                  1 |
| Frank      |                  1 |
| Grace      |                  1 |
| Carol      |                  0 |
| Hank       |                  0 |
+------------+--------------------+
```

Reading it: (1) every city stays thanks to the two LEFT JOINs; COUNT(DISTINCT ...) counts customers and orders, not order lines; the NULL cities form one '(no city)' group (Eve, Hank: one order between them); the four revenues add up to 237.29, the store's total. (2) DISTINCT, because Bob and Grace both bought 1984. (3) The WHERE-versus-ON trap on a new question: as shipped the query shows six customers (Carol and Hank vanish); with the date in ON, eight.

HANDOUT: LABS/MODULE-07/L4_3-JOINS/DEBUG_JOINS.SQL (DELIBERATELY BROKEN)

As shipped, MySQL prints (Bug D's table comes first because it is the only statement that runs):

```
ERROR 1052 (23000): Column 'id' in field list is ambiguous
ERROR 1054 (42S22): Unknown column 'books.title' in 'field list'
ERROR 1066 (42000): Not unique table/alias: 'employees'
+------------+--------+
| first_name | orders |
+------------+--------+
| Alice      |      4 |
| Bob        |      3 |
+------------+--------+
```

1. **Bug A, 1052:** `id` exists in orders and customers. Fix: `orders.id` (or aliases).
2. **Bug B, 1054:** books has the alias b, so `books.title` refers to nothing. Fix: `b.title`.
3. **Bug C, 1066:** employees joined to itself without aliases. Fix: `employees AS e JOIN employees AS m ON e.manager_id = m.id`.
4. **Bug D (silent):** Alice gets 4 and Bob 3: the join to order_items makes one row per order LINE. Fix: `COUNT(DISTINCT o.id)` (3 and 2), or drop the order_items join.

Fixed (first rows only for A and C):

```
+----+------------+----------+
| id | order_date | city     |
+----+------------+----------+
|  1 | 2026-01-15 | New York |
|  4 | 2026-01-22 | Houston  |
|  5 | 2026-02-03 | New York |
+----+------------+----------+
+--------------------+
| title              |
+--------------------+
| Lord of the Rings |
| The Hobbit         |
+--------------------+
+----------+---------+
| employee | manager |
+----------+---------+
| Daniel   | Maria   |
| Priya    | Daniel  |
| Tom      | Priya   |
+----------+---------+
+------------+--------+-------------+
| first_name | orders | order_lines |
+------------+--------+-------------+
| Alice      |      3 |           4 |
| Bob        |      2 |           3 |
+------------+--------+-------------+
```

MariaDB words 1052 and 1054 slightly differently ("in SELECT" instead of "in field list"); the numbers are the same.

- PART 1 ASSESSMENT: LABS/MODULE-07/PART1-ASSESSMENT/PART1_ASSESSMENT_SOLUTION.SQL (REFERENCE RUN)

The instructor's reference solution, run start to finish on MySQL 9.7 (rubric, scoring guidance and the common wrong answers with their real numbers: `final-assessment.html`, Part 1 assessment section). It prints, in order: A2's counts, the classes after A3, then B1 to B8:

```
Database changed
+-------------+-------+
| enrollments | rated |
+-------------+-------+
|          15 |    12 |
+-------------+-------+
+----+---------------------+-------+
| id | title               | price |
+----+---------------------+-------+
|  1 | Fresh Pasta by Hand | 70.00 |
|  2 | Risotto Nights      | 60.00 |
|  3 | Sushi Basics        | 70.00 |
|  4 | Ramen from Scratch  | 60.00 |
|  5 | Sourdough Starter   | 45.00 |
|  6 | French Pastry       | 75.00 |
|  7 | Tacos and Salsas    | 50.00 |
+----+---------------------+-------+
+-------------------+------------+-------+
| title             | class_date | price |
+-------------------+------------+-------+
| Sourdough Starter | 2026-03-21 | 45.00 |
| Tacos and Salsas  | 2026-03-28 | 50.00 |
+-------------------+------------+-------+
+----------+---------+-----------+
| cuisine  | classes | avg_price |
+----------+---------+-----------+
| Italian  |       2 |     65.00 |
| Japanese |       2 |     65.00 |
| Baking   |       2 |     60.00 |
| Mexican  |       1 |     50.00 |
+----------+---------+-----------+
+------------+-----------+-------------+
| first_name | last_name | enrollments |
+------------+-----------+-------------+
| Ana        | Lopez     |           3 |
| Chloe      | Martin    |           3 |
| Emma       | Novak     |           3 |
+------------+-----------+-------------+
+---------------------+-------------+---------+------------+
| title               | enrollments | ratings | avg_rating |
+---------------------+-------------+---------+------------+
| Fresh Pasta by Hand |           3 |       2 |        4.5 |
| Risotto Nights      |           2 |       2 |        4.5 |
| Sushi Basics        |           2 |       2 |        3.5 |
| Ramen from Scratch  |           2 |       2 |        4.0 |
| Sourdough Starter   |           2 |       1 |        5.0 |
| French Pastry       |           2 |       1 |        5.0 |
| Tacos and Salsas    |           2 |       2 |        3.0 |
```

```
+--------------------+------------+---------+------------+
+---------------------+-------+
| title               | price |
+---------------------+-------+
| French Pastry       | 75.00 |
| Fresh Pasta by Hand | 70.00 |
| Sushi Basics        | 70.00 |
+---------------------+-------+
+-------------+-------------+--------------------+--------------+
| enrolled_on | student     | class              | instructor   |
+-------------+-------------+--------------------+--------------+
| 2026-04-01  | Felix Grant | Risotto Nights     | Marco Rossi  |
| 2026-04-02  | Ben Okoro   | Ramen from Scratch | Aiko Tanaka  |
| 2026-04-05  | Felix Grant | Ramen from Scratch | Aiko Tanaka  |
| 2026-04-20  | Chloe Martin | French Pastry     | Lena Fischer |
| 2026-04-22  | Emma Novak  | French Pastry      | Lena Fischer |
+-------------+-------------+--------------------+--------------+
+------------+-----------+-----------------------+
| first_name | last_name | email                 |
+------------+-----------+-----------------------+
| Gina       | Russo     | gina.russo@example.com |
| Hugo       | Weber     | hugo.weber@example.com |
+------------+-----------+-----------------------+
+---------------+-------------+---------+
| name          | enrollments | revenue |
+---------------+-------------+---------+
| Marco Rossi   |           5 |  330.00 |
| Aiko Tanaka   |           4 |  260.00 |
| Lena Fischer  |           4 |  240.00 |
| Diego Morales |           2 |  100.00 |
| Nadia Haddad  |           0 |    0.00 |
+---------------+-------------+---------+
```

Part A3a, run at the prompt: `ERROR 1452 (23000)` for student 9 (a child with no parent) and `ERROR 3819 (HY000): Check constraint 'chk_enrollments_rating' is violated.` for rating 6 (on MariaDB: `ERROR 4025 ... CONSTRAINT chk_enrollments_rating failed`).

THE LESSON PAGE'S OWN QUICK QUIZ

LESSON 4.3 · JOINS

1. **A** 11: An inner join returns one row per matching pair. Every book has exactly one author, so there are 11 pairs, and Asimov, with no partner, is left out.

2. **C** FROM customers c LEFT JOIN orders o ON o.customer_id = c.id WHERE o.id IS NULL: The LEFT JOIN keeps every customer and fills the order columns with NULL when there is none; WHERE o.id IS NULL keeps exactly those customers.

3. **B** The join gave him one row of NULLs, and COUNT(*) counts rows: The LEFT JOIN gives Asimov one row padded with NULLs, and COUNT(*) counts rows. COUNT(b.id) counts only non-NULL book ids and returns 0.

4. **B** Tom, Lena and Ben, each with Priya as manager: It is a self-join: m plays the manager. The rows where the manager is Priya are her three reports, Tom, Lena and Ben (Ben is inactive, but nothing filters on that).

5. **A** In the ON clause, after the key condition: In the ON clause the condition only decides which orders match, so customers without a May order still appear with NULLs. In WHERE it would remove them, turning the LEFT JOIN into an inner join.

EXIT TICKET

1. **JOIN vs LEFT JOIN:** JOIN returns one row per customer-order pair (12 rows; Carol and Hank, who never ordered, are not there). LEFT JOIN also keeps every customer without an order, once, with NULL in the order columns (14 rows).

2. **Hardest task:** a reflection. Tally by task: many A1 answers point to Lesson 2.2 and 4.2 (constraints, keys); B4 to the NULL rule (3.3); B7 and B8 to the anti-join and LEFT JOIN with GROUP BY (4.3). If Part 2 follows, open Session 8's refresher with the most frequent one.

3. **Use and practice:** a reflection; read them for the course evaluation.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 B (12).** Thinks LEFT JOIN returns the same rows as JOIN when "every order has a customer". It is the customers without orders that LEFT JOIN adds.
- **Q2 B.** Uses IS NULL after an inner join: has the pattern's words but not its logic. Draw the NULL-padded rows.
- **Q3 3.** Counts orders, not order lines: the fan-out habit is not there yet. The board question is the fix.
- **Q6 B or C.** Looks for a syntax cause of a logic bug. Run it without the WHERE and watch the NULL rows appear.
- **Q10 A or C.** The written order is still shaky; HAVING before GROUP BY is a 1064 they will meet again.
- **Q12 A or B.** Has not connected order_items with junction tables. Session 6's relationship map is the remedy.

MYSQL FOUNDATIONS · ANSWER KEY

Part 2: Beyond the Basics

Sessions 8 to 14. Modules 5 to 8: CTEs and window functions, indexes, views and design, stored programs, transactions, JSON, users and backups, ending with the final project.

INSTRUCTOR KIT · ANSWER KEY

Session 8: CTEs and Window Functions

Answers and reasoning for every Session 8 item: the Part 1 refresher, both self-checks, the predict-then-run items, the lab reference outputs (environment check included), the debug handouts, the lesson pages' own quizzes and the exit ticket.

Session 8 | 12 self-check items | Confidential

START HERE · PART 1 REFRESHER

Answers to W1 to W5 (not graded). W1 and W2 with MySQL's real output on the freshly loaded bookstore:

W1

Which customers have never placed an order? Say the query shape, then the names.

A LEFT JOIN from customers to orders, keeping the customers whose orders side is NULL (or HAVING COUNT(o.id) = 0): Carol and Hank. Lesson 4.3, the anti-join.

```
+------------+--------+
| first_name | orders |
+------------+--------+
| Carol      |      0 |
| Hank       |      0 |
+------------+--------+
```

W2

employees has a manager_id. How do you show each employee next to their manager's name? Who is Sofia's manager?

A self-join: the table twice with two aliases, ON m.id = e.manager_id. Sofia (id 7) reports to Marcus.

```
+----------+---------+
| employee | manager |
+----------+---------+
| Sofia    | Marcus  |
+----------+---------+
```

W3

SELECT a.name, COUNT(*) FROM authors a LEFT JOIN books b ON b.author_id = a.id GROUP BY a.id, a.name: what does it say for Isaac Asimov, who has no books, and how do you fix it?

1, because COUNT(*) counts the NULL-padded row the LEFT JOIN made. Count a column of the right table, COUNT(b.id), which skips NULL: 0. (Lesson 4.3, Common Mistake 4.)

W4

Books per author and copies sold in one query: Orwell shows 5 books, but he wrote 2. What happened?

Fan-out: joining order_items repeats each book once per order line. COUNT(DISTINCT b.id) fixes it. Ask what one row of the join stands for. (Lesson 4.3, Common Mistake 5.)

W5

A subquery in FROM (a derived table) fails with error 1248. What is missing?

An alias: every derived table needs its own name, (...) AS t (Lesson 3.4). Today's CTEs give such steps a name up front, and read top to bottom.

5.1 · CTES & RECURSIVE QUERIES: SELF-CHECK

Q1

A statement starts `WITH monthly AS (...)`. How long does `monthly` exist?

A) Until you disconnect

B) Only while that one statement runs

C) Until you `DROP` it

D) Until the next `COMMIT`

Why: A CTE is a named result that belongs to the one statement it is written in. After the semicolon it is gone: a second `SELECT * FROM monthly;` answers 1146 (the lesson's Common Mistake 4).
A is wrong: That is a user variable (@name) or a temporary table.
C is wrong: Only stored objects (tables, views) are dropped; a CTE is never stored.
D is wrong: Transactions have nothing to do with a CTE's lifetime.

Q2

What does this print?

```
WITH RECURSIVE s AS (
    SELECT 2 AS n
    UNION ALL
    SELECT n * 2 FROM s WHERE n < 20
)
SELECT GROUP_CONCAT(n ORDER BY n) AS series FROM s;
```

`2,4,8,16,32` **(one row, column** `series` **)**

Why: Each round doubles the row the previous round made: 2, 4, 8, 16. The round that reads 16 still passes n < 20 and makes 32; the round that reads 32 fails the test, makes no row, and the recursion stops. The stop condition tests the row a round reads, so the last row can be past the limit.

Q3

A CTE named `org` **joins to itself, but the statement starts** `WITH org AS (` **without RECURSIVE. What happens?**

A) `ERROR 1146 (42S02): Table 'bookstore.org' doesn't exist`

B) `ERROR 3636 (HY000): Recursive query aborted after 1001 iterations`

C) It runs, but only the anchor's rows come back

D) `ERROR 1406 (22001): Data too long for column`

Why: Without RECURSIVE a CTE cannot see its own name, so the JOIN inside it looks for a real table called org (the live demo's last step, and the debug handout's query A).

B is wrong: 3636 needs a working recursion that never stops.

C is wrong: MySQL does not quietly drop the recursive member; it refuses the statement.

D is wrong: 1406 is a column too narrow for what the recursive member builds.

Q4

A recursive CTE named `chain` starts from one employee. Which condition in the recursive member walks UP to the owner?

A) `JOIN chain c ON e.manager_id = c.id`

B) `JOIN chain c ON e.id = c.id`

C) `JOIN chain c ON e.id = c.manager_id`

D) `JOIN chain c ON e.manager_id = c.manager_id`

> **Why:** "Who is this person's manager?": the next row is the employee whose id is the manager_id of the row found last. That is how the demo climbs from Sofia to Marcus, Daniel and Maria.
> **A** is wrong: That walks DOWN: "whose manager is this person?" (the org chart).
> **B** is wrong: It finds the same person again every round: the recursion would never move.
> **D** is wrong: It finds colleagues with the same manager, not the manager.

Q5

The bookstore has 10 employees. This query prints 30. What is wrong?

```
WITH RECURSIVE org AS (
    SELECT id, 1 AS level FROM employees
    UNION ALL
    SELECT e.id, o.level + 1
    FROM employees e JOIN org o ON e.manager_id = o.id
)
SELECT COUNT(*) AS rows_back FROM org;
```

A) RECURSIVE is missing

B) `UNION ALL` should be `UNION`

C) The recursion limit is too low

D) The anchor has no WHERE, so every employee starts a walk of their own

> **Why:** With no `WHERE manager_id IS NULL` in the anchor, all ten employees are starting points, and each deeper person is found again under every manager above them. No error: the lesson's silent Common Mistake 5. Check with COUNT(*) against COUNT(DISTINCT id).
> **A** is wrong: RECURSIVE is there, and without it the query would fail with 1146.
> **B** is wrong: UNION would hide some duplicates and the wrong anchor with them: fix the anchor instead.
> **C** is wrong: The limit is 1000 rounds; this walk needs four.

Q6

In one sentence: why does a monthly report LEFT JOIN a generated calendar of months to the orders, instead of grouping the orders by month directly?

> **GROUP BY over orders can only show months that have orders; the recursive calendar has every month, and the LEFT JOIN keeps the empty ones (July shows 0)**

> **Why:** Real data has holes. The series comes first, then the data is hung on it; `COUNT(o.id)` counts 0 for a month with no matching order.

5.1 · CTES & RECURSIVE QUERIES: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
WITH RECURSIVE countdown AS (
    SELECT 10 AS n
    UNION ALL
    SELECT n - 3 FROM countdown WHERE n > 3
)
SELECT n FROM countdown;
```

```
+------+
| n    |
+------+
|   10 |
|    7 |
|    4 |
|    1 |
+------+
```

> **Why:** Anchor 10; each round subtracts 3 while the row it reads is above 3: 7, 4, 1. The round that reads 1 fails n > 3 and stops.

P2

Predict the output, then run it.

```
WITH by_city AS (
    SELECT city, COUNT(*) AS customers
    FROM customers
    GROUP BY city
)
SELECT COUNT(*) AS city_groups, SUM(customers) AS customers
FROM by_city;
```

```
+-------------+-----------+
| city_groups | customers |
+-------------+-----------+
|           5 |         8 |
+-------------+-----------+
```

Why: GROUP BY city makes one group per city, and the two customers with no city (Eve, Hank) form one more group: NULL. Five groups, eight customers. (The column is not called `groups`: GROUPS is a reserved word in MySQL 8 and later, error 1064.)

5.2 · WINDOW FUNCTIONS: SELF-CHECK

Q7

`employees` **has 10 rows in 4 departments. How many rows does this return?** `SELECT first_name, AVG(salary) OVER (PARTITION BY department) FROM employees;`

A) 4

B) 1

C) 10

D) None: `ERROR 1140` (a column without GROUP BY)

Why: A window function never collapses rows: every employee comes back, each with their own department's average. GROUP BY department would return 4.

A is wrong: That is GROUP BY department.

B is wrong: That is AVG(salary) without OVER.

D is wrong: OVER makes the mix of a plain column and an aggregate legal.

Q8

Copies sold, best first, are 5, 4, 4 and 3. What do `RANK()` and `DENSE_RANK()` (both `OVER (ORDER BY copies DESC)`) give the book with 3?

`RANK` 4, `DENSE_RANK` 3

> **Why:** RANK skips after a tie (1, 2, 2, 4: three books are ahead); DENSE_RANK does not (1, 2, 2, 3). ROW_NUMBER would give 4 too, but it would also give the tied books 2 and 3.

Q9

`SELECT title FROM books WHERE RANK() OVER (ORDER BY price DESC) <= 3;` fails with error 3593. What is the standard fix?

A) Move the condition to `HAVING`

B) Compute the rank in a CTE, then filter on it in the outer query

C) Replace `WHERE` with `QUALIFY`

D) Give the rank a column alias and filter on the alias in WHERE

> **Why:** Window functions run after WHERE, GROUP BY and HAVING, so none of those can see them. One step computes the window, the next filters it: a CTE (or a derived table) is that next step.
> **A** is wrong: HAVING runs before the windows too (3593 again).
> **C** is wrong: QUALIFY is error 6037 on MySQL 9.7 (hypergraph optimizer only) and 1064 on MariaDB.
> **D** is wrong: WHERE cannot see SELECT aliases: error 1054, Unknown column.

Q10

In `SUM(price) OVER (ORDER BY price)`, The Catcher in the Rye and The Great Gatsby (both 8.99) show the same running total, 34.46. Why?

A) `SUM` skips duplicate values

B) Prices are sorted as text

C) The window has no `PARTITION BY`

D) The default frame is RANGE: rows with the same ORDER BY value are added in one step

> **Why:** With ORDER BY and no frame, the frame is RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW, which lumps ties together. Write `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` and make the order unique (`ORDER BY price, title`): 25.47 then 34.46.
> **A** is wrong: Both prices are added (34.46 includes both); the problem is when.
> **B** is wrong: price is DECIMAL; the order is numeric.
> **C** is wrong: PARTITION BY would restart the total per group; it does not change how ties are handled.

Q11

Bob (customer 2) has two orders. What does this print?

```sql
SELECT id,
       order_date,
       LEAD(order_date) OVER (ORDER BY order_date) AS next_order
FROM orders
WHERE customer_id = 2
ORDER BY order_date;
```

Two rows: order 3, 2026-03-10, next_order 2026-03-28; order 7, 2026-03-28, next_order `NULL`

> **Why:** LEAD reads the next row of the window. Order 7 is Bob's last, so there is no next row and LEAD returns NULL (a third argument, LEAD(order_date, 1, ...), would replace it).

Q12

You need exactly one top earner per department, even if two salaries ever tie. Which function numbers the rows?

A) `ROW_NUMBER()` with a tie-breaker, e.g. `ORDER BY salary DESC, id`

B) `RANK()`

C) `DENSE_RANK()`

D) `NTILE(1)`

> **Why:** ROW_NUMBER never ties, so pos = 1 is exactly one row per partition; the tie-breaker makes the choice repeatable. RANK and DENSE_RANK would return both tied people.
> **B** is wrong: Two tied top salaries both get rank 1.
> **C** is wrong: Same: both get 1.
> **D** is wrong: NTILE(1) puts every row in bucket 1.

5.2 · WINDOW FUNCTIONS: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
SELECT first_name,
       COUNT(*) OVER ()                        AS everyone,
       COUNT(*) OVER (PARTITION BY department) AS in_dept
FROM employees
WHERE department = 'support'
ORDER BY first_name;
```

```
+------------+----------+---------+
| first_name | everyone | in_dept |
+------------+----------+---------+
| James      |        2 |       2 |
| Nina       |        2 |       2 |
+------------+----------+---------+
```

> **Why:** WHERE keeps only the two support employees BEFORE the windows are computed, so even OVER () sees two rows. Windows never see rows WHERE removed.

P4

Predict the output, then run it.

```
SELECT id,
       LAG(id, 1, 0) OVER (ORDER BY id) AS previous_id
FROM orders
WHERE customer_id = 1
ORDER BY id;
```

```
+----+-------------+
| id | previous_id |
+----+-------------+
|  1 |           0 |
|  2 |           1 |
|  8 |           2 |
+----+-------------+
```

> **Why:** LAG's third argument replaces the NULL on the first row: 0. Alice's orders are 1, 2 and 8, so the previous ids are 0, 1, 2.

LAB REFERENCE SOLUTIONS

- PART 2 ENVIRONMENT CHECK: LABS/MODULE-08/L5_0-PART2-START/CHECK_ENVIRONMENT.SQL (+ .SH / .PS1)

A tool, not an exercise. On a ready MySQL 9.7 server with a freshly loaded bookstore it prints (the `.sh` and `.ps1` helpers first print the client version line):

```
+----------------+------------------------------+
| server_version | edition                      |
+----------------+------------------------------+
| 9.7.2          | MySQL Community Server - GPL |
+----------------+------------------------------+
+-------------------------------------------------+
| server_check                                    |
+-------------------------------------------------+
| OK     MySQL 9.7 LTS, the version the course uses |
+-------------------------------------------------+
+-------------------------------+-------------------------------+
| charset_check                 | sql_mode_check                |
+-------------------------------+-------------------------------+
| OK     utf8mb4 (full Unicode) | OK     default strict SQL mode |
+-------------------------------+-------------------------------+
+-----------------------------------------------------------------+
| write_check                                                     |
+-----------------------------------------------------------------+
| OK     created a database and a table, and read back: it works |
+-----------------------------------------------------------------+
+----------------------------------+
| bookstore_check                  |
+----------------------------------+
| OK     bookstore has its 8 tables |
+----------------------------------+
+-------------------------------------------------------------------------+
| data_check                                                              |
+-------------------------------------------------------------------------+
| OK     fresh row counts: 11 books, 8 customers, 12 orders, 10 employees |
+-------------------------------------------------------------------------+
+------------------+
| verdict          |
+------------------+
| READY for Part 2 |
+------------------+
```

Without bookstore: `NOT READY no bookstore database` and the verdict `NOT READY: load bookstore.sql`. With a leftover ninth table: `NOT READY bookstore has 9 tables`. With changed rows (for example after Lesson 2.3 practice): `CHECK not a fresh copy`. On 8.4 or MariaDB the verdict is `READY WITH NOTES`.

- `PART 1 REFRESHER: LABS/MODULE-08/L5_0-PART2-START/REFRESHER_STARTER.SQL → REFRESHER_SOLUTION.SQL`

The solution prints:

```
+----------+------------+--------------------+----------+
| order_id | first_name | title              | quantity |
+----------+------------+--------------------+----------+
|        1 | Alice      | 1984               |        2 |
|        1 | Alice      | Dune               |        1 |
|        4 | Dave       | Lord of the Rings  |        1 |
|        4 | Dave       | The Hobbit         |        1 |
+----------+------------+--------------------+----------+
+------------+--------+
| first_name | orders |
+------------+--------+
| Alice      |      3 |
| Bob        |      2 |
| Dave       |      2 |
| Frank      |      2 |
| Grace      |      2 |
| Eve        |      1 |
| Carol      |      0 |
| Hank       |      0 |
+------------+--------+
+------------+-------+
| first_name | spent |
+------------+-------+
| Alice      | 58.95 |
| Dave       | 45.96 |
| Frank      | 41.96 |
+------------+-------+
+----------+---------+
| employee | manager |
+----------+---------+
| Ben      | Priya   |
| Lena     | Priya   |
| Priya    | Daniel  |
| Tom      | Priya   |
+----------+---------+
```

Sticking points: TODO 1 learners add the join but keep `ORDER BY o.id, oi.id` (fine, a different order); TODO 2 with LEFT JOIN but COUNT(*) gives Carol and Hank 1 (W3); TODO 4 the ON line `m.id = e.manager_id` versus `e.id = m.manager_id` (the second lists each manager's reports instead).

LAB 5.1-A: LABS/MODULE-08/L5_1-CTES/STAFF_TREE_STARTER.SQL → STAFF_TREE_SOLUTION.SQL

The solution prints:

```
+-----------+--------+
| month_num | copies |
+-----------+--------+
|         1 |      5 |
|         2 |      5 |
|         3 |      3 |
|         4 |      3 |
|         5 |      3 |
|         6 |      2 |
|         7 |      0 |
|         8 |      0 |
|         9 |      0 |
|        10 |      0 |
|        11 |      0 |
|        12 |      0 |
+-----------+--------+
+-------+------------+----------+
| level | first_name | division |
+-------+------------+----------+
|     1 | Maria      | (owner)  |
|     2 | Daniel     | Daniel   |
|     3 | Marcus     | Daniel   |
|     3 | Priya      | Daniel   |
|     4 | Ben        | Daniel   |
|     4 | Lena       | Daniel   |
|     4 | Sofia      | Daniel   |
|     4 | Tom        | Daniel   |
|     2 | James      | James    |
|     3 | Nina       | James    |
+-------+------------+----------+
+----------+--------+-----------+
| division | people | payroll   |
+----------+--------+-----------+
| Daniel   |      7 | 310500.00 |
| James    |      2 |  88500.00 |
+----------+--------+-----------+
```

Reading it: the month series has 12 rows because the round that reads 12 fails `month_num < 12`; months 7 to 12 have no orders, and the LEFT JOINs keep them with COALESCE turning NULL into 0. In the tree, the anchor's `CAST(NULL AS CHAR(50))` makes the division column wide enough (a bare NULL would make a useless column type); level 2 starts a division with COALESCE, everyone below copies it. The aggregate runs in the main query after the recursion: Daniel's division is everyone under him plus himself (7 people), James's is 2; Maria (the owner) has no division and is filtered out.

- LAB 5.1-B: LABS/MODULE-08/L5_1-CTES/CATEGORY_STEPS_STARTER.SQL → CATEGORY_STEPS_SOLUTION.SQL

The solution prints:

```
+-----------------+--------+---------+
| category        | copies | revenue |
+-----------------+--------+---------+
| Fiction         |     21 |  237.29 |
| Classic         |     10 |   96.90 |
| Science Fiction |      6 |   75.44 |
| Dystopian       |      7 |   70.43 |
| Fantasy         |      5 |   64.95 |
+-----------------+--------+---------+
+----------+---------+-------------+
| category | revenue | avg_revenue |
+----------+---------+-------------+
| Fiction  |  237.29 |      109.00 |
+----------+---------+-------------+
+-----------+--------+---------+------------+
| month_num | orders | busiest | is_busiest |
+-----------+--------+---------+------------+
|         1 |      2 |       3 |          0 |
|         2 |      3 |       3 |          1 |
|         3 |      2 |       3 |          0 |
|         4 |      2 |       3 |          0 |
|         5 |      2 |       3 |          0 |
|         6 |      1 |       3 |          0 |
+-----------+--------+---------+------------+
```

Every book is in Fiction, so Fiction's revenue (237.29) is the whole half year, and the other categories overlap (a book in two categories counts in both). Only Fiction beats the average category (109.00): a good moment to ask whether "average category" is a meaningful number when categories overlap. Query 3 reads monthly three times; `is_busiest` is a comparison, so it prints 1 or 0.

HANDOUT: LABS/MODULE-08/L5_1-CTES/DEBUG_CTES.SQL (DELIBERATELY BROKEN)

As shipped, MySQL prints:

```
ERROR 1146 (42S02): Table 'bookstore.countdown' doesn't exist
ERROR 3636 (HY000): Recursive query aborted after 1001 iterations. Try increasing @@cte
ERROR 1406 (22001): Data too long for column 'path' at row 1
```

1. **A, 1146:** the CTE names itself without RECURSIVE. Fix: `WITH RECURSIVE countdown AS (...)`: 3, 2, 1.
2. **B, 3636:** nothing stops the series. Fix: `SELECT d + INTERVAL 1 DAY FROM days WHERE d < '2026-12-31'`: 365.
3. **C, 1406:** the anchor's literal `'Maria'` fixed a five-character column, and `Maria > Daniel` does not fit. Fix: `CAST('Maria' AS CHAR(200))` or, better, `CAST(first_name AS CHAR(200))`.

The checker runs these in a scratch database, so its 1146 message names that database instead of `bookstore`. On MariaDB, query B does not fail: it returns 1001 with warning 1931.

- LAB 5.2-A: LABS/MODULE-08/L5_2-WINDOW-FUNCTIONS/LEADERBOARDS_STARTER.SQL → LEADERBOARDS_SOLUTION.SQL

The solution prints:

```
+------------+--------+---------+----------+-----------+
| first_name | copies | row_num | rank_num | dense_num |
+------------+--------+---------+----------+-----------+
| Alice      |      5 |       1 |        1 |         1 |
| Dave       |      4 |       2 |        2 |         2 |
| Frank      |      4 |       3 |        2 |         2 |
| Bob        |      3 |       4 |        4 |         3 |
| Grace      |      3 |       5 |        4 |         3 |
| Eve        |      2 |       6 |        6 |         4 |
+------------+--------+---------+----------+-----------+
+-----------------+-----+---------------------+-------+
| category        | pos | title               | price |
+-----------------+-----+---------------------+-------+
| Classic         |   1 | Pride and Prejudice |  7.99 |
| Classic         |   2 | Animal Farm         |  8.49 |
| Dystopian       |   1 | Brave New World     |  9.49 |
| Dystopian       |   2 | 1984                |  9.99 |
| Fantasy         |   1 | The Hobbit          | 10.99 |
| Fantasy         |   2 | Lord of the Rings   | 15.99 |
| Fiction         |   1 | Pride and Prejudice |  7.99 |
| Fiction         |   2 | Animal Farm         |  8.49 |
| Science Fiction |   1 | Brave New World     |  9.49 |
| Science Fiction |   2 | Fahrenheit 451      | 10.49 |
+-----------------+-----+---------------------+-------+
+------------+------------+----------+--------------+-------------+
| department | first_name | salary   | dept_payroll | pct_of_dept |
+------------+------------+----------+--------------+-------------+
| sales      | Priya      | 46000.00 |    158500.00 |        29.0 |
| sales      | Lena       | 39500.00 |    158500.00 |        24.9 |
| sales      | Tom        | 38000.00 |    158500.00 |        24.0 |
| sales      | Ben        | 35000.00 |    158500.00 |        22.1 |
| inventory  | Marcus     | 47000.00 |     84000.00 |        56.0 |
| inventory  | Sofia      | 37000.00 |     84000.00 |        44.0 |
+------------+------------+----------+--------------+-------------+
```

Reading it: RANK skips after the ties (1, 2, 2, 4, 4, 6), DENSE_RANK does not (1, 2, 2, 3, 3, 4); ROW_NUMBER separates ties by the tie-breaker (first name). In the top-2 query Pride and Prejudice appears in Classic and in Fiction: a book in two categories is ranked in both partitions. The payroll shares add up to 100 within each department (sales 29.0 + 24.9 + 24.0 + 22.1).

LAB 5.2-B: LABS/MODULE-08/L5_2-WINDOW-FUNCTIONS/ORDER_TRENDS_STARTER.SQL → ORDER_TRENDS_SOLUTION.SQL

The solution prints:

```
+----+------------+-------+---------------+
| id | order_date | total | running_total |
+----+------------+-------+---------------+
|  1 | 2026-01-15 | 34.97 |         34.97 |
|  4 | 2026-01-22 | 26.98 |         61.95 |
|  5 | 2026-02-03 | 19.98 |         81.93 |
|  6 | 2026-02-14 | 23.98 |        105.91 |
|  2 | 2026-02-20 |  7.99 |        113.90 |
|  3 | 2026-03-10 | 14.99 |        128.89 |
|  7 | 2026-03-28 | 18.48 |        147.37 |
|  8 | 2026-04-05 | 15.99 |        163.36 |
|  9 | 2026-04-18 | 22.98 |        186.34 |
| 10 | 2026-05-02 | 21.98 |        208.32 |
| 11 | 2026-05-19 |  9.99 |        218.31 |
| 12 | 2026-06-07 | 18.98 |        237.29 |
+----+------------+-------+---------------+
+------------+----------+------------+-------+------------------+
| first_name | order_no | order_date | total | change_from_last |
+------------+----------+------------+-------+------------------+
| Alice      |        1 | 2026-01-15 | 34.97 |             NULL |
| Alice      |        2 | 2026-02-20 |  7.99 |           -26.98 |
| Alice      |        3 | 2026-04-05 | 15.99 |             8.00 |
| Bob        |        1 | 2026-03-10 | 14.99 |             NULL |
| Bob        |        2 | 2026-03-28 | 18.48 |             3.49 |
| Dave       |        1 | 2026-01-22 | 26.98 |             NULL |
| Dave       |        2 | 2026-06-07 | 18.98 |            -8.00 |
| Eve        |        1 | 2026-04-18 | 22.98 |             NULL |
| Frank      |        1 | 2026-02-03 | 19.98 |             NULL |
| Frank      |        2 | 2026-05-02 | 21.98 |             2.00 |
| Grace      |        1 | 2026-02-14 | 23.98 |             NULL |
| Grace      |        2 | 2026-05-19 |  9.99 |           -13.99 |
+------------+----------+------------+-------+------------------+
+------------+----------+------------+
| first_name | order_id | order_date |
+------------+----------+------------+
| Alice      |        1 | 2026-01-15 |
| Dave       |        4 | 2026-01-22 |
| Frank      |        5 | 2026-02-03 |
| Grace      |        6 | 2026-02-14 |
| Bob        |        3 | 2026-03-10 |
| Eve        |        9 | 2026-04-18 |
+------------+----------+------------+
```

The running total passes 61.95 after order 4 (all of January) and ends at 237.29, the lesson's half-year revenue. `ORDER BY order_date, id` with ROWS makes the order unique, so no two orders share a running total. In query 2 every customer's first row is NULL: LAG has no previous row in that customer's partition. Query 3 is the top-1-per-group pattern; WHERE order_no = 1 works only because the numbering happened in the CTE.

HANDOUT: LABS/MODULE-08/L5_2-WINDOW-FUNCTIONS/DEBUG_WINDOWS.SQL (DELIBERATELY BROKEN)

As shipped, MySQL prints:

```
ERROR 3593 (HY000): You cannot use the window function 'row_number' in this context.'
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresp
FROM books
ORDER BY price DESC
LIMIT 3' at line 1
ERROR 6037 (HY000): 'QUALIFY clause' can be used only if the hypergraph optimizer is e
```

1. **A, 3593:** a window function in WHERE. Fix: `WITH priced AS (SELECT title, price, ROW_NUMBER() OVER (ORDER BY price DESC) AS pos FROM books) SELECT title, price FROM priced WHERE pos <= 3 ORDER BY pos;` (Lord of the Rings, Dune, To Kill a Mockingbird).
2. **B, 1064:** `rank` is a reserved word. Fix: `AS rank_num` (or backticks).
3. **C, 6037:** QUALIFY runs only under the hypergraph optimizer. Fix: the CTE pattern from A.

On MariaDB: A is error 4015, B runs (MariaDB still accepts `AS rank`), C is 1064.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 5.1 · CTES & RECURSIVE QUERIES

1. **B** Error 1146: the table does not exist: A CTE exists only inside the statement it is written in. After the semicolon it is gone, so MySQL looks for a real table named monthly and reports error 1146.
2. **A** The anchor, before UNION ALL: The anchor, the query before UNION ALL, produces the starting rows once. The recursive member then runs round after round until a round produces no rows.
3. **C** 4: The anchor gives 1, then the rounds give 2, 3 and 4. The round that reads 4 fails n less than 4, produces nothing, and the recursion stops: 1, 2, 3, 4.
4. **B** Widen the column in the anchor, for example CAST('Maria' AS CHAR(200)): The anchor fixes each column's type and width, and the recursive member can never widen it. CAST('Maria' AS CHAR(200)) in the anchor gives the path room to grow.
5. **C** In a view, stored once in the database: A view is stored in the database, so every script and every person can read it by name. CTEs are best for steps that belong to one query.

LESSON 5.2 · WINDOW FUNCTIONS

1. **C** 11: A window function never collapses rows. OVER () makes the whole result the window, so each of the 11 books gets the same average next to its title.

2. **A** 4: RANK gives the two books with 3 copies the same rank, 2, then skips: three books are ahead of the one with 2 copies, so it gets 4. DENSE_RANK would give 3.

3. **B** Compute the row number in a CTE and filter it in the outer query: Window functions are computed after WHERE, GROUP BY and HAVING. Compute ROW_NUMBER in a CTE, then filter its result with WHERE in the outer query.

4. **C** NULL: January is the first row of the window, so there is no previous row and LAG returns NULL. LAG(revenue, 1, 0) would return 0 instead.

5. **A** 1: The frame asks for two rows before January, but there are none, so the frame holds only January itself. February averages two months, and from March on every row averages three.

EXIT TICKET

1. **Recursive skeleton:** `WITH RECURSIVE s AS (SELECT 1 AS n UNION ALL SELECT n + 1 FROM s WHERE n < 10) SELECT n FROM s;` Anchor = `SELECT 1 AS n` (runs once); recursive member = `SELECT n + 1 FROM s ...` (runs per round); stop condition = `WHERE n < 10`. Accept `<= 9`; reject a stop condition in the outer SELECT (it would not stop the recursion).

2. **GROUP BY vs PARTITION BY:** one row per department versus every employee row with the department's figure added. Accept "collapses" versus "keeps the rows".

3. **WHERE and windows:** windows are computed after WHERE (and GROUP BY, HAVING), so WHERE cannot see them (3593); compute the window in a CTE (or derived table) and filter in the outer query.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 A ("until you disconnect").** Mixes a CTE with a user variable or a temporary table. Show the 1146 from the demo's step 5 again.
- **Q2** `2,4,8,16` **.** Reads the stop condition as a filter on the output. Draw the rounds: the round that reads 16 still makes 32.
- **Q4 A.** Walks down instead of up: the most common hierarchy slip. Say the two questions aloud: "whose manager is this?" (down) versus "who is this person's manager?" (up).
- **Q7 A (4 rows).** Still thinks of grouping. Run it: ten rows.
- **Q8 "3 and 3" or "4 and 4".** Mixed up which function skips. The board table: 1, 2, 2, 4 / 1, 2, 2, 3.
- **Q9 A (HAVING).** HAVING runs before windows too; the CTE is the only portable fix.

- **Exit ticket 1 with the WHERE in the outer SELECT.** The recursion has no stop condition at all (3636). Worth two minutes at the start of Session 9.

INSTRUCTOR KIT · ANSWER KEY

Session 9: Indexes and Views

Answers and reasoning for every Session 9 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs, both debug handouts, the lesson pages' own quizzes and the exit ticket.

Session 9 | 12 self-check items | Confidential

WARM-UP

Answers to W1 to W5, with MySQL's real output where the question shows code:

W1

What does this print?

```
WITH RECURSIVE s AS (
    SELECT 1 AS n UNION ALL SELECT n + 2 FROM s WHERE n < 7
)
SELECT GROUP_CONCAT(n ORDER BY n) AS odd FROM s;
```

1,3,5,7: the round that reads 7 fails n < 7 and stops.

```
+---------+
| odd     |
+---------+
| 1,3,5,7 |
+---------+
```

W2

Which price_level do The Catcher in the Rye and The Great Gatsby (both 8.99) get, and Animal Farm after them?

```
SELECT title, price,
       DENSE_RANK() OVER (ORDER BY price DESC) AS price_level
FROM books
WHERE price < 9.50
ORDER BY price DESC, title;
```

Both 2, Animal Farm 3: DENSE_RANK ties without a gap (RANK would give Animal Farm 4).

```
+------------------------+-------+-------------+
| title                  | price | price_level |
+------------------------+-------+-------------+
| Brave New World        |  9.49 |           1 |
| The Catcher in the Rye |  8.99 |           2 |
| The Great Gatsby       |  8.99 |           2 |
| Animal Farm            |  8.49 |           3 |
| Pride and Prejudice    |  7.99 |           4 |
+------------------------+-------+-------------+
```

W3

Why can't WHERE filter on ROW_NUMBER(), and what do you write instead?

Windows are computed after WHERE (3593): number the rows in a CTE, filter in the outer query.

W4

In a recursive CTE over employees, which join walks UP the hierarchy?

e.id = cte.manager_id ("who is this person's manager?"); e.manager_id = cte.id walks down.

W5

AVG(salary) with GROUP BY department versus AVG(salary) OVER (PARTITION BY department): how many rows from the 10 employees?

4 versus 10: GROUP BY collapses, OVER keeps every row.

6.1 · INDEXES & QUERY OPTIMIZATION: SELF-CHECK

Q1

Which index does every InnoDB table have, even though you never created one by name?

A) One index per column

B) The primary key: the rows themselves are stored in its order (the clustered index)

C) An index on every VARCHAR column

D) None: MySQL scans every table until you add indexes

> **Why:** InnoDB stores the rows in a B-tree sorted by the primary key, so WHERE id = ... is always fast. UNIQUE constraints and foreign keys add more indexes automatically (email, fk_orders_customer ...).
> **A** is wrong: Only the columns you (or a constraint) choose are indexed.
> **C** is wrong: Types do not decide it; constraints and CREATE INDEX do.
> **D** is wrong: The primary key is always there.

Q2

`sales_log` has an index on `(customer_id, sold_on)` and no other index. Which query can NOT seek with it?

A) `WHERE customer_id = 3`

B) `WHERE customer_id = 3 AND sold_on >= '2025-03-01'`

C) `WHERE sold_on >= '2025-03-01' AND customer_id = 3`

D) `WHERE sold_on BETWEEN '2025-03-01' AND '2025-03-31'`

> **Why:** A composite index serves its leftmost prefix. The dates are sorted only within each customer, so a date range alone has no place to jump to (at best a skip scan). The order of the conditions in WHERE does not matter, which is why C works as well as B.
> **A** is wrong: customer_id is the first column: a seek.
> **B** is wrong: Both columns, in index order: the best case.
> **C** is wrong: Same as B: MySQL does not care about the order of the AND.

Q3

`sold_on` has an index. Rewrite `WHERE YEAR(sold_on) = 2024` so MySQL can use it.

```
WHERE sold_on >= '2024-01-01' AND sold_on < '2025-01-01'
```

> **Why:** The index is sorted by the column, not by YEAR(column). A range on the bare column (up to, but not including, the next January 1) gives the same rows and stays correct if the column ever becomes DATETIME.

Q4

`receipt_no` is `CHAR(7)` with a unique index, yet `WHERE receipt_no = 4242` shows `Table scan`. Why?

A) The text column is converted to a number on every row, so the index order no longer helps

B) Unique indexes cannot be used for equality

C) The table needs ANALYZE TABLE first

D) 4242 is too small: receipt numbers start at 1000000

> **Why:** A string column compared with a number is converted row by row ('0004242' becomes 4242), and converted values are not in the index's sort order. `WHERE receipt_no = '0004242'` is a single-row lookup again. SHOW WARNINGS after the EXPLAIN even says so (warning 1739).
> **B** is wrong: Equality on a unique index is the best case: Rows fetched before execution.
> **C** is wrong: Fresh statistics do not change a type conversion.
> **D** is wrong: The value matches a real row (the count is 1); only the plan is slow.

Q5

An index `idx_sales_customer_date (customer_id, sold_on)` exists. You run `CREATE INDEX idx_sales_customer ON sales_log (customer_id);`. What happens?

A) `ERROR 1061: Duplicate key name`

B) MySQL refuses: the column is already indexed

C) It succeeds without an error or a warning, and the new index is pure overhead

D) It replaces the composite index

> **Why:** The composite index already serves customer_id alone (its leftmost prefix), so the new one only slows every write. MySQL warns only when the columns are exactly the same (1831). Check SHOW INDEX before adding one.
> **A** is wrong: 1061 is about the NAME, and this name is new.
> **B** is wrong: MySQL lets you index a column many times.
> **D** is wrong: Indexes are never replaced silently; both now exist.

Q6

`DROP INDEX fk_orders_customer ON orders;` **fails. Which error number, and why does MySQL refuse?**

`1553` **: the foreign key from orders to customers needs an index on customer_id, and this is the only one**

> **Why:** InnoDB created fk_orders_customer for the foreign key. To replace it, create another index that starts with customer_id first; InnoDB then drops its own.

6.1 · INDEXES & QUERY OPTIMIZATION: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
SELECT INDEX_NAME, COLUMN_NAME
FROM information_schema.STATISTICS
WHERE TABLE_SCHEMA = 'bookstore' AND TABLE_NAME = 'categories'
ORDER BY INDEX_NAME;
```

```
+------------+-------------+
| INDEX_NAME | COLUMN_NAME |
+------------+-------------+
| name       | name        |
| PRIMARY    | id          |
+------------+-------------+
```

> **Why:** Two indexes: the primary key and `name`, the unique index MySQL created for the UNIQUE constraint on categories.name (named after the column).

P2

Predict the output, then run it.

```
EXPLAIN SELECT * FROM employees WHERE email = 'nina.kowalski@bookstore.example'\G
```

```
*************************** 1. row ***************************
EXPLAIN: -> Rows fetched before execution  (cost=0..0 rows=1)
```

> **Why:** email is UNIQUE, so a constant comparison finds at most one row while MySQL is still planning: Rows fetched before execution, the same line as a primary-key lookup.

6.2 · VIEWS: SELF-CHECK

Q7

What does MySQL store when you create a view?

A) Only the query: the rows are computed every time the view is read

B) A copy of the rows, refreshed every night

C) A copy of the rows, refreshed on every INSERT

D) An index on the base tables

> **Why:** A view is a named, saved SELECT (with its definer and security settings). Querying it runs the query against the base tables, so it is always current and exactly as fast as that query.
> **B** is wrong: That is a summary table you refresh yourself; MySQL has no materialized views.
> **C** is wrong: Views hold no rows at all.
> **D** is wrong: Views have no indexes; the base tables' indexes do the work.

Q8

What happens when the second statement runs?

```
CREATE VIEW v_ny AS
SELECT id, first_name, city
FROM customers
WHERE city = 'New York'
WITH CHECK OPTION;
UPDATE v_ny SET city = 'Boston' WHERE id = 1;
```

```
ERROR 1369 (HY000): CHECK OPTION failed 'bookstore.v_ny'
```

> **Why:** Alice would leave the view she was changed through. WITH CHECK OPTION rejects any write through the view that produces a row the view itself would not show; nothing is changed.

Q9

Which of these views accepts an UPDATE that passes through to its base table?

A) `SELECT author_id, COUNT(*) AS books FROM books GROUP BY author_id`

B) `SELECT DISTINCT city FROM customers`

C) `SELECT id, title, price FROM books WHERE price < 10`

D) `SELECT title FROM books UNION SELECT name FROM authors`

> **Why:** Each row of C is exactly one row of books, so MySQL knows which row to change. Grouping, DISTINCT and UNION make one view row stand for several base rows (error 1288).
> **A** is wrong: A count has no single base row behind it.
> **B** is wrong: DISTINCT merges rows: not updatable (IS_UPDATABLE says NO).
> **D** is wrong: UNION combines two tables' rows.

Q10

`v_all` was created as `SELECT * FROM products`. Then a column `color` is added to products. What does `SELECT * FROM v_all` return?

A) `ERROR 1356`

B) All columns, including color

C) An empty result until the view is refreshed

D) The old columns only, without color

> **Why:** The * was expanded into a fixed column list when the view was created, and that list is what MySQL saved. List columns explicitly and replace the view when the table gains one it should show.
> **A** is wrong: 1356 is for a renamed or dropped column the view still names.
> **B** is wrong: The column list is frozen at creation.
> **C** is wrong: Views are never "refreshed"; the rows are current, only the column list is fixed.

Q11

Why does the course put ORDER BY in the query that reads a view, and not inside the view?

MySQL ignores a view's ORDER BY when the outer query has its own, and the SQL standard promises no order from a view: sort where you read

> **Why:** The lesson's "Performance and Limitations" section. The project's checklist says it too: no view contains an ORDER BY; every query that reads one does.

Q12

An account has `SELECT` on `v_staff_directory` only. The view uses the default `SQL SECURITY DEFINER`. What can the account do?

A) Read every column of employees, salaries included

B) Read the directory through the view, but not the employees table itself

C) Nothing: it has no privilege on employees

D) Update salaries through the view

> **Why:** The view reads employees with its definer's rights, so it works for the account; SELECT on employees directly fails with 1142. The view is the security boundary: never select more columns than the audience should see.
> **A** is wrong: Only the view's columns (no salary) are visible.
> **C** is wrong: DEFINER security is exactly what makes the view work for it.
> **D** is wrong: The grant is SELECT only, and the view has no salary column.

6.2 · VIEWS: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
CREATE VIEW v_pricey AS SELECT title FROM books WHERE price > 12;
START TRANSACTION;
UPDATE books SET price = 9.00 WHERE id = 1;
SELECT title FROM v_pricey ORDER BY title;
ROLLBACK;
DROP VIEW v_pricey;
```

```
+-------------------+
| title             |
+-------------------+
| Lord of the Rings |
+-------------------+
```

> **Why:** Only Lord of the Rings: Dune's price dropped to 9.00 inside the transaction, and the view runs its query against the current rows. After ROLLBACK the view shows Dune again; nothing needs refreshing.

P4

Predict the output, then run it.

```
CREATE VIEW v_cities AS SELECT DISTINCT city FROM customers;
SELECT TABLE_NAME, IS_UPDATABLE
FROM information_schema.VIEWS
WHERE TABLE_SCHEMA = 'bookstore' AND TABLE_NAME = 'v_cities';
DROP VIEW v_cities;
```

```
+------------+--------------+
| TABLE_NAME | IS_UPDATABLE |
+------------+--------------+
| v_cities   | NO           |
+------------+--------------+
```

> **Why:** NO: DISTINCT merges rows, so one view row can stand for several customers and MySQL cannot know which one to change.

LAB REFERENCE SOLUTIONS

EXPLAIN estimates (cost, rows) differ between servers and runs; the plan shapes and the counts must match.

- LAB 6.1-A: LABS/MODULE-09/L6_1-INDEXES/PLAN_READING_STARTER.SQL → PLAN_READING_SOLUTION.SQL

The solution prints:

```
+---------------------+---------+----------+----------+
| Table               | Op      | Msg_type | Msg_text |
+---------------------+---------+----------+----------+
| bookstore.visit_log | analyze | status   | OK       |
+---------------------+---------+----------+----------+
+--------+------------+------------+
| visits | first_day  | last_day   |
+--------+------------+------------+
| 100000 | 2025-01-01 | 2026-05-15 |
+--------+------------+------------+
*************************** 1. row ***************************
EXPLAIN: -> Filter: (visit_log.visited_on = DATE'2025-06-01')  (cost=10083 rows=10011)
    -> Table scan on visit_log  (cost=10083 rows=100110)
*************************** 1. row ***************************
EXPLAIN: -> Index lookup on visit_log using idx_visits_on (visited_on = DATE'2025-06-01
+------------------+
| visits_on_june_1 |
+------------------+
|              200 |
+------------------+
*************************** 1. row ***************************
EXPLAIN: -> Aggregate: count(0)  (cost=303 rows=1)
    -> Filter: ((visit_log.customer_id = 3) and (visit_log.visited_on >= DATE'2025-06-0
        -> Covering index range scan on visit_log using idx_visits_customer_on over (cu
+--------------------+
| customer_3_in_june |
+--------------------+
|                700 |
+--------------------+
*************************** 1. row ***************************
EXPLAIN: -> Rows fetched before execution  (cost=0..0 rows=1)
+------------------------+-----------------------+------------+
| INDEX_NAME             | columns_in_order      | NON_UNIQUE |
+------------------------+-----------------------+------------+
| idx_visits_customer_on | customer_id,visited_on |          1 |
| idx_visits_on          | visited_on            |          1 |
| PRIMARY                | id                    |          0 |
| uq_visits_ticket       | ticket                |          0 |
+------------------------+-----------------------+------------+
```

Reading it: before the index, a Filter over a Table scan of all 100,000 rows; after, one Index lookup line. The composite index answers customer 3 in June as a covering range scan (COUNT(*) needs only the indexed columns). The unique ticket is found while planning. A learner who writes `(visited_on, customer_id)` in TODO 2 gets a different plan (an index range scan over the whole of June, checking the customer on each entry): a good discussion of column order.

- LAB 6.1-B: LABS/MODULE-09/L6_1-INDEXES/INDEX_KILLERS_STARTER.SQL → INDEX_KILLERS_SOLUTION.SQL

The solution prints:

```
+---------------------+---------+----------+----------+
| Table               | Op      | Msg_type | Msg_text |
+---------------------+---------+----------+----------+
| bookstore.visit_log | analyze | status   | OK       |
+---------------------+---------+----------+----------+
+-----------+
| slow_june |
+-----------+
|      6000 |
+-----------+
+-----------+
| fast_june |
+-----------+
|      6000 |
+-----------+
*************************** 1. row ***************************
EXPLAIN: -> Filter: ((year(visit_log.visited_on) = 2025) and (month(visit_log.visited_
    -> Table scan on visit_log  (cost=10083 rows=100110)
*************************** 1. row ***************************
EXPLAIN: -> Index range scan on visit_log using idx_visits_on over ('2025-06-01' <= vis
+-----------------+
| slow_first_week |
+-----------------+
|             200 |
+-----------------+
+-----------------+
| fast_first_week |
+-----------------+
|             200 |
+-----------------+
*************************** 1. row ***************************
EXPLAIN: -> Index range scan on visit_log using idx_visits_on over (visited_on <= '2025
+-------------+
| slow_ticket |
+-------------+
|           1 |
+-------------+
+-------------+
| fast_ticket |
+-------------+
|           1 |
+-------------+
*************************** 1. row ***************************
EXPLAIN: -> Filter: (visit_log.ticket = 4242)  (cost=10083 rows=10011)
    -> Table scan on visit_log  (cost=10083 rows=100110)
+---------+------+-----------------------------------------------------------------------
| Level   | Code | Message
+---------+------+-----------------------------------------------------------------------
```

```
| Warning | 1739 | Cannot use ref access on index 'uq_visits_ticket' due to type or col
| Warning | 1739 | Cannot use range access on index 'uq_visits_ticket' due to type or c
+---------+------+-------------------------------------------------------------------
*************************** 1. row ***************************
EXPLAIN: -> Rows fetched before execution  (cost=0..0 rows=1)
+-------------+
| starts_0042 |
+-------------+
|         100 |
+-------------+
+----+-------------+-----------+------------+------+---------------+------+---------+--
| id | select_type | table     | partitions | type | possible_keys | key  | key_len | r
+----+-------------+-----------+------------+------+---------------+------+---------+--
|  1 | SIMPLE      | visit_log | NULL       | ALL  | NULL          | NULL | NULL    | N
+----+-------------+-----------+------------+------+---------------+------+---------+--
+----+-------------+-----------+------------+-------+------------------+--------------
| id | select_type | table     | partitions | type  | possible_keys    | key
+----+-------------+-----------+------------+-------+------------------+--------------
|  1 | SIMPLE      | visit_log | NULL       | range | uq_visits_ticket | uq_visits_tick
+----+-------------+-----------+------------+-------+------------------+--------------
```

Same counts in every pair (6000, 200, 1, 100): the rewrites change the plan, not the answer. SHOW WARNINGS after the number comparison prints warning 1739, MySQL's own explanation (the checker shows it as a WARN line). The leading-% query cannot be rewritten without changing the question: "ends with 4242" really needs the scan (or a different design).

HANDOUT: LABS/MODULE-09/L6_1-INDEXES/DEBUG_INDEXES.SQL (DELIBERATELY BROKEN)

As shipped, MySQL prints (the index list first: nothing changed):

```
ERROR 1061 (42000): Duplicate key name 'fk_books_author'
ERROR 1091 (42000): Can't DROP 'idx_books_price'; check that column/key exists
ERROR 1553 (HY000): Cannot drop index 'fk_orders_customer': needed in a foreign key con
ERROR 1062 (23000): Duplicate entry 'New York' for key 'customers.uq_customers_city'
+------------+--------------------+-------------+
| TABLE_NAME | INDEX_NAME         | COLUMN_NAME |
+------------+--------------------+-------------+
| books      | fk_books_author    | author_id   |
| books      | PRIMARY            | id          |
| customers  | email              | email       |
| customers  | PRIMARY            | id          |
| orders     | fk_orders_customer | customer_id |
| orders     | PRIMARY            | id          |
+------------+--------------------+-------------+
```

Fixes are in the guide's clinic block. Note D's message: the first duplicate found is New York (Alice and Frank), not Chicago.

- LAB 6.2-A: LABS/MODULE-09/L6_2-VIEWS/REPORT_VIEWS_STARTER.SQL → REPORT_VIEWS_SOLUTION.SQL

The solution prints:

```
+---------------------+-------+--------+---------+
| author              | books | copies | revenue |
+---------------------+-------+--------+---------+
| J.R.R. Tolkien      |     2 |      5 |   64.95 |
| George Orwell       |     2 |      6 |   56.94 |
| Frank Herbert       |     1 |      3 |   44.97 |
| Harper Lee          |     1 |      2 |   23.98 |
| Ray Bradbury        |     1 |      2 |   20.98 |
| Jane Austen         |     1 |      2 |   15.98 |
| Aldous Huxley       |     1 |      1 |    9.49 |
| F. Scott Fitzgerald |     1 |      0 |    0.00 |
| Isaac Asimov        |     0 |      0 |    0.00 |
| J.D. Salinger       |     1 |      0 |    0.00 |
+---------------------+-------+--------+---------+
+---------------------+-------+
| author              | books |
+---------------------+-------+
| F. Scott Fitzgerald |     1 |
| Isaac Asimov        |     0 |
| J.D. Salinger       |     1 |
+---------------------+-------+
+-----------+--------+-------------+
| city      | orders | first_order |
+-----------+--------+-------------+
| New York  |      5 | 2026-01-15  |
| Chicago   |      4 | 2026-02-14  |
| Houston   |      2 | 2026-01-22  |
| (no city) |      1 | 2026-04-18  |
+-----------+--------+-------------+
+----------------+-------+----------+---------+
| author         | books | cheapest | revenue |
+----------------+-------+----------+---------+
| George Orwell  |     2 |     8.49 |   56.94 |
| J.R.R. Tolkien |     2 |    10.99 |   64.95 |
+----------------+-------+----------+---------+
+---------------------+------------+
| Tables_in_bookstore | Table_type |
+---------------------+------------+
| v_author_sales      | VIEW       |
| v_city_orders       | VIEW       |
+---------------------+------------+
+----------------+--------------+
| TABLE_NAME     | IS_UPDATABLE |
+----------------+--------------+
| v_author_sales | NO           |
| v_city_orders  | YES          |
+----------------+--------------+
```

COUNT(DISTINCT b.id) keeps the book count right after the join to order_items multiplies the rows (Lesson 4.3's fan-out). Asimov has 0 books and Fitzgerald and Salinger have books nobody ordered. The second view groups nothing, so it is updatable; the first groups, so it is not. The checker shows the views under its scratch database name.

- LAB 6.2-B: LABS/MODULE-09/L6_2-VIEWS/GUARDED_VIEWS_STARTER.SQL → GUARDED_VIEWS_SOLUTION.SQL

The solution prints:

```
+----------+
| on_shelf |
+----------+
|        9 |
+----------+
+----+------------+-------+
| id | title      | price |
+----+------------+-------+
|  6 | The Hobbit | 11.99 |
+----+------------+-------+
+----+------------+-------+
| id | title      | price |
+----+------------+-------+
|  6 | The Hobbit | 10.99 |
+----+------------+-------+
+-------------+-----------+
| city        | customers |
+-------------+-----------+
| Chicago     |         2 |
| New York    |         2 |
| Houston     |         1 |
| Los Angeles |         1 |
+-------------+-----------+
+-----------------------------------------------------------------------------------+
| Grants for kit_marketing@localhost                                                |
+-----------------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `kit_marketing`@`localhost`                                 |
| GRANT SELECT ON `bookstore`.`v_customer_cities` TO `kit_marketing`@`localhost` |
+-----------------------------------------------------------------------------------+
+-------------------+--------------+--------------+
| TABLE_NAME        | IS_UPDATABLE | CHECK_OPTION |
+-------------------+--------------+--------------+
| v_customer_cities | YES          | NONE         |
| v_shelf           | YES          | CASCADED     |
+-------------------+--------------+--------------+
```

TODO 1's prompt experiment, run for real: with the check option, `UPDATE v_shelf SET in_stock = FALSE WHERE id = 1;` prints

```
ERROR 1369 (HY000): CHECK OPTION failed 'bookstore.v_shelf'
```

Without it, the update succeeds and Dune silently leaves the view (reload bookstore.sql afterward). The Note 3162 line in the checker's output comes from `DROP USER IF EXISTS` when the account does not exist yet: harmless.

HANDOUT: LABS/MODULE-09/L6_2-VIEWS/DEBUG_VIEWS.SQL (DELIBERATELY BROKEN)

As shipped, MySQL prints:

```
ERROR 1050 (42S01): Table 'v_dbg_cheap' already exists
ERROR 1288 (HY000): The target table v_dbg_counts of the UPDATE is not updatable
ERROR 1369 (HY000): CHECK OPTION failed 'bookstore.v_dbg_cheap'
ERROR 1356 (HY000): View 'bookstore.v_dbg_flyer_prices' references invalid table(s) or
+----+-------+-------+
| id | title | price |
+----+-------+-------+
|  2 | 1984  |  9.99 |
+----+-------+-------+
```

The SELECT at the end proves C changed nothing (1984 is still 9.99). Fixes are in the guide's clinic block.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 6.1 · INDEXES & QUERY OPTIMIZATION

1. **B** It returns the right rows after reading the whole table: Without an index MySQL reads every row and tests each one: a full table scan. The result is correct, it just takes longer as the table grows.
2. **A** WHERE customer_id = 7 AND sold_on >= '2026-01-01': customer_id is the leftmost column, so MySQL can jump to customer 7, and inside it the dates are sorted, so the date range is a tight slice.
3. **C** Every row is read and then filtered: no index is used: Table scan means every row is read and the Filter keeps the matches. On a big table that is the step to fix, with an index or a rewritten condition.
4. **C** The index is sorted by the column, not by the function's result: The index is sorted by sold_on, not by YEAR(sold_on), so MySQL must compute the function for every row. A range on the bare column (from 2026-01-01 up to 2027-01-01) can use the index.
5. **A** Make it INVISIBLE and watch the slow queries: An invisible index is ignored by the optimizer but still maintained, so you can watch for slow queries and make it VISIBLE again instantly if something needed it.

LESSON 6.2 · VIEWS

1. **C** The new numbers, including the new order: A view stores only its query. Every SELECT runs that query against the current base tables, so the new order is counted immediately.

2. **B** A GROUP BY with COUNT(*): With GROUP BY, one view row stands for many base rows, so MySQL cannot tell which row an update should change. Such views report IS_UPDATABLE = NO and fail with error 1288.

3. **A** Rejects writes through the view that would create rows the view cannot show: It rejects inserts and updates through the view that would produce a row the view's WHERE clause excludes, with error 1369.

4. **B** The old columns only, without color: The star was expanded into a fixed column list when the view was created, so the new column is silently missing until the view is replaced.

5. **C** Read the directory through the view, but not the table itself: The view reads employees with its definer's privileges, so the account sees exactly the view's columns and rows, and nothing else of the table.

EXIT TICKET

1. **Index:** `(customer_id, book_id, sold_on)`: both equality columns first (either order), the sort column last. `(customer_id, sold_on)` alone still has to check the book on every row; `(sold_on, ...)` first cannot seek.

2. **Index killers:** a function or arithmetic around the column (`YEAR(sold_on)` , `price * 1.1`); a leading `%` ; a text column compared with a number; OR across columns. Accept "a condition that matches most of the table" as a reason the optimizer ignores an index.

3. **Views:** reasons: reuse a long query, a safe updatable slice (with a check option), a security boundary. Not: stored rows, indexes, speed. Accept "refresh itself" only if they explain it is always current because it stores no rows.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q2 C.** Believes the order of the conditions in WHERE matters. It does not; the order of the columns in the index does.
- **Q4 B or C.** Has not met type conversion. Run the lesson's step with SHOW WARNINGS: 1739 says it in words.
- **Q5 A.** Mixes up an index's name with its columns.
- **Q7 B.** Expects a cache. MySQL has no materialized views; P3 shows the view is always current.
- **Q9 A or B.** Thinks "simple-looking" means updatable. The test: one view row = one base row.
- **Q10 B.** The most common view surprise; the lesson's Common Mistake 5.
- **Exit ticket 1 with** `(sold_on, customer_id, book_id)`**.** The sort column first: worth two minutes of the Session 10 warm-up.

INSTRUCTOR KIT · ANSWER KEY

Session 10: Normalization and Stored Programs

Answers and reasoning for every Session 10 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs, both debug handouts, the lesson pages' own quizzes and the exit ticket.

Session 10 | 12 self-check items | Confidential

WARM-UP

Answers to W1 to W5, with MySQL's real output where the question shows code:

W1

Which plan line do you expect, and why is there an index at all?

```
EXPLAIN SELECT * FROM orders WHERE customer_id = 4\G
```

Index lookup on orders using fk_orders_customer: InnoDB created that index for the foreign key.

```
*************************** 1. row ***************************
EXPLAIN: -> Index lookup on orders using fk_orders_customer (customer_id = 4)  (c
```

W2

Is this view updatable?

```
CREATE VIEW v_cheap AS SELECT id, title, price FROM books WHERE price < 9;
SELECT TABLE_NAME, IS_UPDATABLE FROM information_schema.VIEWS
WHERE TABLE_SCHEMA = 'bookstore' AND TABLE_NAME = 'v_cheap';
DROP VIEW v_cheap;
```

YES: each view row is one books row (no grouping, DISTINCT or UNION).

```
+------------+--------------+
| TABLE_NAME | IS_UPDATABLE |
+------------+--------------+
| v_cheap    | YES          |
+------------+--------------+
```

W3

An index on (customer_id, sold_on): can a query on a sold_on range alone seek with it?

No: the dates are sorted only within each customer (leftmost prefix).

W4

What does MySQL store when you create a view?

Only the query; rows are computed on every read.

W5

What does WITH CHECK OPTION refuse?

A write through the view that would produce a row the view cannot show (1369).

6.3 · DATABASE DESIGN & NORMALIZATION: SELF-CHECK

Q1

Alice's email is stored on three order rows. Someone corrects it on one row only, and now the table holds two emails for her. Which anomaly is this?

A) An update anomaly

B) An insert anomaly

C) A delete anomaly

D) A first normal form violation

Why: A fact stored several times was changed in one place: the copies drift apart and no query can tell which is right. The cause is the same as for all three anomalies: facts about a customer stored in an order table.
B is wrong: An insert anomaly is when you cannot add a fact (a customer with no order) without another one.
C is wrong: A delete anomaly is losing a fact (a customer) by deleting another (their only order).
D is wrong: Every cell here holds one value; the problem is repetition, which 2NF and 3NF deal with.

Q2

A contacts table has the columns `name`, `phone1`, `phone2` and `phone3`. Which normal form does it break?

A) Second normal form

B) Third normal form

C) First normal form: a repeating group of columns

D) None: three columns are fine

Why: Numbered columns are a list spread across columns: a fourth phone needs a new column, and "who has this number?" must check three columns. 1NF: one row per phone in a child table (contact_id, phone).
A is wrong: 2NF is about partial dependencies on a composite key.
B is wrong: 3NF is about non-key columns that depend on other non-key columns.
D is wrong: It works until someone has four numbers, and every search gets three times longer.

Q3

`enrollments(student_id, course, course_room, grade)` has the key `(student_id, course)`. Each course meets in one room. Which rule does `course → course_room` break?

A) 1NF

B) 2NF: course_room depends on only part of the key

C) 3NF: a transitive dependency

D) None: course is part of the key

> **Why:** course_room needs only course, not the whole key, so every student's row repeats the room. Fix: courses(course, room) and enrollments(student_id, course, grade).
> **A** is wrong: Every cell holds one value.
> **C** is wrong: Transitive means through a NON-key column; course is part of the key.
> **D** is wrong: Being part of the key is exactly what makes it a partial dependency.

Q4

`employees(id, name, dept_id, dept_name)` has the key `id`. Which dependency breaks 3NF, and what is the fix?

`dept_id → dept_name` (a transitive dependency: id → dept_id → dept_name); move dept_name to `departments(id, name)` and keep dept_id as a foreign key

> **Why:** dept_name describes the department, not the employee. As long as it lives in employees, renaming a department means updating every employee in it.

Q5

You split a flat table into four, then run two EXCEPT queries in opposite directions (old rows EXCEPT the rebuilt join, and the other way). Both print Empty set. What does that prove?

A) Every new table has a primary key

B) The new tables contain no duplicates

C) The design is in 3NF

D) The four tables, joined back, hold exactly the original rows: the split lost nothing and added nothing

> **Why:** EXCEPT returns rows of the first query missing from the second. One direction catches lost rows, the other catches invented ones (a wrong join). Both empty = lossless.
> **A** is wrong: Keys are a separate check (SHOW CREATE TABLE).
> **B** is wrong: EXCEPT compares the two sides; it says nothing about duplicates inside one table.
> **C** is wrong: A lossless split can still keep a transitive dependency.

Q6

`SELECT member_email FROM club_signups GROUP BY member_email HAVING COUNT(DISTINCT member_name) > 1;` prints Empty set. What can you conclude?

A) `member_email → member_name` is proved

B) The data does not contradict member_email → member_name; whether it is a rule is a business decision

C) `member_name → member_email` holds

D) The table is in 3NF

> **Why:** Data can disprove a dependency (one counterexample is enough) but never prove one: the next sign-up might break it. The rule comes from the business ("an email belongs to one member").
> **A** is wrong: Six rows cannot prove a rule about all future rows.
> **C** is wrong: The query tests the other direction.
> **D** is wrong: One dependency says nothing about the whole table's normal form.

6.3 · DATABASE DESIGN & NORMALIZATION: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
CREATE TABLE sheet (member VARCHAR(30), club VARCHAR(30));
INSERT INTO sheet VALUES ('Ana', 'Chess'), ('Ben', 'Chess'), ('Ana', 'Poetry'),
                         ('Cy', 'Poetry'), ('Ben', 'Chess');
CREATE TABLE members (id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(30) UNIQUE);
INSERT INTO members (name) SELECT DISTINCT member FROM sheet ORDER BY member;
SELECT COUNT(*) AS sheet_rows, (SELECT COUNT(*) FROM members) AS members FROM sheet
```

```
+------------+---------+
| sheet_rows | members |
+------------+---------+
|          5 |       3 |
+------------+---------+
```

Why: Five sheet rows, three distinct members (Ana, Ben, Cy): SELECT DISTINCT collapses the copies, and the UNIQUE on name would refuse a second run (1062).

P2

Predict the output, then run it.

```
SELECT title FROM books WHERE price < 9
EXCEPT
SELECT title FROM books WHERE in_stock = FALSE
ORDER BY title;
```

```
+---------------------+
| title               |
+---------------------+
| Animal Farm         |
| Pride and Prejudice |
+---------------------+
```

Why: Books under $9 are The Great Gatsby, The Catcher in the Rye, Animal Farm and Pride and Prejudice; EXCEPT removes the two that are out of stock. The ORDER BY sorts the whole EXCEPT result.

7.1 · STORED PROCEDURES & FUNCTIONS: SELF-CHECK

Q7

In the mysql client, without changing the delimiter, you type a procedure whose body is `BEGIN SELECT ...; END;`. What happens?

A) Two `ERROR 1064` lines: the client sent the text in two pieces at the semicolons

B) `ERROR 1418`

C) The procedure is created

D) `ERROR 1304: PROCEDURE already exists`

> **Why:** The client splits at every ; and sends the unfinished CREATE ... BEGIN SELECT ... (1064 near ''), then the leftover END (1064 near 'END'). DELIMITER // ... // ... DELIMITER ; sends the whole body at once (the debug handout's section A).
> **B** is wrong: 1418 is for functions without characteristics.
> **C** is wrong: Only a one-statement body without BEGIN and END works without a delimiter change.
> **D** is wrong: Nothing was created, so nothing exists yet.

Q8

What does this print? (Books under $9: The Great Gatsby, The Catcher in the Rye, Animal Farm, Pride and Prejudice.)

```
CREATE PROCEDURE count_cheaper(IN p_price DECIMAL(8, 2), OUT p_n INT)
    SELECT COUNT(*) INTO p_n FROM books WHERE price < p_price;
CALL count_cheaper(9.00, @n);
SELECT @n AS cheaper;
```

`4` (in a column named `cheaper`)

> **Why:** The one-statement body needs no BEGIN and no delimiter change. SELECT ... INTO stores the count in the OUT parameter, which lands in @n; the CALL itself prints nothing.

Q9

A stored function returns `ROUND(p_amount * 1.08, 2)` and reads no table. Which characteristics are true of it?

A) `READS SQL DATA`

B) `MODIFIES SQL DATA`

C) `NOT DETERMINISTIC` (the default)

D) `DETERMINISTIC` and `NO SQL`

> **Why:** Same input, same output, and no SQL statement in the body: the lesson's with_tax. With binary logging on, one of DETERMINISTIC, NO SQL or READS SQL DATA is required (1418), and the promise must be true.
> **A** is wrong: It reads no table.
> **B** is wrong: It changes nothing.
> **C** is wrong: False modesty is legal but untrue, and on its own it is refused with 1418.

Q10

A procedure runs `SELECT title INTO p_title FROM books WHERE author_id = p_author_id;`. What happens when it is called for J.R.R. Tolkien, who has two books?

A) p_title gets the first title in id order

B) p_title is NULL

C) `ERROR 1172 (42000): Result consisted of more than one row`

D) p_title holds both titles, separated by a comma

> **Why:** One variable cannot hold two rows. The procedure was created without complaint; the error appears only on a call that hits it, which is why routines are tested with awkward inputs. Fix: an aggregate (MIN(title)) or ORDER BY ... LIMIT 1.
> **A** is wrong: MySQL does not pick one for you.
> **B** is wrong: NULL comes from an aggregate over no rows, not from two rows.
> **D** is wrong: That would be GROUP_CONCAT.

Q11

What does the CALL print?

```
DELIMITER //
CREATE PROCEDURE check_qty(IN p_qty INT)
BEGIN
    IF p_qty < 1 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Quantity must be at least 1';
    END IF;
    SELECT p_qty AS accepted;
END //
DELIMITER ;
CALL check_qty(0);
```

`ERROR 1644 (45000): Quantity must be at least 1`

> **Why:** SIGNAL SQLSTATE '45000' raises your own error, reported as 1644 with your message, and stops the procedure: the SELECT after it never runs.

Q12

Which of these can a stored function NOT do?

A) Return one value

B) Read tables (declared READS SQL DATA)

C) Run `COMMIT` or `START TRANSACTION`

D) Be used inside `WHERE`

> **Why:** Functions cannot control transactions and cannot return result sets; procedures can (Lesson 7.2 puts transactions inside procedures). Used in WHERE, a function runs once per row, so keep it cheap.
> **A** is wrong: That is what a function is for.
> **B** is wrong: copies_sold and author_revenue do exactly that.
> **D** is wrong: Anywhere an expression fits: the functions lab filters with copies_sold(id) = 0.

7.1 · STORED PROCEDURES & FUNCTIONS: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
DELIMITER //
CREATE PROCEDURE double_it(INOUT p_n INT)
BEGIN
    SET p_n = p_n * 2;
END //
DELIMITER ;
SET @n = 10;
CALL double_it(@n);
CALL double_it(@n);
SELECT @n AS n;
```

```
+------+
| n    |
+------+
|   40 |
+------+
```

Why: INOUT reads the caller's value and replaces it: 10 → 20 → 40.

P4

Predict the output, then run it.

```
DELIMITER //
CREATE PROCEDURE first_title(IN p_author_id INT, OUT p_title VARCHAR(200))
BEGIN
    SELECT MIN(title) INTO p_title FROM books WHERE author_id = p_author_id;
END //
DELIMITER ;
SET @t = 'nothing yet';
CALL first_title(10, @t);
SELECT @t AS asimov_first_title;
```

```
+--------------------+
| asimov_first_title |
+--------------------+
| NULL               |
+--------------------+
```

Why: NULL, not 'nothing yet': an OUT parameter starts as NULL inside the procedure (it does not carry the caller's value in), and MIN over no rows is NULL. (With INOUT, 'nothing yet' would come in, but MIN over no rows would still overwrite it with NULL.)

LAB REFERENCE SOLUTIONS

- LAB 6.3-A: LABS/MODULE-10/L6_3-NORMALIZATION/ANOMALIES_STARTER.SQL → ANOMALIES_SOLUTION.SQL

The solution prints (the interactive client also prints Empty set for the host_email → host_name test):

```
Database changed
+---------------+---------------------+---------------+------------+--------------+---
| attendee_name | workshop            | workshop_date | room       | host_name    | fe
+---------------+---------------------+---------------+------------+--------------+---
| Alice Johnson | Bookbinding Basics  | 2026-07-11    | Back room  | Lena Park    | 15
| Bob Smith     | Bookbinding Basics  | 2026-07-11    | Back room  | Lena Park    | 15
| Eve Davis     | Bookbinding Basics  | 2026-07-11    | Back room  | Lena Park    | 15
| Alice Johnson | Writing Fan Fiction | 2026-07-18    | Front room | James Okafor | 10
| Grace Wilson  | Writing Fan Fiction | 2026-07-18    | Front room | James Okafor | 10
| Frank Miller  | Speed Reading       | 2026-07-25    | Back room  | Lena Park    | 12
+---------------+---------------------+---------------+------------+--------------+---
+--------------------+---------------+
| workshop           | workshop_date |
+--------------------+---------------+
| Bookbinding Basics | 2026-07-11    |
| Bookbinding Basics | 2026-07-12    |
+--------------------+---------------+
+--------------------------+
| rows_about_speed_reading |
+--------------------------+
|                        0 |
+--------------------------+
+---------------------+-------+-------+-------+------+
| workshop            | dates | rooms | hosts | fees |
+---------------------+-------+-------+-------+------+
| Bookbinding Basics  |     1 |     1 |     1 |    1 |
| Speed Reading       |     1 |     1 |     1 |    1 |
| Writing Fan Fiction |     1 |     1 |     1 |    1 |
+---------------------+-------+-------+-------+------+
Empty set
+------------------------------+-----------+
| host_email                   | workshops |
+------------------------------+-----------+
| lena.park@bookstore.example |         2 |
+------------------------------+-----------+
+-------------+----------------+--------------------+
| signup_rows | distinct_pairs | distinct_attendees |
+-------------+----------------+--------------------+
|           6 |              6 |                  5 |
+-------------+----------------+--------------------+
```

The dependencies: attendee_email → attendee_name; workshop → workshop_date, room, host_email, fee; host_email → host_name. The key: (attendee_email, workshop). host_email → workshop is broken by Lena (two workshops), which is why workshops point at hosts and not the other way around. Room → host is a trap: the data shows one host per room today (both back-room workshops are Lena's), but no business rule says so.

- LAB 6.3-B: LABS/MODULE-10/L6_3-NORMALIZATION/NORMALIZE_STARTER.SQL → NORMALIZE_SOLUTION.SQL

The solution prints (interactively, the two EXCEPT queries print Empty set between the counts and the last table):

```
Database changed
+-----------+-------+-----------+---------+
| attendees | hosts | workshops | signups |
+-----------+-------+-----------+---------+
|         5 |     2 |         3 |       6 |
+-----------+-------+-----------+---------+
Empty set
Empty set
+---------------+--------------------+---------------+
| attendee      | title              | workshop_date |
+---------------+--------------------+---------------+
| Alice Johnson | Bookbinding Basics | 2026-07-12    |
| Bob Smith     | Bookbinding Basics | 2026-07-12    |
| Eve Davis     | Bookbinding Basics | 2026-07-12    |
+---------------+--------------------+---------------+
```

Before TODO 1 and 2 the counts are 5, 0, 0, 0 and the first EXCEPT lists all six sheet rows: a missing parent makes the children's joins find nothing, which the proof catches. The second direction would catch a wrong join that invents rows (for example joining workshops on room instead of title).

- HANDOUT: LABS/MODULE-10/L6_3-NORMALIZATION/DEBUG_DESIGN.SQL (DELIBERATELY BROKEN)

As shipped, MySQL prints:

```
Database changed
ERROR 1062 (23000): Duplicate entry 'lena.park@bookstore.example' for key 'hosts.email
ERROR 3780 (HY000): Referencing column 'workshop_id' and referenced column 'id' in fore
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fail
+-------+-----------+
| hosts | workshops |
+-------+-----------+
|     2 |         2 |
+-------+-----------+
```

Fixes are in the guide's clinic block. The final counts (2 hosts, 2 workshops) show that nothing got in and nothing got lost.

- LAB 7.1-A: LABS/MODULE-10/L7_1-STORED-PROCEDURES/PROCEDURES_STARTER.SQL → PROCEDURES_SOLUTION.SQL

The solution prints (the Note 1305 lines in the checker's output come from DROP ... IF EXISTS on a first run):

```
+-----------------+-------+
| title           | price |
+-----------------+-------+
| Brave New World |  9.49 |
| 1984            |  9.99 |
| Fahrenheit 451  | 10.49 |
+-----------------+-------+
+--------------+---------------+--------------+---------------+
| orwell_books | orwell_copies | asimov_books | asimov_copies |
+--------------+---------------+--------------+---------------+
|            2 |             6 |            0 |             0 |
+--------------+---------------+--------------+---------------+
+----+------------+-------+
| id | title      | price |
+----+------------+-------+
|  6 | The Hobbit | 12.09 |
+----+------------+-------+
+----+------------+-------+
| id | title      | price |
+----+------------+-------+
|  6 | The Hobbit | 10.99 |
+----+------------+-------+
+----------------+--------------+
| ROUTINE_NAME   | ROUTINE_TYPE |
+----------------+--------------+
| author_stats   | PROCEDURE    |
| category_shelf | PROCEDURE    |
| raise_price    | PROCEDURE    |
+----------------+--------------+
```

TODO 3's test, run for real after the solution file: `CALL raise_price(6, 80);` prints

```
ERROR 1644 (45000): Percent must be between 1 and 50
+----+------------+-------+
| id | title      | price |
+----+------------+-------+
|  6 | The Hobbit | 10.99 |
+----+------------+-------+
```

and The Hobbit keeps its price, because the guard runs before the UPDATE.

- LAB 7.1-B: LABS/MODULE-10/L7_1-STORED-PROCEDURES/FUNCTIONS_STARTER.SQL → FUNCTIONS_SOLUTION.SQL

The solution prints:

```
+----------+-------+----------+
| order_id | items | shipping |
+----------+-------+----------+
|        1 |     3 |     6.99 |
|        2 |     1 |     4.99 |
|        4 |     2 |     5.99 |
|        8 |     1 |     4.99 |
|       12 |     2 |     5.99 |
+----------+-------+----------+
+--------------------+--------+
| title              | copies |
+--------------------+--------+
| 1984               |      4 |
| The Hobbit         |      3 |
| Animal Farm        |      2 |
| Lord of the Rings  |      2 |
| The Great Gatsby   |      0 |
+--------------------+--------+
+-------------------------+
| title                   |
+-------------------------+
| The Catcher in the Rye  |
| The Great Gatsby        |
+-------------------------+
+--------------+------------------+-----------------+
| ROUTINE_NAME | IS_DETERMINISTIC | SQL_DATA_ACCESS |
+--------------+------------------+-----------------+
| copies_sold  | NO               | READS SQL DATA  |
| shipping_fee | YES              | NO SQL          |
+--------------+------------------+-----------------+
```

shipping_fee is DETERMINISTIC and NO SQL: the same item count always gives the same fee. copies_sold reads order_items, whose rows change, so READS SQL DATA (DETERMINISTIC would be a false promise the server cannot check). Used in WHERE, copies_sold runs once per book: fine for 11 rows, a reason to prefer a JOIN ... GROUP BY on big tables.

- HANDOUT: LABS/MODULE-10/L7_1-STORED-PROCEDURES/DEBUG_ROUTINES.SQL (DELIBERATELY BROKEN)

As shipped, MySQL prints:

```
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresp
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresp
ERROR 1418 (HY000): This function has none of DETERMINISTIC, NO SQL, or READS SQL DATA
ERROR 1172 (42000): Result consisted of more than one row
ERROR 1304 (42000): PROCEDURE dbg_count_books already exists
ERROR 1644 (45000): Pages must be a positive number
+----+----------------+-------+
| id | title          | pages |
+----+----------------+-------+
| 10 | Fahrenheit 451 |   158 |
+----+----------------+-------+
```

Fixes are in the guide's clinic block. Section B fails only with binary logging on (the MySQL 8.0+ default); on MariaDB or a server with `--skip-log-bin` the function is created, which is exactly the trap the lesson describes.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 6.3 · DATABASE DESIGN & NORMALIZATION

1. **A** First normal form: 1NF requires one value per cell. A comma list puts two values in one cell, which is why SQL cannot sum or join on them.
2. **C** A non-key column that depends on another non-key column: A non-key column determined by another non-key column, like order_id determining cust_email and cust_email determining cust_city. 3NF removes it by moving the customer columns to their own table.
3. **B** 2NF: it depends on part of the key: order_date depends on order_id, which is only part of the composite key: a partial dependency, so the table is not in 2NF. That is why the date repeats on every line of an order.
4. **A** No: it records what was paid on that order, a different fact from today's price: price_at_time is a fact about the sale, determined by (order_id, book_id). books.price is the current price. They only coincide until the first price change, and then old orders must keep what was actually paid.
5. **B** When a measured query is too slow and you have a plan to keep the total in sync: Denormalize only for a measured problem that indexes and better queries did not solve, and only with a plan (a transaction, a trigger or a refresh) that keeps the stored value correct.

LESSON 7.1 · STORED PROCEDURES & FUNCTIONS

1. **B** The mysql command-line client: DELIMITER is a client command. The mysql client uses it to decide where one statement ends before sending it; the server only ever receives the finished CREATE PROCEDURE statement.
2. **C** Every book in the table: Inside the routine the name id means the parameter, so the condition is 1 = 1 on every row and all the books come back, with no error or warning.

3. **A** READS SQL DATA: The function reads tables and changes nothing, so READS SQL DATA is the honest description. It is also enough to avoid error 1418.

4. **C** A stored function: Only a stored function returns a single value that can be used inside an expression, once per row of the query.

5. **A** Error 1644 with the message from SIGNAL, and nothing is inserted: The IF check runs first and SIGNAL raises error 1644 with the procedure's own message. The procedure stops there, before either INSERT, so no author and no book are added.

EXIT TICKET

1. **Partial dependency:** `order_id → order_date` (moves to orders) or `book_title → book_price` (moves to books). Reject `(order_id, book_title) → quantity`: that is the whole key, which is fine.

2. **Check before change:** so a refused call leaves the data untouched (statements that already ran are not undone by SIGNAL); `SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = '...'`, reported as error 1644.

3. **Function / procedure:** a function (one value, inside a SELECT); a procedure (it inserts rows and returns the new id through an OUT parameter, like the project's add_book).

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q2 A or B.** Thinks numbered columns are a dependency problem. They are a list: 1NF.
- **Q3 C.** Mixes up "part of the key" (2NF) with "another non-key column" (3NF).
- **Q6 A.** Treats data as proof; worth repeating with the worked example's price_at_time.
- **Q7 C.** Believes the server reads DELIMITER, or has only seen one-statement bodies.
- **Q9 A.** Picks READS SQL DATA by reflex: the function reads no table.
- **Q10 A.** Expects MySQL to pick a row: it refuses (1172).
- **Exit ticket 3 "procedure" for both.** Ask them to write the SELECT with a CALL inside it: impossible.

INSTRUCTOR KIT · ANSWER KEY

Session 11: Transactions and JSON

Answers and reasoning for every Session 11 item: the warm-up, Checkpoint 2 (rubric, model answer, weak answers), both self-checks, the predict-then-run items, the lab reference outputs, the lesson pages' own quizzes and the exit ticket.

Session 11 | 12 self-check items | Checkpoint 2 | Confidential

WARM-UP

1. **W1** (6.3) 1NF: one value per cell. Fix: one row per book in a child table (member_id, book_id).
2. **W2** (6.3) "The whole key" is 2NF (no column depends on only part of a composite key); "nothing but the key" is 3NF (no column depends on another non-key column).
3. **W3** (7.1) The mysql client. It changes where the client cuts statements; the server never sees the word DELIMITER.
4. **W4** (7.1) A stored function: it returns one value and can sit inside an expression. A procedure runs with CALL.
5. **W5** (7.1) 11.99 for Dune and 7.99 for 1984: 80% of the price, rounded to cents.

W5 prints:

```
+-------+-------+---------+
| title | price | on_sale |
+-------+-------+---------+
| Dune  | 14.99 |   11.99 |
| 1984  |  9.99 |    7.99 |
+-------+-------+---------+
```

CHECKPOINT 2: RUBRIC, MODEL ANSWER AND WEAK ANSWERS

THE TASK

Thirty minutes, alone, open notes. Parts 1 to 4 in `labs/module-11/checkpoint-2/checkpoint2_starter.sql`, Part 5 on paper in the workbook (the class_signups table).

PART	LESSONS	TASK
Part 1	5.1, 5.2	The best seller(s) of each category by copies sold (SUM of order_items.quantity), ties sharing first place: a CTE for the copies and a window function to rank. Columns category, title, copies, sorted by category.
Part 2	5.1	Everyone who reports to Daniel Reyes (employee 2), directly or through someone else, with a level column (1 = reports to Daniel directly), sorted by level and id: a recursive CTE.
Part 3	6.2	The view v_customer_revenue (customer_id, customer, orders, revenue) for customers who have ordered, then a query on the view: customers with revenue above 30, highest first.
Part 4	6.1	The plan of SELECT id, first_name FROM customers WHERE city = 'Chicago' before and after an index that helps it: the first line of each plan.
Part 5	6.3	On paper: the table class_signups below. Name its key, the dependencies that break 2NF and 3NF, and the tables (with keys) you would split it into.

Rubric. Five rows, four levels, 20 points. "Proficient" in every row (15 points) is a successful checkpoint. Circle one cell per row and write one sentence of advice.

CRITERION	1 · BEGINNING	2 · DEVELOPING	3 · PROFICIENT	4 · EXEMPLARY
CTE and window function (Part 1)	No ranking: a GROUP BY with MAX(copies) that cannot say which title won, or no result.	A ranking, but over the whole shop (no PARTITION BY), or the window function filtered in the same query's WHERE (`ERROR 3593`).	A CTE for the copies, RANK() OVER (PARTITION BY category ORDER BY copies DESC) in a second step, filtered on 1 outside: five rows (Classic, Dystopian and Fiction 1984 with 4; Fantasy The Hobbit 3; Science Fiction Dune 3).	Also says why the filter needs a second step (window functions run after WHERE) and why RANK, not ROW_NUMBER, keeps ties.

CRITERION	1 · BEGINNING	2 · DEVELOPING	3 · PROFICIENT	4 · EXEMPLARY
Recursive CTE (Part 2)	Only the direct reports (WHERE manager_id = 2), or a self-join with a fixed depth.	A recursive CTE that starts from Daniel himself (adds him as level 1) or has no working level column.	Anchor manager_id = 2 at level 1, recursive member joined on e.manager_id = t.id with level + 1: six rows, Priya and Marcus at 1; Tom, Lena, Sofia and Ben at 2.	Also explains why the recursion stops (no one reports to the level-2 people) and notices Ben is inactive, keeping or filtering him on purpose.
View (Part 3)	No view: the query typed directly, or a view that does not run.	A view that runs but counts order LINES as orders (`COUNT(*)` gives Alice 4) or leaves out the name.	COUNT(DISTINCT o.id) and SUM(quantity * price_at_time) in the view; the query on the view returns Alice Johnson 3 / 58.95, Dave Brown 45.96, Frank Miller 41.96, Grace Wilson 33.97, Bob Smith 33.47.	Also says a view stores the query, not the rows (it is always current), and that a LEFT JOIN would list customers who never ordered.
Index and EXPLAIN (Part 4)	No plan read, or an index on a column the search does not use.	An index on city, but the plans are not read (no before/after) or misread.	Before: `Table scan on customers` under a Filter; after `CREATE INDEX ... (city)`: `Index lookup on customers using idx_customers_city`.	Also reads the row estimates (8 scanned, then 2) and says what the index costs (space, slower writes) and why 8 rows hide the gain.

CRITERION	1 · BEGINNING	2 · DEVELOPING	3 · PROFICIENT	4 · EXEMPLARY
Normalization (Part 5)	A list of tables with no key or reasons.	Finds the 2NF problems (student_name; class_title, room, teacher) or the 3NF one (teacher -> teacher_phone), not both.	Key (student_email, class_code); 2NF: student_name depends on student_email, class_title/room/teacher on class_code; 3NF: teacher_phone depends on teacher. Tables: students, teachers, classes (teacher FK), signups (both FKs as the key).	Also names the anomalies (Lee Wong's phone in three rows; a class nobody signed up for cannot exist; deleting Cy's signup loses Glass Fusing) and questions whether room belongs to the class.

MODEL ANSWER, PARTS 1 TO 4: LABS/MODULE-11/CHECKPOINT-2/CHECKPOINT2_SOLUTION.SQL

The file prints (MySQL 9.7, fresh bookstore):

```
+-----------------+------------+--------+
| category        | title      | copies |
+-----------------+------------+--------+
| Classic         | 1984       |      4 |
| Dystopian       | 1984       |      4 |
| Fantasy         | The Hobbit |      3 |
| Fiction         | 1984       |      4 |
| Science Fiction | Dune       |      3 |
+-----------------+------------+--------+
+-------+------+---------------+------------+
| level | id   | employee      | manager_id |
+-------+------+---------------+------------+
|     1 |    3 | Priya Nair    |          2 |
|     1 |    6 | Marcus Hill   |          2 |
|     2 |    4 | Tom Becker    |          3 |
|     2 |    5 | Lena Park     |          3 |
|     2 |    7 | Sofia Alvarez |          6 |
|     2 |   10 | Ben Carter    |          3 |
+-------+------+---------------+------------+
+---------------+--------+---------+
| customer      | orders | revenue |
+---------------+--------+---------+
| Alice Johnson |      3 |   58.95 |
| Dave Brown    |      2 |   45.96 |
| Frank Miller  |      2 |   41.96 |
| Grace Wilson  |      2 |   33.97 |
| Bob Smith     |      2 |   33.47 |
+---------------+--------+---------+
*************************** 1. row ***************************
EXPLAIN: -> Filter: (customers.city = 'Chicago')  (cost=1.05 rows=1)
    -> Table scan on customers  (cost=1.05 rows=8)
*************************** 1. row ***************************
EXPLAIN: -> Index lookup on customers using idx_customers_city (city = 'Chicago')  (cos
```

Reading it: Part 1 shows 1984 winning three categories (4 copies) and no tie this time; RANK would keep a tie, ROW_NUMBER would silently drop one. Part 2's recursion stops at level 2 because nobody reports to Tom, Lena, Sofia or Ben. Part 3: Alice's 3 orders hold 4 order lines, the reason COUNT(DISTINCT o.id) matters. Part 4: the cost numbers differ by machine; the access method (table scan, then index lookup) is the answer.

MODEL ANSWER, PART 5 (NORMALIZATION)

- **Key:** (student_email, class_code): one student in one class.
- **2NF (partial dependencies):** student_email → student_name; class_code → class_title, room, teacher. They depend on only part of the key, so Ana's name is stored twice and Pottery Basics' details twice.
- **3NF (transitive dependency):** teacher → teacher_phone. The phone describes the teacher, not the sign-up or the class; Lee Wong's number is stored three times.

- **Tables:** students(email PK, name); teachers(id PK, name, phone); classes(code PK, title, room, teacher_id FK → teachers); signups(student_email FK → students, class_code FK → classes, PRIMARY KEY (student_email, class_code)).
- **Anomalies it removes:** update (change Lee Wong's phone in three rows), insert (a new class with no sign-ups has nowhere to go), delete (cancelling Cy's sign-up erases Glass Fusing).

TWO WEAK ANSWERS, RUN FOR REAL

Part 3 with COUNT(*) instead of COUNT(DISTINCT o.id): it counts order lines, so Alice gets 4 "orders" (Developing in the View row):

```
SELECT CONCAT(c.first_name, ' ', c.last_name) AS customer,
       COUNT(*) AS orders,
       SUM(oi.quantity * oi.price_at_time) AS revenue
FROM customers AS c
JOIN orders AS o       ON o.customer_id = c.id
JOIN order_items AS oi ON oi.order_id = o.id
WHERE c.id = 1
GROUP BY c.id, c.first_name, c.last_name;
```

```
+---------------+--------+---------+
| customer      | orders | revenue |
+---------------+--------+---------+
| Alice Johnson |      4 |   58.95 |
+---------------+--------+---------+
```

Part 1 with the window function in WHERE: MySQL refuses, because window functions are computed after WHERE (Developing in the first row; the fix is the second step):

```
SELECT c.name AS category, b.title, SUM(oi.quantity) AS copies,
       RANK() OVER (PARTITION BY c.id ORDER BY SUM(oi.quantity) DESC) AS place
FROM categories AS c
JOIN book_categories AS bc ON bc.category_id = c.id
JOIN books AS b            ON b.id = bc.book_id
JOIN order_items AS oi     ON oi.book_id = b.id
WHERE RANK() OVER (PARTITION BY c.id ORDER BY SUM(oi.quantity) DESC) = 1
GROUP BY c.id, c.name, b.id, b.title;
```

```
ERROR 3593 (HY000): You cannot use the window function 'rank' in this context.'
```

7.2 · TRANSACTIONS: SELF-CHECK

Q1

You type `START TRANSACTION;` and an UPDATE that succeeds, then close the terminal window without COMMIT. What happens to the change?

A) It is kept: the UPDATE succeeded

B) The server rolls it back when the connection ends

C) It waits, and is committed the next time you log in

D) It is kept, but only your account can see it

> **Why:** Nothing is committed unless you say COMMIT (or autocommit does it for you). A dropped connection ends the open transaction with a rollback (the lesson's Common Questions).
> **A** is wrong: Succeeding is not the same as being committed; inside a transaction the change is still provisional.
> **C** is wrong: Transactions do not survive the connection; a new login starts with nothing open.
> **D** is wrong: Uncommitted changes are visible to your own session only while it is open; once it closes they are gone.

Q2

What does the last statement print?

```
CREATE TABLE stock (id INT PRIMARY KEY, qty INT NOT NULL);
INSERT INTO stock VALUES (1, 5), (2, 5);
START TRANSACTION;
UPDATE stock SET qty = qty - 1 WHERE id = 1;
SAVEPOINT s1;
UPDATE stock SET qty = qty - 1 WHERE id = 2;
ROLLBACK TO SAVEPOINT s1;
UPDATE stock SET qty = qty + 3 WHERE id = 2;
COMMIT;
SELECT id, qty FROM stock ORDER BY id;
```

Two rows: id 1 qty `4`, id 2 qty `8`

> **Why:** The first UPDATE (id 1 to 4) happened before the savepoint and stays. ROLLBACK TO SAVEPOINT s1 undoes only the -1 on id 2. The +3 on id 2 comes after the rollback, so id 2 goes from 5 to 8, and COMMIT keeps both.

Q3

Which statement, typed inside an open transaction, commits that transaction on the spot?

A) `SAVEPOINT before_change;`

B) `SELECT price FROM books WHERE id = 1 FOR UPDATE;`

C) `ALTER TABLE books ADD COLUMN rating INT;`

D) `ROLLBACK TO SAVEPOINT before_change;`

Why: Schema changes (CREATE, ALTER, DROP, RENAME, TRUNCATE) cause an implicit commit before they run, so a later ROLLBACK has nothing left to undo: the trap the demo showed with CREATE TABLE.
A is wrong: A savepoint is a bookmark inside the transaction; the transaction stays open.
B is wrong: FOR UPDATE takes row locks inside the transaction; it does not end it.
D is wrong: Rolling back to a savepoint undoes part of the work and keeps the transaction open.

Q4

Window A runs `START TRANSACTION;` and `UPDATE accounts SET balance = 0 WHERE id = 1;` and does not commit. Window B then runs `SELECT balance FROM accounts WHERE id = 1 FOR UPDATE;`. What does B get?

A) 0, A's new value

B) The last committed balance, at once

C) ERROR 1213 at once: a deadlock

D) Nothing yet: B waits for A's row lock, and fails with ERROR 1205 if A holds it past the timeout

Why: A locking read (FOR UPDATE) needs the same row lock A's UPDATE holds, so it waits like a writer. A plain SELECT would not wait: it reads the committed snapshot.
A is wrong: No session ever sees another's uncommitted value under REPEATABLE READ.
B is wrong: That is what a PLAIN SELECT gets; FOR UPDATE must lock the row first.
C is wrong: A deadlock needs two transactions each waiting for the other; here only B waits.

Q5

In the transfer procedure's handler, why does RESIGNAL come right after ROLLBACK?

A) So the caller still receives the original error after the half-done changes are undone

B) RESIGNAL commits whatever the handler did not roll back

C) Without it, the handler would run again and again

D) Every handler must end with RESIGNAL or the procedure does not compile

> **Why:** ROLLBACK cleans up; RESIGNAL passes the same error (1644, 3819, ...) on, so the app knows the call failed and why. A handler that only rolls back hides the failure.
> **B** is wrong: RESIGNAL raises an error; it never commits anything.
> **C** is wrong: An EXIT handler runs once and leaves the procedure.
> **D** is wrong: RESIGNAL is optional syntax; leaving it out compiles fine, which is exactly why the silent version is a trap.

Q6

An app's checkout fails with `ERROR 1213 (40001): Deadlock found when trying to get lock; try restarting transaction`. What should the app do?

A) Nothing: the statements before the deadlock were saved

B) Run only the statement that failed again

C) Raise innodb_lock_wait_timeout so it never happens again

D) Run the whole transaction again from the start

> **Why:** InnoDB picked this transaction as the deadlock victim and rolled ALL of it back. The message is the advice: restart the transaction. Short transactions that lock rows in the same order make it rare.
> **A** is wrong: The victim's whole transaction was rolled back; nothing of it was saved.
> **B** is wrong: The earlier statements were undone too; repeating one statement alone leaves the job half done.
> **C** is wrong: A deadlock is detected at once, not after a timeout; the timeout setting does not affect it.

7.2 · TRANSACTIONS: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
CREATE TABLE counter (n INT);
INSERT INTO counter VALUES (1);
SET autocommit = 0;
UPDATE counter SET n = n + 10;
ROLLBACK;
SET autocommit = 1;
SELECT n FROM counter;
```

```
+------+
| n    |
+------+
|    1 |
+------+
```

Why: With autocommit off, the UPDATE waited for a decision; ROLLBACK discarded it, so n is still 1. Switching autocommit off for a whole session is rare: one forgotten COMMIT and the work is lost at disconnect.

P2

Predict the output, then run it.

```
CREATE TABLE seats (id INT PRIMARY KEY, taken INT NOT NULL);
INSERT INTO seats VALUES (1, 0), (2, 0);
START TRANSACTION;
UPDATE seats SET taken = 1 WHERE id = 1;
SAVEPOINT one_seat;
UPDATE seats SET taken = 1 WHERE id = 2;
ROLLBACK TO SAVEPOINT one_seat;
COMMIT;
SELECT SUM(taken) AS seats_taken FROM seats;
```

```
+-------------+
| seats_taken |
+-------------+
|           1 |
+-------------+
```

Why: Seat 1 was taken before the savepoint and kept; seat 2 was taken after it and rolled back. COMMIT keeps what is left: one seat.

7.3 · JSON COLUMNS: SELF-CHECK

Q7

On the web page, `CONCAT(title, ': ', attributes->'$.series')` prints `Dune: "Dune Chronicles"`. Which change removes the quotes?

A) Wrap the whole CONCAT in `TRIM()`

B) Use `JSON_EXTRACT` instead of `->`

C) Use `->>` instead of `->`

D) Store the series without quotes in the document

Why: -> returns a JSON value, and a JSON string keeps its double quotes. ->> (JSON_UNQUOTE of JSON_EXTRACT) returns plain text: use it whenever a value leaves JSON for display, CONCAT or comparison.

A is wrong: TRIM removes spaces, not quote characters.

B is wrong: JSON_EXTRACT is exactly what -> means; same quotes.

D is wrong: Every JSON string is stored with double quotes; that is the format, not a mistake.

Q8

What does the SELECT print?

```
CREATE TABLE shelf (id INT PRIMARY KEY, info JSON);
INSERT INTO shelf VALUES (1, '{"tags": ["new", "signed", "gift"], "copies": 2}');
SELECT info->>'$.tags[last]' AS last_tag, JSON_LENGTH(info->'$.tags') AS tag_count
FROM shelf;
```

`gift` and `3`

> **Why:** [last] is the last element of the array, and ->> returns it as plain text; JSON_LENGTH counts the three elements.

Q9

A document already has a `format` key. What does `JSON_INSERT(doc, '$.format', 'ebook')` return?

A) The document unchanged

B) The document with format replaced by ebook

C) The document with a second format key

D) An error: the key already exists

> **Why:** JSON_INSERT only adds missing keys; JSON_SET adds or replaces; JSON_REPLACE only replaces existing keys. All three return a changed copy, never an error, for a key that is or is not there.
>
> **B** is wrong: That is JSON_SET (or JSON_REPLACE).
>
> **C** is wrong: A JSON object never holds the same key twice.
>
> **D** is wrong: The JSON_* change functions do not raise errors for existing or missing keys.

Q10

`WHERE attributes->>'$.format' = 'hardcover'` is slow on a table of a million books. What is MySQL's fix?

A) `CREATE INDEX idx_attr ON books (attributes);`

B) A generated column for $.format, with an index on that column

C) Change the column type from JSON to TEXT

D) Write `JSON_EXTRACT` instead of `->>`

> **Why:** A JSON column cannot be indexed as a whole (error 3152). A VIRTUAL generated column pulls one path out, and its index is used even by the original ->> expression, as the demo's EXPLAIN showed.
> **A** is wrong: That is error 3152: JSON columns are indexed only through generated columns.
> **C** is wrong: TEXT loses validation and is no easier to index by path.
> **D** is wrong: Same lookup, same full scan.

Q11

What does this print?

```
SELECT JSON_OBJECT('title', 'Dune', 'id', 1, 'price', 14.99) AS doc;
```

`{"id": 1, "price": 14.99, "title": "Dune"}`

> **Why:** MySQL stores JSON in a binary form with the keys of every object sorted: shorter keys first, then alphabetically. The data is the same; only the order changed, so never rely on key order.

Q12

The store wants to record four things. Which ONE belongs in a JSON column rather than a regular column or table?

A) The customer an order belongs to

B) A book's price

C) The line items of an order

D) An optional gift-wrap note some orders have, with a color and a message

> **Why:** Optional, varied, display-only details are what JSON is for. The other three are core data: you join on the customer, sum the price, and line items are a list of related things that need their own table with foreign keys.
> **A** is wrong: You join on it and it needs a foreign key: a column.
> **B** is wrong: Every book has one, and every total uses it: a DECIMAL column.
> **C** is wrong: A list of related rows is a child table (order_items), the lesson's 'when not to use JSON' table.

7.3 · JSON COLUMNS: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
SELECT JSON_VALID('{"a": 1}') AS ok,
       JSON_VALID("{'a': 1}")  AS single_quotes,
       JSON_TYPE('[1, 2]')     AS kind;
```

```
+------+---------------+-------+
| ok   | single_quotes | kind  |
+------+---------------+-------+
|    1 |             0 | ARRAY |
+------+---------------+-------+
```

> **Why:** JSON_VALID accepts double quotes only (single quotes give 0); JSON_TYPE names the kind of value: ARRAY.

P4

Predict the output, then run it.

```
SET @doc = '{"format": "paperback"}';
SELECT JSON_INSERT(@doc, '$.format', 'ebook')  AS ins,
       JSON_REPLACE(@doc, '$.pages', 300)      AS rep;
```

```
+-------------------------+-------------------------+
| ins                     | rep                     |
+-------------------------+-------------------------+
| {"format": "paperback"} | {"format": "paperback"} |
+-------------------------+-------------------------+
```

> **Why:** JSON_INSERT never replaces an existing key (format stays paperback); JSON_REPLACE never adds a missing key (no pages). Both return the document unchanged here; JSON_SET would have done both.

LAB REFERENCE SOLUTIONS

- LAB 7.2-A: LABS/MODULE-11/L7_2-TRANSACTIONS/LOYALTY_POINTS_STARTER.SQL → LOYALTY_POINTS_SOLUTION.SQL

The solution prints:

```
+-------------+--------+
| customer_id | points |
+-------------+--------+
|           6 |     15 |
|           7 |     25 |
+-------------+--------+
+---------------+--------------+
| orders_inside | items_inside |
+---------------+--------------+
|             7 |           10 |
+---------------+--------------+
+--------------+-------------+
| orders_after | items_after |
+--------------+-------------+
|           12 |          18 |
+--------------+-------------+
+-------------+--------+
| customer_id | points |
+-------------+--------+
|           1 |    350 |
|           2 |    170 |
|           4 |     10 |
|           6 |     15 |
|           7 |     25 |
+-------------+--------+
+-------------+--------+
| customer_id | points |
+-------------+--------+
|           2 |    170 |
+-------------+--------+
+----------+
| log_rows |
+----------+
|        0 |
+----------+
+------------+-----------------+
| autocommit | isolation       |
+------------+-----------------+
|          1 | REPEATABLE-READ |
+------------+-----------------+
```

What each TODO teaches: (1) two UPDATEs that only make sense together, committed as one; (2) a rehearsal: the DELETE of 5 orders cascades to 8 order lines inside the transaction, and ROLLBACK brings all of them back; (3) ROLLBACK TO SAVEPOINT keeps the bonus and drops the slip that zeroed everyone (as shipped, the zeros are committed); (4) the implicit commit: as shipped, CREATE TABLE ends the transaction, the INSERT then autocommits and the ROLLBACK undoes nothing (Bob 10, log_rows 1); moved above START TRANSACTION, the ROLLBACK undoes both (Bob 170, log_rows 0).

LAB 7.2-B: LABS/MODULE-11/L7_2-TRANSACTIONS/RETURN_ITEM_STARTER.SQL → RETURN_ITEM_SOLUTION.SQL

The solution prints:

```
+--------+-------+
| orders | items |
+--------+-------+
|     12 |    18 |
+--------+-------+
+--------+-------+
| orders | items |
+--------+-------+
|     11 |    16 |
+--------+-------+
+-------------+--------+
| customer_id | points |
+-------------+--------+
|           1 |    307 |
|           4 |     18 |
+-------------+--------+
+----+-------------+
| id | customer_id |
+----+-------------+
| 12 |           4 |
+----+-------------+
```

Then, at the prompt (real output, same session as the solution):

```
CALL return_item(99);
CALL return_item(13);
SELECT (SELECT COUNT(*) FROM orders) AS orders,
       (SELECT COUNT(*) FROM order_items) AS items,
       (SELECT COUNT(*) FROM order_items WHERE id = 13) AS line_13_kept;
```

```
ERROR 1644 (45000): No such order line
ERROR 1644 (45000): Customer has no loyalty account
+--------+-------+--------------+
| orders | items | line_13_kept |
+--------+-------+--------------+
|     11 |    16 |            1 |
+--------+-------+--------------+
```

Reading it: line 17 was one of order 12's two lines, so order 12 stays; line 3 was order 2's only line, so the order goes too. Points are FLOOR(quantity × price): 8 for Dave, 7 for Alice. Eve has no loyalty row, so the UPDATE changes nothing, the SIGNAL fires after both DELETEs, and the handler rolls them back: line 13 is still there. Without TODOs

1 to 3 (the starter as shipped), the same call deletes Eve's line and THEN fails: the half-done return the lesson warns about.

HANDOUT: LABS/MODULE-11/L7_2-TRANSACTIONS/DEBUG_LOST_POINTS.SQL (DELIBERATELY BROKEN)

As shipped, MySQL prints (the table, then the two errors at the lines where they happened):

```
ERROR 3819 (HY000): Check constraint 'chk_points' is violated.
ERROR 1305 (42000): SAVEPOINT before_bonuses does not exist
+-------------+--------+
| customer_id | points |
+-------------+--------+
|           1 |      0 |
|           2 |    340 |
|           4 |     20 |
|           6 |     80 |
|           7 |      0 |
+-------------+--------+
```

1. **A, 3819:** Grace has 0 points, so her -50 breaks the CHECK. The failed UPDATE undid only itself; COMMIT then saved Bob's +50: 50 points from nothing. Fix: ROLLBACK after the error (in code, a handler).
2. **B, 1305:** the savepoint is `before_bonus`, the ROLLBACK names `before_bonuses`. The transaction stays open with everyone doubled, and COMMIT keeps it (Bob 170 × 2 = 340). Fix: the right name.
3. **C, silent:** CREATE TABLE committed Alice's zero before it ran, so the ROLLBACK had nothing to undo. Fix: create the table before START TRANSACTION.

Fixed (real run; the 3819 still appears, because the statement still fails, but nothing is saved):

```
ERROR 3819 (HY000): Check constraint 'chk_points' is violated.
+-------------+--------+
| customer_id | points |
+-------------+--------+
|           1 |    300 |
|           2 |    120 |
|           4 |     10 |
|           6 |     40 |
|           7 |      0 |
+-------------+--------+
```

LAB 7.3-A: LABS/MODULE-11/L7_3-JSON-COLUMNS/READER_PREFS_STARTER.SQL → READER_PREFS_SOLUTION.SQL

The solution prints:

```
+----+------------+
| id | first_name |
+----+------------+
|  1 | Alice      |
|  6 | Frank      |
|  7 | Grace      |
+----+------------+
+----+------------+
| id | first_name |
+----+------------+
|  1 | Alice      |
|  7 | Grace      |
+----+------------+
+----+------------+--------------------------------+-----------------+-------------+---
| id | first_name | genres                         | first_genre     | genre_count | en
+----+------------+--------------------------------+-----------------+-------------+---
|  1 | Alice      | ["Science Fiction", "Classic"] | Science Fiction |           2 | tr
|  4 | Dave       | NULL                           | NULL            |        NULL | NU
|  6 | Frank      | []                             | NULL            |           0 | NU
|  7 | Grace      | ["Fantasy", "Classic"]         | Fantasy         |           2 | NU
+----+------------+--------------------------------+-----------------+-------------+---
+----+------------+------------+
| id | first_name | has_genres |
+----+------------+------------+
|  1 | Alice      |          1 |
|  2 | Bob        |          1 |
|  3 | Carol      |       NULL |
|  4 | Dave       |          0 |
|  5 | Eve        |       NULL |
|  6 | Frank      |          1 |
|  7 | Grace      |          1 |
|  8 | Hank       |       NULL |
+----+------------+------------+
+----+------------+-----------------------------------------------------------------------
| id | first_name | prefs
+----+------------+-----------------------------------------------------------------------
|  2 | Bob        | {"genres": ["Dystopian"], "newsletter": true}
|  7 | Grace      | {"genres": ["Fantasy", "Classic", "Science Fiction"], "newsletter":
+----+------------+-----------------------------------------------------------------------
+------------+-----------+
| newsletter | customers |
+------------+-----------+
| NULL       |         3 |
| false      |         1 |
| true       |         4 |
+------------+-----------+
```

Reading it: a JSON `true` read with `->>` is the text 'true' (comparing with the number 1 or TRUE finds nothing). Dave's prefs exist without a genres key, so JSON_CONTAINS_PATH says 0; Carol, Eve and Hank have no document at all, so it says NULL: the difference between "missing key" and "missing document". Grace's document lost the language key and gained a genre, and its keys come back sorted. The generated column turns three states into a GROUP BY: true, false and NULL (no prefs).

- LAB 7.3-B: LABS/MODULE-11/L7_3-JSON-COLUMNS/ORDER_DOCUMENTS_STARTER.SQL → ORDER_DOCUMENTS_SOLUTION.SQL

The solution prints (an empty first line marks the silent SET):

```
+----------------------------------------------------------------------+
| order_doc                                                            |
+----------------------------------------------------------------------+
| {"date": "2026-01-15", "lines": 2, "total": 34.97, "order_id": 1} |
| {"date": "2026-02-20", "lines": 1, "total": 7.99, "order_id": 2}  |
| {"date": "2026-04-05", "lines": 1, "total": 15.99, "order_id": 8} |
+----------------------------------------------------------------------+
+---------------------------------------+
| best_sellers                          |
+---------------------------------------+
| {"1984": 4, "Dune": 3, "The Hobbit": 3} |
+---------------------------------------+
+---------+------+------------------------+----------+
| book_id | qty  | title                  | in_stock |
+---------+------+------------------------+----------+
|       4 |    5 | The Great Gatsby       |        0 |
|       8 |    3 | The Catcher in the Rye |        0 |
|      42 |    1 | NULL                   |     NULL |
+---------+------+------------------------+----------+
+-----------------------+--------------------+
| supplier              | still_out_of_stock |
+-----------------------+--------------------+
| Riverside Book Supply |                  0 |
+-----------------------+--------------------+
```

Reading it: JSON_OBJECT accepts aggregates in a grouped query; the total stays a number. JSON_OBJECTAGG sorts its keys (shortest first), so the order is stable even though the rows are not ordered. JSON_TABLE's LEFT JOIN exposes the supplier's mistake (book 42 does not exist) before anything is updated; the UPDATE joins only real books. Learners who used an inner JOIN in step 3 never see book 42: ask what the supplier should be told.

- HANDOUT: LABS/MODULE-11/L7_3-JSON-COLUMNS/DEBUG_BAD_JSON.SQL (DELIBERATELY BROKEN)

As shipped, MySQL prints:

```
ERROR 3140 (22032): Invalid JSON text: "Missing a name for object member." at position
ERROR 3143 (42000): Invalid JSON path expression. The error is around character positi
ERROR 3152 (42000): JSON column 'items' supports indexing only via generated columns o
+--------------------+
| label              |
+--------------------+
| Alice ("birthday") |
+--------------------+
Empty set
+-------------+-------+-------------------------------------------------------------+
| customer_id | owner | items                                                       |
+-------------+-------+-------------------------------------------------------------+
|           1 | Alice | {"public": true, "book_ids": [1, 9], "occasion": "birthday"} |
|           7 | Grace | {"public": false, "book_ids": [1, 6]}                       |
+-------------+-------+-------------------------------------------------------------+
```

1. **A, 3140:** single quotes inside the document. Fix: `'{"book_ids": [2, 11], "public": true}'` (or JSON_OBJECT).
2. **B, 3143:** the path has no `$`. Fix: `items->>'$.book_ids'`.
3. **C, 3152:** a JSON column cannot be indexed whole. Fix: a generated column for one path (`occasion`) with an index on it.
4. **D, silent:** `->` keeps the JSON quotes: `Alice ("birthday")`. Fix: `->>`.
5. **E, silent:** `$.bookIds` does not exist (the key is `book_ids`), so MEMBER OF finds nobody and nothing warns. Fix: the real key name.

Fixed (real run):

```
+-------+---------+
| owner | books   |
+-------+---------+
| Alice | [1, 9]  |
| Bob   | [2, 11] |
| Grace | [1, 6]  |
+-------+---------+
+------------------+
| label            |
+------------------+
| Alice (birthday) |
+------------------+
+-------+
| owner |
+-------+
| Alice |
| Grace |
+-------+
+-------------+-------+-------------------------------------------------------------------+
| customer_id | owner | items                                                             |
+-------------+-------+-------------------------------------------------------------------+
|           1 | Alice | {"public": true, "book_ids": [1, 9], "occasion": "birthday"} |
|           2 | Bob   | {"public": true, "book_ids": [2, 11]}                        |
|           7 | Grace | {"public": false, "book_ids": [1, 6]}                        |
+-------------+-------+-------------------------------------------------------------------+
```

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 7.2 · TRANSACTIONS

1. **A** Atomicity: Atomicity: all of a transaction's changes are kept, or none of them. The order row and its item rows succeed or fail together.

2. **C** The last committed value, immediately: InnoDB gives plain readers the last committed version of the row, without waiting. B sees A's change only after A commits.

3. **B** The first UPDATE only: In MySQL a failed statement rolls back only itself and the transaction stays open, so COMMIT saves the first UPDATE. Roll back yourself, or use a handler.

4. **B** CREATE TABLE demo_audit (note VARCHAR(100)): DDL such as CREATE TABLE causes an implicit commit before it runs, so a later ROLLBACK cannot undo the earlier changes.

5. **C** Another transaction held the row's lock longer than the timeout: Another transaction held a lock on the row, and your statement gave up after innodb_lock_wait_timeout seconds (50 by default). Only that statement failed.

LESSON 7.3 · JSON COLUMNS

1. **C** attributes->>'$.format': The double arrow extracts and unquotes in one step: it is short for JSON_UNQUOTE(JSON_EXTRACT(attributes, '$.format')).

2. **A** JSON keys are case-sensitive, so $.FORMAT is NULL for every book: JSON keys are case-sensitive. No document has a FORMAT key, so the expression is NULL for every row, and NULL never equals anything. MySQL does not warn about a missing key.

3. **B** Add a generated column for $.format and index that column: A generated column extracts the value from each document, and an ordinary index on that column can be used, even by queries that write the original JSON expression.

4. **C** It fails with error 3140, because JSON keys must be double-quoted strings: The key format has no double quotes, so the text is not valid JSON and the JSON column rejects it with error 3140. The row keeps its old value.

5. **B** The price, which every book has and every order total uses: Every book has a price, it needs an exact DECIMAL type and NOT NULL, and orders and reports calculate with it all the time. That is regular-column data.

EXIT TICKET

1. **Committed after 3819:** every change made before the failed statement in that transaction; the failed statement undid only itself. ROLLBACK after an error, or a handler.

2. **The handler:** `DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;` (declared after the variables, before the other statements).

3. **JSON or column:** JSON for optional, varied, display-only details (series, edition, awards, illustrated); a column for what every row has or what you join, sum or constrain (price, author_id, customer_id). Accept any pair with a reason from the lesson's table.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 A ("kept, because the UPDATE succeeded").** Success is not commitment. Rerun the two-window step: B never saw the change until COMMIT.
- **Q3 A or D (a savepoint statement ends the transaction).** Confuses bookmarks with endings; only COMMIT, ROLLBACK and implicit commits end it.
- **Q4 B ("the committed value at once").** Right for a plain SELECT, wrong for FOR UPDATE: the whole point of a locking read is to wait.
- **Q6 B ("repeat the failed statement").** The victim's whole transaction is gone; repeating one statement leaves half a transfer.

- **Q7 B (JSON_EXTRACT).** Thinks the function is different from the arrow; it is the same thing. Only ->> (or JSON_UNQUOTE) removes quotes.
- **Q12 C (line items in JSON).** The most tempting design mistake of the lesson: a list of related things with prices and books needs its own table and foreign keys.
- **Checkpoint, Part 3 orders = 4 for Alice.** Counting rows after a join counts the many side; COUNT(DISTINCT) or aggregate before joining.

INSTRUCTOR KIT · ANSWER KEY

Session 12: Users, Permissions and Backups

Answers and reasoning for every Session 12 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs (with the logins as demo accounts), the 8.1 shell runs, the lesson pages' own quizzes and the exit ticket.

Session 12 | 12 self-check items | Confidential

WARM-UP

1. **W1** (7.2) No. ALTER TABLE caused an implicit commit before it ran, so the UPDATE was already committed and the ROLLBACK had nothing to undo.
2. **W2** (7.2) 1205: a lock wait timed out; only the waiting statement failed and the transaction is still open. 1213: a deadlock; InnoDB rolled the whole transaction back, so retry all of it.
3. **W3** (7.3) ->> : it returns plain text; -> would return the JSON string "hardcover" with its quotes, and the comparison would fail.
4. **W4** (7.3) It turns a JSON array (in a column or a literal) into rows with typed columns that you can join, group and filter.
5. **W5** (7.2 + 7.3) 2 and ["gift"]: the JSON_SET happened inside the transaction and the ROLLBACK discarded it.

W5 prints:

```
+------+----------+
| qty  | tags     |
+------+----------+
| 2    | ["gift"] |
+------+----------+
```

7.4 · USER MANAGEMENT & PERMISSIONS: SELF-CHECK

Q1

A server has `'app'@'localhost'` and `'app'@'%'`. How many accounts is that?

A) One account that may connect from two places

B) Two separate accounts, each with its own password and privileges

C) One account: % already includes localhost, so the second row is ignored

D) None: the second CREATE USER failed because the name was taken

> **Why:** An account is the pair 'user'@'host'. The same name with another host is another account, which is why SHOW GRANTS FOR 'shop_reports'@'%' fails with 1141 when only @'localhost' exists.
> **A** is wrong: Privileges and passwords belong to each pair separately.
> **C** is wrong: Both rows are real accounts; MySQL picks the most specific host that matches a login.
> **D** is wrong: The pair is the identity; the same user name with a different host is allowed.

Q2

How many lines does the SHOW GRANTS print, and what are they?

```
CREATE USER 'kit_quiz'@'localhost' IDENTIFIED BY 'CHANGE-ME-quiz-1';
GRANT SELECT ON bookstore.* TO 'kit_quiz'@'localhost';
GRANT INSERT ON bookstore.orders TO 'kit_quiz'@'localhost';
SHOW GRANTS FOR 'kit_quiz'@'localhost';
```

Three: `GRANT USAGE ON *.*`, `GRANT SELECT ON \`bookstore\`.*`, `GRANT INSERT ON \`bookstore\`.\`orders\``

> **Why:** Every account has the USAGE line ("may connect"). Then one line per level: the database-level SELECT and the table-level INSERT stay separate lines because they are at different levels.

Q3

An app account gets `ERROR 1143 (42000): SELECT command denied ... for column 'city' in table 'customers'`. What does it mean?

A) The password is wrong

B) The account may not read the customers table at all

C) The account is locked

D) The account has column privileges on customers, and city is not one of them

> **Why:** 1143 is the column-level cousin of 1142: the account may read some columns of customers (granted with SELECT (col, ...)) but not this one. SELECT * fails with 1142 for the same reason.
> **A** is wrong: A wrong password is 1045, at login.
> **B** is wrong: That is 1142 (table level); 1143 names a column.
> **C** is wrong: A locked account is 3118, at login.

Q4

Tom and Lena have the role bookstore_sales as a default role. An administrator runs `GRANT INSERT ON bookstore.customers TO 'bookstore_sales';`. When can they insert customers?

A) Right away, both of them, and every future account given the role

B) Only after FLUSH PRIVILEGES

C) Only after each is granted INSERT on customers directly

D) Only accounts created after the GRANT

> **Why:** A role is a bundle: changing it changes every account that has it, at once. That is the whole point of roles (and GRANT never needs FLUSH PRIVILEGES).
> **B** is wrong: GRANT, REVOKE and CREATE USER update the privileges in memory immediately.
> **C** is wrong: That would defeat the role.
> **D** is wrong: Existing members get it too.

Q5

`shop_reports` has `GRANT SELECT ON bookstore.*`. What does `REVOKE SELECT ON bookstore.books FROM 'shop_reports'@'localhost';` do?

A) Hides books and keeps every other table readable

B) Removes SELECT on the whole database

C) Fails with ERROR 1147: there is no table-level grant on books to take away

D) Works only after FLUSH PRIVILEGES

> **Why:** Revoke at the level you granted. A database-level grant cannot have one table cut out of it; to hide one table, grant table by table (or through views).
> **A** is wrong: MySQL has no "all tables except" grant.
> **B** is wrong: The table-level REVOKE does not touch the database-level grant.
> **D** is wrong: The statement fails; FLUSH PRIVILEGES changes nothing about that.

Q6

Write the statement that disables `'kit_mailer'@'localhost'` without losing its grants.

```
ALTER USER 'kit_mailer'@'localhost' ACCOUNT LOCK;
```

> **Why:** A locked account keeps its privileges; a login fails with 3118 until ALTER USER ... ACCOUNT UNLOCK. DROP USER would delete the account and its grants.

7.4 · USER MANAGEMENT & PERMISSIONS: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
SELECT COUNT(*) AS kit_accounts
FROM mysql.user
WHERE user LIKE 'kit\_%';
```

```
+--------------+
| kit_accounts |
+--------------+
|            0 |
+--------------+
```

Why: A fresh 9.7 server has no kit_ accounts (the labs drop theirs at the end). If this is not 0 on a learner's laptop, a lab stopped halfway: run its DROP USER line.

8.1 · IMPORTING & EXPORTING DATA: SELF-CHECK

Q7

Where do you type mysqldump?

A) At the mysql> prompt, ending with ;

B) In phpMyAdmin's SQL tab

C) At the terminal's shell prompt ($ or PS>), not inside the mysql client

D) Inside a SOURCE file

Why: mysqldump is a separate program that connects like any client. Typed at mysql> the server receives it as SQL and answers ERROR 1064 (debug_import.sql, bug A).

A is wrong: That is the 1064 the demo showed.

B is wrong: phpMyAdmin sends SQL to the server too; it has its own Export tab instead.

D is wrong: A SOURCE file is SQL, read by the client and sent to the server.

Q8

Which mysqldump option puts the stored procedures and functions into the backup?

A) `--routines`

B) `--events`

C) `--single-transaction`

D) `--databases`

> **Why:** Routines are left out by default; --routines adds them (--events adds scheduled events; triggers are in by default).
> **B** is wrong: --events adds scheduled events, not procedures.
> **C** is wrong: --single-transaction makes the dump consistent without locking tables.
> **D** is wrong: --databases adds CREATE DATABASE and USE lines.

Q9

What does this print?

```
SELECT STR_TO_DATE('07/04/1976', '%m/%d/%Y') AS published,
       NULLIF('', '') AS pages;
```

`1976-07-04` and `NULL`

> **Why:** STR_TO_DATE reads the text with the US pattern month/day/year and returns a real DATE. NULLIF returns NULL when its two arguments are equal: the trick that turns empty CSV fields into NULL.

Q10

You restore a dump made WITHOUT `--databases` using `mysql -u root -p < bookstore_backup.sql` and get `ERROR 1046 (3D000) at line 22: No database selected`. What is the fix?

A) Take the dump again with `--set-gtid-purged=OFF`

B) Run `SET FOREIGN_KEY_CHECKS = 0` first

C) Use `SOURCE` instead of `<`

D) Name the target database: `mysql -u root -p bookstore_test < bookstore_backup.sql` (or dump with --databases)

> **Why:** Without --databases the file has no CREATE DATABASE or USE line, so you choose the target when you restore (real output from the demo's shell run).
> **A** is wrong: That fixes ERROR 3546 (GTIDs), a different failure.
> **B** is wrong: Foreign keys are not the problem; no database is selected.
> **C** is wrong: SOURCE without a current database fails the same way.

Q11

Why load a stranger's CSV into a staging table before touching books?

A) LOAD DATA cannot load into a table that has foreign keys

B) To look at, convert and check the rows (dates, empty values, missing authors) before any real table changes

C) Staging tables load faster than real tables

D) LOAD DATA LOCAL only works with staging tables

> **Why:** A staging table with the file's columns lets you SELECT what arrived, find what is missing with a LEFT JOIN, and then copy only the good rows in one transaction.
> **A** is wrong: LOAD DATA can load into any table you may INSERT into.
> **C** is wrong: Speed is not the reason.
> **D** is wrong: LOCAL works with any table.

Q12

You run `SELECT title FROM books INTO OUTFILE '/var/lib/mysql-files/titles.txt';` a second time. What happens?

A) The file is overwritten with the new rows

B) The rows are added to the end of the file

C) ERROR 1086: the file already exists

D) MySQL writes titles(1).txt

> **Why:** INTO OUTFILE never overwrites, which protects earlier exports and system files. Write to a new name (add the date) or delete the old file at the terminal.
> **A** is wrong: Overwriting is exactly what MySQL refuses to do.
> **B** is wrong: There is no append mode.
> **D** is wrong: MySQL never renames for you.

8.1 · IMPORTING & EXPORTING DATA: PREDICT, THEN RUN

P2

Predict the output, then run it.

```
SELECT STR_TO_DATE('31.12.1999', '%d.%m.%Y') AS eu_date,
       STR_TO_DATE('31.12.1999', '%m/%d/%Y') AS us_pattern;
```

```
+------------+------------+
| eu_date    | us_pattern |
+------------+------------+
| 1999-12-31 | NULL       |
+------------+------------+
```

> **Why:** The European pattern reads the text; the US pattern does not match it, so STR_TO_DATE returns NULL (with warning 1411). In an INSERT, strict mode turns that into an error (debug_import.sql, bug C).

P3

Predict the output, then run it.

```
SELECT NULLIF('', '') AS empty_to_null,
       NULLIF('412', '') AS pages_text,
       COALESCE(NULL, '') AS for_people;
```

```
+---------------+------------+------------+
| empty_to_null | pages_text | for_people |
+---------------+------------+------------+
| NULL          | 412        |            |
+---------------+------------+------------+
```

> **Why:** NULLIF('', '') is NULL (the empty CSV field becomes a real NULL); NULLIF('412', '') keeps the text; COALESCE(NULL, '') turns NULL into an empty string, for files people open.

SELF-CHECK CODE, RUN FOR REAL

Q2's SHOW GRANTS prints:

```
+-----------------------------------------------------------------+
| Grants for kit_quiz@localhost                                   |
+-----------------------------------------------------------------+
| GRANT USAGE ON *.* TO `kit_quiz`@`localhost`                    |
| GRANT SELECT ON `bookstore`.* TO `kit_quiz`@`localhost`         |
| GRANT INSERT ON `bookstore`.`orders` TO `kit_quiz`@`localhost` |
+-----------------------------------------------------------------+
```

Q12, the same INTO OUTFILE twice (the second line is the real answer):

```
ERROR 1086 (HY000): File '/var/lib/mysql-files/titles.txt' already exists
```

LAB REFERENCE SOLUTIONS

LAB 7.4-A: LABS/MODULE-12/L7_4-USER-MANAGEMENT/WAREHOUSE_ACCOUNTS_STARTER.SQL → WAREHOUSE_ACCOUNTS_SOLUTION.SQL

The solution prints (as root):

```
+---------------------------------------------------+
| Grants for kit_warehouse@localhost                |
+---------------------------------------------------+
| GRANT USAGE ON *.* TO `kit_warehouse`@`localhost` |
+---------------------------------------------------+
+-----------------------------------------------------------------------------------------
| Grants for kit_warehouse@localhost
+-----------------------------------------------------------------------------------------
| GRANT USAGE ON *.* TO `kit_warehouse`@`localhost`
| GRANT SELECT ON `bookstore`.`authors` TO `kit_warehouse`@`localhost`
| GRANT SELECT, UPDATE (`in_stock`) ON `bookstore`.`books` TO `kit_warehouse`@`localhos
+-----------------------------------------------------------------------------------------
+-----------------------------------------------------------------------------------------
| Grants for kit_accounting@localhost
+-----------------------------------------------------------------------------------------
| GRANT USAGE ON *.* TO `kit_accounting`@`localhost`
| GRANT SELECT (`first_name`, `id`, `last_name`) ON `bookstore`.`customers` TO `kit_acc
| GRANT SELECT ON `bookstore`.`order_items` TO `kit_accounting`@`localhost`
| GRANT SELECT ON `bookstore`.`orders` TO `kit_accounting`@`localhost`
+-----------------------------------------------------------------------------------------
+------------------------------+-------------+----------------+
| GRANTEE                      | TABLE_NAME  | PRIVILEGE_TYPE |
+------------------------------+-------------+----------------+
| 'kit_accounting'@'localhost' | orders      | SELECT         |
| 'kit_accounting'@'localhost' | order_items | SELECT         |
| 'kit_warehouse'@'localhost'  | authors     | SELECT         |
| 'kit_warehouse'@'localhost'  | books       | SELECT         |
+------------------------------+-------------+----------------+
+------------------------------+------------+-------------+----------------+
| GRANTEE                      | TABLE_NAME | COLUMN_NAME | PRIVILEGE_TYPE |
+------------------------------+------------+-------------+----------------+
| 'kit_accounting'@'localhost' | customers  | first_name  | SELECT         |
| 'kit_accounting'@'localhost' | customers  | id          | SELECT         |
| 'kit_accounting'@'localhost' | customers  | last_name   | SELECT         |
| 'kit_warehouse'@'localhost'  | books      | in_stock    | UPDATE         |
+------------------------------+------------+-------------+----------------+
+-----------------------------------------------------------------------------------------
| Grants for kit_accounting@localhost
+-----------------------------------------------------------------------------------------
| GRANT USAGE ON *.* TO `kit_accounting`@`localhost`
| GRANT SELECT (`first_name`, `id`, `last_name`) ON `bookstore`.`customers` TO `kit_acc
| GRANT SELECT ON `bookstore`.`orders` TO `kit_accounting`@`localhost`
+-----------------------------------------------------------------------------------------
+----------------+-----------+-----------------------+----------------+
| user           | host      | plugin                | account_locked |
+----------------+-----------+-----------------------+----------------+
| kit_accounting | localhost | caching_sha2_password | N              |
```

```
| kit_warehouse  | localhost | caching_sha2_password | Y              |
+----------------+-----------+-----------------------+----------------+
```

TODO 6, logged in as kit_warehouse in a second session before the lock (real output):

```
UPDATE books SET in_stock = FALSE WHERE id = 1;
UPDATE books SET price = 0 WHERE id = 1;
```

```
Query OK, 1 row affected
Rows matched: 1  Changed: 1  Warnings: 0
ERROR 1143 (42000): UPDATE command denied to user 'kit_warehouse'@'localhost' for colu
```

Reading it: MySQL merges the table-level SELECT and the column-level UPDATE on books into one SHOW GRANTS line, but information_schema keeps them apart (TABLE_PRIVILEGES vs COLUMN_PRIVILEGES). The REVOKE names the table it was granted on. A locked account keeps its grants (SHOW GRANTS still lists them) and its login fails with 3118. The UPDATE of in_stock also needs SELECT on books for its WHERE: that is why the warehouse has both.

- LAB 7.4-B: LABS/MODULE-12/L7_4-USER-MANAGEMENT/STAFF_ROLES_STARTER.SQL → STAFF_ROLES_SOLUTION.SQL

The solution prints:

```
+---------+-------------------+
| user    | default_role_user |
+---------+-------------------+
| kit_ana | kit_clerk         |
| kit_ben | kit_cashier       |
| kit_ben | kit_clerk         |
+---------+-------------------+
+-----------------------------------------------------------------------+
| Grants for kit_ben@localhost                                          |
+-----------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `kit_ben`@`localhost`                           |
| GRANT SELECT ON `bookstore`.`authors` TO `kit_ben`@`localhost`        |
| GRANT SELECT ON `bookstore`.`books` TO `kit_ben`@`localhost`          |
| GRANT INSERT ON `bookstore`.`order_items` TO `kit_ben`@`localhost`    |
| GRANT INSERT ON `bookstore`.`orders` TO `kit_ben`@`localhost`         |
| GRANT `kit_cashier`@`%`,`kit_clerk`@`%` TO `kit_ben`@`localhost`      |
+-----------------------------------------------------------------------+
+-------------------------------------------------------------------+
| Grants for kit_ana@localhost                                      |
+-------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `kit_ana`@`localhost`                       |
| GRANT SELECT ON `bookstore`.`authors` TO `kit_ana`@`localhost`    |
| GRANT SELECT ON `bookstore`.`books` TO `kit_ana`@`localhost`      |
| GRANT SELECT ON `bookstore`.`customers` TO `kit_ana`@`localhost`  |
| GRANT `kit_clerk`@`%` TO `kit_ana`@`localhost`                    |
+-------------------------------------------------------------------+
+-----------------------------------------------+
| Grants for kit_ben@localhost                  |
+-----------------------------------------------+
| GRANT USAGE ON *.* TO `kit_ben`@`localhost`   |
| GRANT `kit_clerk`@`%` TO `kit_ben`@`localhost` |
+-----------------------------------------------+
+-------------+-----------+-----------------+
| user        | host      | account_locked  |
+-------------+-----------+-----------------+
| kit_ana     | localhost | N               |
| kit_ben     | localhost | N               |
| kit_cashier | %         | Y               |
| kit_clerk   | %         | Y               |
+-------------+-----------+-----------------+
```

As kit_ana, after SET DEFAULT ROLE (real output, second session):

```
SELECT COUNT(*) AS books FROM books;
INSERT INTO orders (customer_id, order_date) VALUES (1, '2026-07-01');
```

```
+-------+
| books |
+-------+
|    11 |
+-------+
ERROR 1142 (42000): INSERT command denied to user 'kit_ana'@'localhost' for table 'orde
```

Reading it: without TODO 3 the default_roles query is empty and Ana's first SELECT would fail with 1142. Step 5 changes kit_clerk once and both clerks follow. After TODO 6 Ben's own grants list only kit_clerk. Roles live in mysql.user as locked accounts with host %.

HANDOUT: LABS/MODULE-12/L7_4-USER-MANAGEMENT/DEBUG_GRANTS.SQL (DELIBERATELY BROKEN)

As shipped:

```
ERROR 1410 (42000): You are not allowed to create a user with GRANT
ERROR 1141 (42000): There is no such grant defined for user 'kit_mailer' on host '%'
ERROR 1396 (HY000): Operation CREATE USER failed for 'kit_mailer'@'localhost' (User acc
ERROR 1147 (42000): There is no such grant defined for user 'kit_mailer' on host 'local
+-----------------------------------------------------------+
| Grants for kit_mailer@localhost                           |
+-----------------------------------------------------------+
| GRANT USAGE ON *.* TO `kit_mailer`@`localhost`            |
| GRANT SELECT ON `bookstore`.* TO `kit_mailer`@`localhost` |
+-----------------------------------------------------------+
```

1. **A, 1410:** `kit_mailr` does not exist, and modern MySQL never creates an account in GRANT. Fix: the right name.
2. **B, 1141:** `'kit_mailer'@'%'` is a different account from `@'localhost'`. Fix: the exact pair.
3. **C, 1396:** the account already exists. Fix: CREATE USER IF NOT EXISTS (or DROP USER IF EXISTS first, as the top of the file does).
4. **D, 1147:** SELECT was granted on `bookstore.*`, not on orders. A database-level grant cannot lose one table; the real fix is to revoke the database-level SELECT and grant only what the tool needs (the customers columns).

Fixed (real run):

```
+-----------------------------------------------------------------------------------------
| Grants for kit_mailer@localhost
+-----------------------------------------------------------------------------------------
| GRANT USAGE ON *.* TO `kit_mailer`@`localhost`
| GRANT SELECT (`email`, `first_name`) ON `bookstore`.`customers` TO `kit_mailer`@`loca
+-----------------------------------------------------------------------------------------
```

- LAB 8.1-A: LABS/MODULE-12/L8_1-IMPORTING-EXPORTING/NEW_TITLES_IMPORT_STARTER.SQL → NEW_TITLES_IMPORT_SOLUTION.SQL

The solution prints:

```
+----------------+-------+-------+------------+
| title          | price | pages | published  |
+----------------+-------+-------+------------+
| Animal Farm    |  8.49 |   112 | 1945-08-17 |
| Dracula        |  8.99 |   418 | 1897-05-26 |
| Foundation     |  9.99 |   255 | 1951-06-01 |
| Persuasion     |  7.49 |  NULL | 1817-12-20 |
| The Two Towers |  NULL |   352 | 1954-11-11 |
+----------------+-------+-------+------------+
+----------------+--------------+-----------+
| title          | verdict      | author_id |
+----------------+--------------+-----------+
| Animal Farm    | already sold |         2 |
| The Two Towers | no price     |         6 |
| Dracula        | ok           |      NULL |
| Foundation     | ok           |        10 |
| Persuasion     | ok           |         5 |
+----------------+--------------+-----------+
+----+------------+--------------+-------+-------+------------+
| id | title      | author       | price | pages | published  |
+----+------------+--------------+-------+-------+------------+
| 12 | Dracula    | Bram Stoker  |  8.99 |   418 | 1897-05-26 |
| 13 | Foundation | Isaac Asimov |  9.99 |   255 | 1951-06-01 |
| 14 | Persuasion | Jane Austen  |  7.49 |  NULL | 1817-12-20 |
+----+------------+--------------+-------+-------+------------+
+-------+---------+
| books | authors |
+-------+---------+
|    14 |      11 |
+-------+---------+
```

Reading it: as shipped, CAST('N/A' AS DECIMAL) is 0.00 with a warning (a free book) and the US pattern makes every date NULL with warning 1411; those warnings are the lesson's "read the status line" in action. Bram Stoker's NULL author_id is the LEFT JOIN finding a missing author; the transaction adds him first. NOT EXISTS skips Animal Farm, so the finished file can run again without duplicates (after a reload it gives the same ids). Persuasion's empty page count is a real NULL, not 0.

- LAB 8.1-B: LABS/MODULE-12/L8_1-IMPORTING-EXPORTING/BACKUP_CHECK_STARTER.SQL → BACKUP_CHECK_SOLUTION.SQL

The solution prints:

```
+------------------+-------------+
| server_side_files | local_infile |
+------------------+-------------+
| one folder only   |           1 |
+------------------+-------------+
+-----------------------------------------------------------------------------------------
| Grants for kit_backup@localhost
+-----------------------------------------------------------------------------------------
| GRANT USAGE ON *.* TO `kit_backup`@`localhost`
| GRANT SELECT, LOCK TABLES, SHOW VIEW, EVENT, TRIGGER ON `bookstore`.* TO `kit_backup`
+-----------------------------------------------------------------------------------------
+-----------------+
| write_privileges |
+-----------------+
|               0 |
+-----------------+
+-------------------+
| global_privileges |
+-------------------+
|                 0 |
+-------------------+
+-------------+-----------+
| table_name  | row_count |
+-------------+-----------+
| books       |        11 |
| customers   |         8 |
| orders      |        12 |
| order_items |        18 |
+-------------+-----------+
+---------------+
| revenue_check |
+---------------+
|        237.29 |
+---------------+
+-----------------+------------+---------+
| category        | books_sold | revenue |
+-----------------+------------+---------+
| category        | books_sold | revenue |
| Classic         | 10         | 96.90   |
| Dystopian       | 7          | 70.43   |
| Fantasy         | 5          | 64.95   |
| Fiction         | 21         | 237.29  |
| Science Fiction | 6          | 75.44   |
+-----------------+------------+---------+
```

Reading it: `one folder only` is the safe setting; local_infile shows 1 on the kit's test server and 0 on a fresh install. The backup account's five privileges are what mysqldump needs to read a database with views, triggers and events; 0 and 0 prove it cannot change anything. The counts and revenue_check (237.29) are what a restore

test compares. In the report every value is text because of the header row (the starter, without a header, prints right-aligned numbers).

HANDOUT: LABS/MODULE-12/L8_1-IMPORTING-EXPORTING/DEBUG_IMPORT.SQL (DELIBERATELY BROKEN)

As shipped:

```
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresp
ERROR 1290 (HY000): The MySQL server is running with the --secure-file-priv option so i
ERROR 1411 (HY000): Incorrect datetime value: '12/23/1815' for function str_to_date
ERROR 1292 (22007): Truncated incorrect DECIMAL value: '7,49'
Empty set
```

1. **A, 1064:** mysqldump is a terminal program. Fix: `exit`, then run it at `$` / `PS>` with the 9.7 options.
2. **B, 1290:** /tmp is not the server's file folder. Fix: the secure_file_priv folder (`SHOW VARIABLES LIKE 'secure_file_priv';`). On MariaDB's Docker image the folder is empty (anywhere) and the file IS written.
3. **C, 1411:** the dates are month/day/year, the pattern says day/month: in a strict INSERT the NULL becomes an error. Fix: `'%m/%d/%Y'`.
4. **D, 1292:** Frankenstein's price `7,49` uses a decimal comma. Fix: `REPLACE(price_raw, ',', '.')` before the CAST (or ask the supplier for a dot).

Fixed (real run):

```
+--------------+-------+------------+
| title        | price | published  |
+--------------+-------+------------+
| Emma         |  8.49 | 1815-12-23 |
| Frankenstein |  7.49 | 1818-01-01 |
+--------------+-------+------------+
```

THE SHELL PART OF THE 8.1 DEMO (REAL RUNS ON THE KIT'S 9.7 TEST SERVER)

The commands as learners type them; on the test server they ran with its socket instead of `-u root -p`. The dump and the first restore print nothing.

```
mysqldump -u root -p --single-transaction --routines --events \
    --set-gtid-purged=OFF bookstore > bookstore_backup.sql
```

```
grep -c 'INSERT INTO' bookstore_backup.sql
grep 'DROP TABLE IF EXISTS' bookstore_backup.sql
```

```
8
DROP TABLE IF EXISTS `authors`;
DROP TABLE IF EXISTS `book_categories`;
DROP TABLE IF EXISTS `books`;
DROP TABLE IF EXISTS `categories`;
DROP TABLE IF EXISTS `customers`;
DROP TABLE IF EXISTS `employees`;
DROP TABLE IF EXISTS `order_items`;
DROP TABLE IF EXISTS `orders`;
```

```
mysql -u root -p -e "CREATE DATABASE bookstore_test"
mysql -u root -p bookstore_test < bookstore_backup.sql
```

```
mysql -u root -p -t -e "SELECT
    (SELECT COUNT(*) FROM bookstore.order_items) AS items_original,
    (SELECT COUNT(*) FROM bookstore_test.order_items) AS items_copy"
```

```
+----------------+------------+
| items_original | items_copy |
+----------------+------------+
|             18 |         18 |
+----------------+------------+
```

```
mysqldump -u root -p --single-transaction bookstore > plain_dump.sql
```

```
Warning: A partial dump from a server that has GTIDs will by default include the GTIDs
```

```
mysql -u root -p -e "CREATE DATABASE bookstore_test2"
mysql -u root -p bookstore_test2 < plain_dump.sql
```

```
ERROR 3546 (HY000) at line 24: @@GLOBAL.GTID_PURGED cannot be changed: the added gtid s
```

```
mysql -u root -p < bookstore_backup.sql
```

```
ERROR 1046 (3D000) at line 22: No database selected
```

The last one is self-check Q10: a dump without --databases has no USE line, so the target must be named. The lesson says a plain dump prints "several" GTID warnings; with `--single-transaction` this run printed one.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 7.4 · USER MANAGEMENT & PERMISSIONS

1. **B** Connect, but read or change nothing: A new account only has USAGE, which means no privileges: it can connect, but every table access fails until you GRANT something.

2. **A** 'shop_reports'@'%' is a different account, and it does not exist: An account is the pair of user and host. shop_reports at % is a separate account that was never created, so it has no grants to show.

3. **C** It fails with error 1524; keep caching_sha2_password and update the app's driver: The plugin was removed in MySQL 9, so the statement fails with ERROR 1524. Use the default caching_sha2_password and update the program's MySQL driver if it cannot connect.

4. **A** The role is not active: run SET DEFAULT ROLE ALL TO 'marcus'@'localhost': Granted roles are inactive until activated. SET DEFAULT ROLE ALL TO marcus activates them at every login (or Marcus runs SET ROLE ALL in his session).

5. **B** Its own account with SELECT on the catalog tables and INSERT on the order tables: The website reads the catalog and adds orders, so it gets SELECT on those tables and INSERT on the order tables. A bug or attack through the site can do nothing more.

LESSON 8.1 · IMPORTING & EXPORTING DATA

1. **B** Take the dump again with --set-gtid-purged=OFF: MySQL 9.7 has GTIDs on by default, and the dump tries to set GTID_PURGED on a server that already has those GTIDs. --set-gtid-purged=OFF leaves that line out.

2. **C** Fails with error 1290: The server only reads and writes files inside its secure_file_priv folder, so a path in /tmp is refused with error 1290.

3. **A** (..., @d) SET published = STR_TO_DATE(@d, '%m/%d/%Y'): Reading the raw text into a user variable and converting it in SET is the general pattern for any value that needs reshaping on the way in.

4. **C** Run SHOW WARNINGS; and inspect the staging rows before using them: With LOCAL, bad values and duplicates become warnings instead of errors. SHOW WARNINGS tells you which rows and why, before the data spreads to real tables.

5. **A** It does not contain the user accounts that may use the database: Accounts and grants live in the mysql system database, so a single-database dump does not include them. Keep a separate script for users.

EXIT TICKET

1. **Website account:** least privilege. It needs to read the catalog and add orders; with no UPDATE or DELETE, a bug or an attacker using its login cannot change prices or erase history.

2. **A trustworthy backup:** (1) a dump with the right options (9.7: --single-transaction --routines --events --set-gtid-purged=OFF), (2) a test restore into another database with the counts compared and logged. A dump that

was never restored is a hope.

3. **The final project:** Rosa Delgado, Maple Street Tool Library. Rules: one open loan per tool; nothing goes out while being repaired; suspended members cannot borrow; at most three tools per member; no safety course, no chainsaw (power tools need Power Tools Basics).

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 C ("% includes localhost").** Treats the host as a filter on one account. Draw two rows of mysql.user.
- **Q3 B (1143 = table).** Has not seen column grants; show the support_desk test again.
- **Q4 B (FLUSH PRIVILEGES).** Old-tutorial habit; GRANT works at once.
- **Q7 A (mysqldump at mysql>).** The most common 8.1 slip; it is bug A of the debug handout.
- **Q10 A (--set-gtid-purged=OFF).** Mixes the two restore failures: 3546 is GTIDs, 1046 is a missing database name.
- **Q12 A (overwritten).** Expects files to behave like UPDATE; MySQL protects existing files.
- **Exit ticket 2 with only "run mysqldump".** The restore test is the half that matters; ask what restore_log.txt would say.

INSTRUCTOR KIT · ANSWER KEY

Session 13: Final Project Kickoff and Studio I

Answers and reasoning for every Session 13 item: the warm-up, the self-check, the predict-then-run items, what a passed milestone check-in looks like, the lab reference outputs and the exit ticket.

Session 13 | 12 self-check items | Confidential

WARM-UP

W1

The treasurer's account was granted the reporting role, but every SELECT gives ERROR 1142. What was forgotten? *(Lesson 7.4)*

The role was never activated: `SET DEFAULT ROLE ... TO ...` (or `SET ROLE` in the session). Lesson 7.4's silent Common Mistake 5; Milestone 6 has the line for both accounts.

W2

Someone types `mysqldump -u root -p bookstore > backup.sql` at the `mysql>` prompt. What happens? *(Lesson 8.1)*

`ERROR 1064`: mysqldump is a separate program, typed in the terminal, not at the mysql> prompt (Lesson 8.1, Common Mistake 1).

W3

A dump of one database restores perfectly on a new laptop, but the web form cannot log in. Why? *(Lesson 8.1)*

A single-database dump has no accounts and no grants: they live in the mysql system database. Keep them in a script of their own (the project's 05_security.sql).

W4

Which mysqldump option puts the stored procedures into the dump? *(Lesson 8.1)*

`--routines` (and `--events` for events; `--single-transaction` gives a consistent copy without locking). Milestone 7 uses all three, plus `--set-gtid-purged=OFF`.

W5

From Rosa's brief: name the tables you would build, and one rule that cannot be a CHECK constraint. *(the brief)*

members, categories, safety_courses, tools, loans, plus the junction member_courses. Rules that look at other rows cannot be a CHECK: one open loan per tool, at most three per member, no course no chainsaw, nothing while in repair or suspended.

8.2 · FINAL PROJECT: DESIGN AND BUILD A DATABASE: SELF-CHECK

Q1

Rosa's rule "a tool can only be out with one person at a time": where does the reference design enforce it?

A) A `CHECK` constraint on `loans`

B) `UNIQUE (tool_id)` on `loans`

C) In `checkout_tool`, inside a transaction that locks the tool's row

D) An `is_available` column on `tools`, set to 0 on check-out

Why: The rule looks at OTHER rows (is there an open loan for this tool?), so no single-row constraint can hold it. The procedure checks it inside a transaction, after locking the tool's row with SELECT ... FOR UPDATE, so two desk volunteers cannot both lend the drill.

A is wrong: A CHECK sees only the row being written; it cannot look for other open loans.

B is wrong: A tool has many loans over time (all returned but one), so a plain UNIQUE would refuse the second loan ever.

D is wrong: That stores derived data twice; a missed reset makes the flag and the loans disagree (the lesson's FAQ).

Q2

In the reference design, which fact belongs in the junction table `member_courses` and nowhere else?

A) The member's email

B) `completed_on`, the date the member finished the course

C) The course's name

D) The member's status

> **Why:** The completion date is a fact about the PAIR (this member, this course): it has no home in members or in safety_courses. That is why the junction table carries its own column.
> **A** is wrong: The email is a fact about the member alone: it lives in members, once.
> **C** is wrong: The course name is a fact about the course: safety_courses.
> **D** is wrong: The status is a fact about the member alone.

Q3

What does the last statement print?

```
CREATE TABLE riders (id INT PRIMARY KEY, name VARCHAR(20));
CREATE TABLE rides  (rider_id INT, started_on DATE);
INSERT INTO riders VALUES (1, 'Ana'), (2, 'Ben'), (3, 'Cy');
INSERT INTO rides  VALUES (1, '2026-06-03'), (1, '2026-06-09'), (2, '2026-05-28');

SELECT r.name, COUNT(x.rider_id) AS june_rides
FROM riders AS r
LEFT JOIN rides AS x ON x.rider_id = r.id
WHERE x.started_on >= '2026-06-01'
GROUP BY r.id, r.name
ORDER BY r.name;
```

One row only: `Ana` with `2` (Ben and Cy are gone)

> **Why:** WHERE runs after the join. Cy has no rides, so x.started_on is NULL and fails the test; Ben's only ride is from May. The LEFT no longer does anything: the lesson's silent Common Mistake 5. Move the date test into ON and all three riders come back (Ben 0, Cy 0).

Q4

What happens when this runs?

```
CREATE TABLE tools (id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(50));
CREATE TABLE loans (
    id      INT AUTO_INCREMENT PRIMARY KEY,
    tool_id INT UNSIGNED NOT NULL,
    CONSTRAINT fk_loans_tool FOREIGN KEY (tool_id) REFERENCES tools (id)
);
```

A) Both tables are created

B) `ERROR 3780 (HY000): Referencing column 'tool_id' and referenced column 'id' in foreign key constraint 'fk_loans_tool' are incompatible.`

C) `ERROR 1824 (HY000): Failed to open the referenced table 'tools'`

D) `ERROR 1452 (23000): Cannot add or update a child row`

> **Why:** A foreign key and the key it points to must have the same type AND signedness: INT and INT UNSIGNED clash. Pick one key type at the start (the reference uses INT everywhere) and use it for every id and every column that points at one.
> **A** is wrong: tools is created, but loans is refused: its foreign key is incompatible.
> **C** is wrong: 1824 means the parent table does not exist yet; tools exists here.
> **D** is wrong: 1452 is about a ROW whose value has no parent; no row is inserted here.

Q5

Why does the reference `01_schema.sql` begin with `DROP DATABASE IF EXISTS tool_library;`?

A) So the script can be run again from scratch and always gives the same result

B) Because MySQL cannot create a table in a database that has data in it

C) To delete the test rows the constraint tests left behind

D) So that the backup in Milestone 7 is smaller

> **Why:** A re-runnable script is the project's safety net: fix a column, run the whole file again, get a clean, identical database. The rubric's Schema row asks for exactly that ("runs twice without errors"), and Milestone 8 rebuilds everything from the files.
> **B** is wrong: MySQL creates tables in databases that hold data all the time.
> **C** is wrong: The tests run inside a transaction and ROLLBACK removes their rows; nothing is left to delete.
> **D** is wrong: The backup copies whatever is there; the DROP plays no part in it.

Q6

Which of these should NOT be a column in the tool library's tables?

A) `tools.replacement_value`

B) `loans.due_on`

C) The number of days a loan is overdue

D) `members.joined_on`

> **Why:** "Days overdue" depends on today's date and the loan's dates, so it changes every day: compute it in the query (DATEDIFF(@today, due_on)). A stored copy would be wrong tomorrow. The same goes for "available" and "times borrowed".
> **A** is wrong: A fact Rosa gave about each tool; nothing else can compute it.
> **B** is wrong: The due date is decided when the loan is made (normally a week later) and must be kept: it is the basis of "overdue".
> **D** is wrong: A fact about the member, recorded once.

Q7

What does the last statement print?

```
CREATE TABLE courses (
    id    INT PRIMARY KEY,
    seats INT NOT NULL,
    CONSTRAINT chk_courses_seats CHECK (seats > 0)
);
START TRANSACTION;
INSERT INTO courses VALUES (1, 12);
INSERT INTO courses VALUES (2, 0);
ROLLBACK;
SELECT COUNT(*) AS courses_left FROM courses;
```

`0` (in a column named `courses_left`)

> **Why:** The second INSERT is refused (3819, the CHECK on seats) and undoes only itself; the transaction stays open with course 1 in it. ROLLBACK then removes course 1 too. That is the lesson's pattern for constraint tests: run them in a transaction, roll it back, and no test row ever stays.

Q8

In which order can 01_schema.sql create these tables without an error?

A) `loans`, `tools`, `members`, `categories`

B) `categories`, `members`, `tools`, `loans`

C) `tools`, `categories`, `loans`, `members`

D) Any order: MySQL checks foreign keys only when rows are inserted

Why: Parents first: a foreign key can only point at a table that already exists (error 1824 otherwise). categories and members point at nothing; tools points at categories; loans points at tools and members, so it comes last. Drop in the reverse order.

A is wrong: loans points at tools and members, which do not exist yet: 1824.

C is wrong: tools points at categories before categories exists: 1824.

D is wrong: CREATE TABLE checks the referenced table immediately (1824).

Q9

Rosa: "Nobody should be able to delete the loan history." How does the reference solution make sure?

A) The loans foreign keys have no ON DELETE rule (refuse), and no account is granted DELETE

B) `ON DELETE CASCADE` on both foreign keys of `loans`

C) A `CHECK (deleted = 0)` constraint on `loans`

D) The desk volunteers are asked not to delete rows

Why: Two layers: a member or tool with history cannot be deleted (the foreign keys refuse), and neither tl_desk nor tl_reports holds DELETE, so the history rows themselves cannot be removed. Members are suspended instead of deleted.

B is wrong: CASCADE does the opposite: deleting a member would silently delete their loans.

C is wrong: A CHECK cannot stop a DELETE; it only tests rows being written.

D is wrong: A rule only people follow is not enforced; the brief asks the database to enforce it.

Q10

Your overdue query and your "never borrowed" query both return nothing on your test data. What is the most likely problem?

A) Both queries need ORDER BY

B) `@today` was set to the wrong year

C) MySQL caches empty results

D) The test data has no overdue loan and every member has borrowed something

> **Why:** Empty results prove nothing unless the data contains rows that SHOULD come back. Milestone 3 asks you to plant the edge cases (an overdue loan, a member who never borrowed, a tie, a tool in repair) so a broken query is caught.
> **A** is wrong: ORDER BY changes the order of rows, never whether they appear.
> **B** is wrong: Possible for the overdue query, but it would not empty the "never borrowed" one, which uses no date.
> **C** is wrong: There is no such cache: every query reads the current data.

Q11

The reference uses an ENUM for `tool_condition` but a table for categories. Give the reason in one sentence.

The library adds and lists categories (they are data); the conditions are a fixed list the code relies on

> **Why:** Both choices are defensible; the rubric rewards writing the reason down. A lookup table is right when the list grows or carries its own facts; an ENUM is fine for a short list that only changes with the application.

Q12

What happens when this runs?

```
CREATE TABLE gear (
    id        INT AUTO_INCREMENT PRIMARY KEY,
    condition VARCHAR(10)
);
```

A) `ERROR 1064 (42000): You have an error in your SQL syntax ... near 'condition VARCHAR(10)`

B) The table is created with a column named condition

C) `ERROR 3814 (HY000): ... disallowed function`

D) MySQL renames the column to condition_1

> **Why:** CONDITION is a reserved word (used in stored programs), so the parser stops there. Rename it (tool_condition). Backticks would work too, but then every query forever needs them.
> **B** is wrong: A reserved word cannot be a bare column name.
> **C** is wrong: 3814 is about functions inside a CHECK constraint; there is no CHECK here.
> **D** is wrong: MySQL never renames your columns.

8.2: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
CREATE TABLE kit_teams (
    id    INT PRIMARY KEY,
    group VARCHAR(10)
);
```

```
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that
)' at line 3
```

> **Why:** GROUP is a reserved word (GROUP BY), so the parser stops at it. Rename the column (`team_group`, `division`).

P2

Predict the output, then run it.

```
CREATE TABLE plots (
    id          INT PRIMARY KEY,
    gardener_id INT NOT NULL,
    CONSTRAINT fk_plots_gardener FOREIGN KEY (gardener_id) REFERENCES gardeners (id
);
```

```
ERROR 1824 (HY000): Failed to open the referenced table 'gardeners'
```

Why: The parent table `gardeners` does not exist yet. Create it first (parents first), or drop and recreate in the right order.

MILESTONES 1 TO 3: WHAT A PASSED CHECK-IN LOOKS LIKE

- **Milestone 1:** all eight requirements have a home. The reference mapping (lesson, hidden block): 1 to 4 = columns, NOT NULL, UNIQUE, ENUMs, foreign keys; 5 = `checkout_tool` in a transaction with a row lock; 6 = `03_queries.sql` and the view `v_open_loans`; 7 = roles `tl_desk` and `tl_reports`, no DELETE granted, loans foreign keys that refuse deleting a member or tool with history; 8 = mysqldump, a tested restore, scripts kept in version control.
- **Milestone 2:** the reference `01_schema.sql` runs twice without an error (measured: the second run prints the same six tables). Its four tests are refused with 3819 (`chk_loans_due`), 3819 (`chk_tools_value`), 1452 (`fk_tools_category`) and 1062 (`members.uq_members_email`), and `members_left` is 0 after ROLLBACK.
- **Milestone 3:** the reference counts are members 8, categories 5, safety_courses 2, member_courses 7, tools 12, loans 16. Edge cases: Victor (8) suspended; Tessa (7) never borrowed; the tile cutter (12) in repair and never lent; the hammer and the socket set tied in Hand Tools; loan 5 returned two days late; loans 13 to 16 still out.
- Scoring, the full rubric, the reference numbers for every milestone and how to grade a design that differs from the reference: `final-assessment.html`, Part 2.

LAB REFERENCE SOLUTIONS

- LAB 8.2-A: LABS/MODULE-13/L8_2-FINAL-PROJECT/GIFT_CARDS_STARTER.SQL → GIFT_CARDS_SOLUTION.SQL

As shipped, the starter prints four constraints, one CHECK clause and a TODO line:

```
+----------------+-----------------------+-----------------+
| TABLE_NAME     | CONSTRAINT_NAME       | CONSTRAINT_TYPE |
+----------------+-----------------------+-----------------+
| gift_card_uses | fk_gcu_card           | FOREIGN KEY     |
| gift_card_uses | fk_gcu_order          | FOREIGN KEY     |
| gift_cards     | chk_gift_cards_amount | CHECK           |
| gift_cards     | PRIMARY               | PRIMARY KEY     |
+----------------+-----------------------+-----------------+
+-----------------------+----------------+
| CONSTRAINT_NAME       | CHECK_CLAUSE   |
+-----------------------+----------------+
| chk_gift_cards_amount | (`amount` > 0) |
+-----------------------+----------------+
+--------------------------+
| todo                     |
+--------------------------+
| TODO 5: balance per card |
+--------------------------+
```

The solution prints:

```
+----------------+------------------------+-----------------+
| TABLE_NAME     | CONSTRAINT_NAME        | CONSTRAINT_TYPE |
+----------------+------------------------+-----------------+
| gift_card_uses | chk_gcu_amount         | CHECK           |
| gift_card_uses | fk_gcu_card            | FOREIGN KEY     |
| gift_card_uses | fk_gcu_order           | FOREIGN KEY     |
| gift_card_uses | PRIMARY                | PRIMARY KEY     |
| gift_cards     | chk_gift_cards_amount  | CHECK           |
| gift_cards     | chk_gift_cards_dates   | CHECK           |
| gift_cards     | fk_gift_cards_customer | FOREIGN KEY     |
| gift_cards     | PRIMARY                | PRIMARY KEY     |
| gift_cards     | uq_gift_cards_code     | UNIQUE          |
+----------------+------------------------+-----------------+
+-----------------------+------------------------------+
| CONSTRAINT_NAME       | CHECK_CLAUSE                 |
+-----------------------+------------------------------+
| chk_gcu_amount        | (`amount_used` > 0)          |
| chk_gift_cards_amount | (`amount` > 0)               |
| chk_gift_cards_dates  | (`expires_on` > `issued_on`) |
+-----------------------+------------------------------+
+----------+--------+-------+---------+
| code     | amount | used  | balance |
+----------+--------+-------+---------+
| GC7Q2M4K |  25.00 |  0.00 |   25.00 |
| GC3X9P1T |  50.00 | 18.98 |   31.02 |
| GC8L5V2N |  20.00 |  0.00 |   20.00 |
+----------+--------+-------+---------+
```

Reading it: nine constraints, named so every error explains itself (the starter's PRIMARY keys are the only unnamed ones: MySQL always calls them PRIMARY). `ORDER BY CONSTRAINT_TYPE` sorts the types alphabetically. The balance is derived (amount minus the SUM of uses, COALESCE for a card never used): Frank's card GC3X9P1T paid 18.98 toward his order 10. Common slips: a comma missing before an added CONSTRAINT line (1064); `CHECK (expires_on > CURDATE())` (3814); the PRIMARY KEY on gift_card_id alone (then each card could pay only ONE order ever: ask why the pair is the key).

- HANDOUT: LABS/MODULE-13/L8_2-FINAL-PROJECT/DEBUG_GIFT_CARD_PROOFS.SQL (A TEST FILE THAT MUST FAIL)

It prints:

```
ERROR 3819 (HY000): Check constraint 'chk_gift_cards_amount' is violated.
ERROR 3819 (HY000): Check constraint 'chk_gift_cards_dates' is violated.
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (
ERROR 1062 (23000): Duplicate entry 'GCTEST01' for key 'gift_cards.uq_gift_cards_code'
ERROR 1265 (01000): Data truncated for column 'status' at row 1
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (
+----------------------+
| cards_in_transaction |
+----------------------+
|                    1 |
+----------------------+
+----------------------+
| cards_after_rollback |
+----------------------+
|                    0 |
+----------------------+
```

1. **Test A, 3819** `chk_gift_cards_amount`: a card worth 0.
2. **Test B, 3819** `chk_gift_cards_dates`: expires before it is issued.
3. **Test C, 1452** `fk_gift_cards_customer`: customer 99 does not exist.
4. **Test D, 1062** `uq_gift_cards_code`: the code GCTEST01 is taken.
5. **Test E, 1265** `Data truncated for column 'status'`: 'lost' is not in the ENUM (strict mode refuses it; without strict mode it would become '' with a warning).
6. **Test F, 1452** `fk_gcu_order`: order 999 does not exist.

`cards_in_transaction` 1 (only the good row) and `cards_after_rollback` 0. With the STARTER's tables, tests B, C and D would be accepted silently: that is the point of writing the tests.

- LAB 8.2-B (HOMEWORK): LABS/MODULE-13/L8_2-FINAL-PROJECT/EDGE_CASES_STARTER.SQL → EDGE_CASES_SOLUTION.SQL

As shipped, the starter prints (query 1 is an empty result: an inner join cannot find a missing row):

```
Empty set
+--------------------------+
| todo                     |
+--------------------------+
| TODO 2: books never sold |
+--------------------------+
+---------------+---------------+
| customer      | recent_orders |
+---------------+---------------+
| Alice Johnson |             1 |
| Dave Brown    |             1 |
| Eve Davis     |             1 |
| Frank Miller  |             1 |
| Grace Wilson  |             1 |
+---------------+---------------+
+-----------------+------------+--------+
| category        | title      | orders |
+-----------------+------------+--------+
| Classic         | 1984       |      3 |
| Dystopian       | 1984       |      3 |
| Fantasy         | The Hobbit |      2 |
| Fiction         | 1984       |      3 |
| Science Fiction | Dune       |      3 |
+-----------------+------------+--------+
```

The solution prints:

```
+----+----------------+
| id | customer       |
+----+----------------+
|  3 | Carol Williams |
|  8 | Hank Taylor    |
+----+----------------+
+----+------------------------+
| id | title                  |
+----+------------------------+
|  4 | The Great Gatsby       |
|  8 | The Catcher in the Rye |
+----+------------------------+
+----------------+---------------+
| customer       | recent_orders |
+----------------+---------------+
| Alice Johnson  |             1 |
| Dave Brown     |             1 |
| Eve Davis      |             1 |
| Frank Miller   |             1 |
| Grace Wilson   |             1 |
| Bob Smith      |             0 |
| Carol Williams |             0 |
| Hank Taylor    |             0 |
+----------------+---------------+
+-----------------+--------------------+--------+
| category        | title              | orders |
+-----------------+--------------------+--------+
| Classic         | 1984               |      3 |
| Dystopian       | 1984               |      3 |
| Fantasy         | Lord of the Rings  |      2 |
| Fantasy         | The Hobbit         |      2 |
| Fiction         | 1984               |      3 |
| Fiction         | Dune               |      3 |
| Science Fiction | Dune               |      3 |
+-----------------+--------------------+--------+
```

What each TODO teaches, with the Milestone 4 query it prepares: (1) LEFT JOIN ... IS NULL for "never" (Saturday question 5: Tessa and Victor); (2) the same for books (Gatsby and Catcher were never sold: both are out of stock); (3) a condition on the right-hand table belongs in ON, or the zero rows vanish (5 rows instead of 8, the lesson's Mistake 5); (4) RANK keeps ties, ROW_NUMBER silently drops one (Saturday question 3: the hammer and the socket set). Query 4 counts ORDERS that include a book, not copies: by copies there is no tie in the bookstore data.

HANDOUT: LABS/MODULE-13/L8_2-FINAL-PROJECT/DEBUG_BUILD_ORDER.SQL (DELIBERATELY BROKEN)

As shipped:

```
Database changed
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresp
)' at line 4
ERROR 3780 (HY000): Referencing column 'rider_id' and referenced column 'id' in foreign
ERROR 3814 (HY000): An expression of a check constraint 'chk_plans_start' contains disa
ERROR 1824 (HY000): Failed to open the referenced table 'parts'
+-------------------------+
| Tables_in_kit_s13_bikes |
+-------------------------+
| parts                   |
| riders                  |
+-------------------------+
```

1. **Bug A, 1064:** `range` is a reserved word. Fix: `range_km`.
2. **Bug B, 3780:** `rider_id BIGINT` against `riders.id INT`. Fix: `rider_id INT`.
3. **Bug C, 3814:** `CURDATE()` in a CHECK. Fix: drop the CHECK (a time rule goes in the code that inserts plans) or compare two columns, e.g. an `ends_on` with `CHECK (ends_on > starts_on)`.
4. **Bug D, 1824:** `repair_parts` before `parts`. Fix: move `CREATE TABLE parts` above it.

Fixed: no ERROR lines and SHOW TABLES lists six tables (bikes, parts, price_plans, repair_parts, rides, riders). As shipped only parts and riders exist.

EXIT TICKET

1. **A rule for a procedure:** one open loan per tool, at most three per member, the chainsaw course, not while in repair, not while suspended. Each needs other rows (open loans, the member's status, member_courses) and a CHECK sees only the row being written.
2. **Re-runnable:** the first line `DROP DATABASE IF EXISTS tool_library;` then `CREATE DATABASE` and `USE`, and every table created again (or DROP TABLE IF EXISTS, children first).
3. **A test:** `INSERT INTO loans (tool_id, member_id, loaned_on, due_on) VALUES (1, 1, '2026-06-10', '2026-06-03');` must give `ERROR 3819 (HY000): Check constraint 'chk_loans_due' is violated.` (or the learner's own constraint name). Inside a transaction, then ROLLBACK.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 B (UNIQUE (tool_id)).** Sees the rule as a property of the table, not of OPEN loans. Ask how many loans the drill has had (four). The lesson's Exercise 4 shows the clever version (a generated column), after Milestone 5.
- **Q3 "three rows".** Expects LEFT JOIN to protect the rows whatever WHERE does: the lesson's silent mistake. Send them to `edge_cases_starter.sql` TODO 3.

- **Q6 B (due_on).** Confuses "decided once" with "computed": the due date is a fact recorded at check-out; overdue is computed from it.
- **Q8 D (any order).** Thinks foreign keys are checked only on INSERT. Run Mistake 4 again: 1824 at CREATE time.
- **Q9 B (CASCADE).** Reads ON DELETE CASCADE as "protects". It deletes the history: the opposite of Rosa's requirement 7.
- **Q10 A or B.** Looks for a syntax cause for an empty result. The data has to contain what the query should find.

INSTRUCTOR KIT · ANSWER KEY

Session 14: Final Project Studio II, Showcase and Next Steps

Answers and reasoning for every Session 14 item: the warm-up, both self-checks, the predict-then-run items, what a passed milestone check-in looks like, the lab reference outputs, both lesson pages' own quizzes and the exit ticket.

Session 14 | 12 self-check items | Confidential

WARM-UP: THE REBUILD CHECK

W1

From a client started in your tool-library folder: `SOURCE 02_data.sql`, then `SOURCE 03_queries.sql`. Which line fails, if any? *(Run your scripts)*

Nothing should fail. 02_data.sql rebuilds the schema and the data; the counts for the reference data are members 8, categories 5, safety_courses 2, member_courses 7, tools 12, loans 16. A failure here is the first thing to fix today (the rubric's Schema row and Milestone 8 depend on it).

W2

Saturday question 3 uses RANK(). What would ROW_NUMBER() lose? *(Milestone 4)*

A tie: with the reference data the hammer and the socket set share first place in Hand Tools (one loan each); ROW_NUMBER keeps only one of them, silently.

W3

Why does every Saturday query read `@today` instead of `CURDATE()`? *(Milestone 4)*

So the results are repeatable and can be checked against known answers (and against the reference: `@today = '2026-06-15'`). The real report sets `@today = CURDATE()` once.

8.2 · FINAL PROJECT: DESIGN AND BUILD A DATABASE: SELF-CHECK

Q1

On the reference tool_library (Milestone 3 data), member 7 is Tessa (active, no safety course) and tool 6 is the chainsaw, on the shelf. What does this print?

```
CALL checkout_tool(7, 6, '2026-06-15');
```

A) A new loan: `loan_id 17, due_on 2026-06-22`

B) `ERROR 1644 (45000): Member is suspended`

C) `ERROR 1644 (45000): Safety course required for this tool`

D) `ERROR 1142 (42000): INSERT command denied`

> **Why:** Tessa is active and the chainsaw is free, so the first checks pass; the course check finds no member_courses row for Chainsaw Safety and SIGNALs. The EXIT HANDLER rolls back and RESIGNALs, so the caller sees the procedure's own message with error 1644 (a user-defined SQLSTATE 45000).
> **A** is wrong: The course rule refuses it before the INSERT.
> **B** is wrong: Tessa is active; that message belongs to Victor (member 8).
> **D** is wrong: root runs it here; and a desk account calling the procedure gets the definer's rights anyway.

Q2

Why does `checkout_tool` declare `EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END`?

A) So a failed check-out never leaves a half-finished transaction, and the caller still receives the real error message

B) So that errors are ignored and the loan is created anyway

C) Because SIGNAL only works inside a handler

D) To make the procedure run with the definer's rights

> **Why:** Any error after START TRANSACTION (a SIGNAL from a rule, or a constraint) jumps to the handler: ROLLBACK undoes the transaction, RESIGNAL passes the original error on, so the desk's web form can show "Tool is already on loan".
> **B** is wrong: EXIT ends the procedure; nothing after the error runs.
> **C** is wrong: SIGNAL works anywhere in a stored program.
> **D** is wrong: Definer's rights come from SQL SECURITY DEFINER (the default), not from a handler.

Q3

Logged in as `tl_desk_app` (the reference roles from Milestone 6), what does this print?

```
DELETE FROM loans WHERE id = 1;
```

```
ERROR 1142 (42000): DELETE command denied to user 'tl_desk_app'@'localhost' for table 'loans'
```

> **Why:** tl_desk has SELECT on everything, INSERT and UPDATE on members and EXECUTE on the two procedures, and nothing else: no DELETE anywhere, so the loan history stays (Rosa's requirement 7). Measured by logging in as the account; INSERT INTO loans is refused the same way, while CALL checkout_tool works.

Q4

Milestone 7: your dump of tool_library restored into tool_library_test with matching counts. What is NOT in that dump?

A) The tables and their rows

B) The view v_open_loans

C) The two procedures (taken with --routines)

D) The accounts tl_desk_app and tl_treasurer and their roles

> **Why:** A single-database dump holds tables, rows, views and (with --routines) procedures, but no accounts and no grants: they live in the mysql system database. 05_security.sql is their backup (measured: the reference dump contains no CREATE USER and no GRANT).
> **A** is wrong: Tables and rows are the core of every dump.
> **B** is wrong: Views are always dumped with their database.
> **C** is wrong: --routines puts them in; the restored copy lists both.

Q5

Why does the reference `checkout_tool` read the tool with `SELECT ... FOR UPDATE`?

A) To make the SELECT faster

B) So a second check-out of the same tool waits until the first commits, then sees the new loan

C) Because SELECT INTO needs FOR UPDATE inside a procedure

D) To update the tool's condition

> **Why:** Without the lock, two desk volunteers could both see "drill available" and both lend it. FOR UPDATE locks the tool's row inside the transaction; the second call waits, then its "already on loan" check finds the first loan.
> **A** is wrong: A lock can only make another session wait; it never speeds anything up.
> **C** is wrong: SELECT ... INTO works without it.
> **D** is wrong: FOR UPDATE changes nothing by itself; it only locks the rows read.

Q6

On the Milestone 3 data, Rosa runs the overdue query (Saturday question 2) with a later report date. How many rows come back, and which loan is first?

```
SET @today = '2026-06-20';
-- then the lesson's query 2: open loans with due_on < @today, worst first
```

Four rows; loan 13 (the extension ladder, `12` days) first

> **Why:** On June 20 all four open loans (13 to 16) are past due: 12, 7, 3 and 1 days. On June 15 only two were. A report that reads @today can be checked against known answers for any date; one that reads CURDATE() changes every day.

8.2: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
CREATE TABLE tool_loans (tool VARCHAR(20), loans INT);
INSERT INTO tool_loans VALUES ('Hammer', 1), ('Socket set', 1), ('Tile cutter', 0);

SELECT tool, loans,
       RANK()       OVER (ORDER BY loans DESC) AS rnk,
       ROW_NUMBER() OVER (ORDER BY loans DESC, tool) AS row_num
FROM tool_loans
ORDER BY loans DESC, tool;
```

```
+-------------+-------+-----+---------+
| tool        | loans | rnk | row_num |
+-------------+-------+-----+---------+
| Hammer      |     1 |   1 |       1 |
| Socket set  |     1 |   1 |       2 |
| Tile cutter |     0 |   3 |       3 |
+-------------+-------+-----+---------+
```

Why: RANK gives equal values the same rank (both 1, then 3); ROW_NUMBER numbers every row differently, so a WHERE row_num = 1 would keep only the Hammer (it sorts first by name). That is why Saturday question 3 uses RANK.

8.3 · NEXT STEPS & BEST PRACTICES: SELF-CHECK

Q7

What does this print on the bookstore?

```
SET @input = 'Chicago'' OR ''x''=''x';
SET @sql = CONCAT('SELECT first_name FROM customers WHERE city = ''', @input, ''' O
PREPARE search FROM @sql;
EXECUTE search;
```

A) `Empty set`

B) Two rows: Bob and Grace

C) `ERROR 1064 (42000): You have an error in your SQL syntax`

D) All eight customers

> **Why:** The quote inside @input closes the string early and OR 'x'='x' becomes part of the SQL: true for every row. That is SQL injection, with no error at all. A placeholder (city = ?) with EXECUTE ... USING @input would print Empty set.
> **A** is wrong: That is what the safe, parameterized version prints.
> **B** is wrong: Only an honest 'Chicago' would give those two.
> **C** is wrong: The quotes are doubled correctly, so the built statement is valid SQL; that is what makes it dangerous.

Q8

Health check 3 lists the bookstore's foreign keys. What happens to an order's rows in order_items when the order is deleted?

A) They are deleted too (fk_items_order is CASCADE)

B) The DELETE is refused (RESTRICT)

C) Their order_id becomes NULL (SET NULL)

D) Nothing: they stay and point at a missing order

> **Why:** The check's output shows order_items · fk_items_order · orders · CASCADE. Deleting a customer with orders, by contrast, is refused (fk_orders_customer RESTRICT), and deleting a manager sets the staff's manager_id to NULL.
> **B** is wrong: RESTRICT is the rule on fk_orders_customer and fk_items_book, not on the order link.
> **C** is wrong: SET NULL is fk_employees_manager; order_items.order_id is NOT NULL anyway.
> **D** is wrong: A foreign key never allows a row that points at nothing.

Q9

What does the DELETE do?

```
SET SQL_SAFE_UPDATES = 1;
DELETE FROM customers WHERE city = 'Houston';
```

It is refused: `ERROR 1175` (safe update mode: no KEY column in the WHERE)

> **Why:** city is not indexed, so safe-update mode treats the statement as a probably-forgotten WHERE and refuses it. Fix: name the rows by key (WHERE id = 4), or preview with SELECT first. Switch it back with SET SQL_SAFE_UPDATES = 0.

Q10

Why does the "tables without a primary key" check use LEFT JOIN ... WHERE c.CONSTRAINT_NAME IS NULL?

A) Because information_schema does not support inner joins

B) To sort the tables by name

C) It is the "find what is missing" pattern: keep every table, then keep only those that found no PRIMARY KEY row

D) To skip views

> **Why:** The same pattern as "members who never borrowed" (Lesson 4.3), pointed at the catalog. The condition CONSTRAINT_TYPE = 'PRIMARY KEY' sits in ON for the reason of the final project's Mistake 5; views are skipped by TABLE_TYPE = 'BASE TABLE'.
>
> **A** is wrong: information_schema is queried like any tables; inner joins work.
>
> **B** is wrong: ORDER BY sorts; the join decides which rows exist.
>
> **D** is wrong: TABLE_TYPE = 'BASE TABLE' skips views.

Q11

Where should a web application's database password live?

A) In the source code, next to the connection string

B) In an environment variable or a secrets manager, never in the code or the repository

C) In a comment in 05_security.sql, so the team can find it

D) Nowhere: use root without a password on the web server

> **Why:** Lesson 8.3's security practice: secrets outside the code (environment variables, a secrets manager, or an option file only the user can read), and commit placeholder passwords only. The drivers' examples read getenv('DB_PASSWORD').
> **A** is wrong: Code is copied, shared and pushed to repositories; the password goes with it.
> **C** is wrong: A shared script is exactly what must not hold real passwords (the final project's FAQ).
> **D** is wrong: root is for administration only, never for an application.

Q12

A statement has two placeholders, `WHERE a.name = ? AND b.price < ?`. How must the values be given to EXECUTE?

`EXECUTE stmt USING @author, @max_price;` : user variables, in the same order as the ? placeholders

> **Why:** EXECUTE ... USING takes user variables (not literals) and fills the placeholders left to right. The lesson's Exercise 1 does exactly this; a driver does the same with a list of values.

8.3: PREDICT, THEN RUN

P2

Predict the output, then run it.

```
SET @who = 'x'' OR ''1''=''1';
PREPARE find FROM 'SELECT first_name FROM customers WHERE last_name = ?';
EXECUTE find USING @who;
DEALLOCATE PREPARE find;
```

```
Statement prepared
Empty set
```

Why: The crafted text is only a value: no customer has that last name, so the result is empty. Glued into the SQL, the same text would have returned every customer.

MILESTONES 5 TO 7: WHAT A PASSED CHECK-IN LOOKS LIKE

- **Milestone 5:** five refusals with their own messages, error 1644 each (the reference: Member is suspended · Tool is out for repair · Tool is already on loan · Safety course required for this tool · Loan limit reached (3 open loans)); a success that prints the new loan (reference: loans 17 and 18, due 2026-06-22); `return_tool` refuses a tool that is not out (Tool is not on loan); `v_open_loans` after the reference calls lists loans 14, 15 and 16.
- **Milestone 6:** SHOW GRANTS for the desk account shows SELECT on the database, INSERT and UPDATE on members, EXECUTE on the two procedures and the role; logged in as it, the CALL works and `DELETE FROM loans` / `INSERT INTO loans` give 1142; the treasurer reads and gets 1142 on UPDATE (all measured on the reference).
- **Milestone 7:** the two counts match (18 and 18 on the reference after Milestone 5); both procedures in the copy; the README has the date and the counts. The accounts are not in the dump (measured: no CREATE USER, no GRANT).
- Scoring with the rubric and the full reference numbers: `final-assessment.html`, Part 2.

LAB REFERENCE SOLUTIONS

- LAB 8.2-A: LABS/MODULE-14/L8_2-FINAL-PROJECT/ADD_ITEM_RULES_STARTER.SQL → ADD_ITEM_RULES_SOLUTION.SQL

Both print the same for the two allowed calls (the difference shows in the failing calls):

```
+---------+---------------+
| item_id | price_at_time |
+---------+---------------+
|      19 |          9.99 |
+---------+---------------+
+---------+---------------+
| item_id | price_at_time |
+---------+---------------+
|      20 |         10.99 |
+---------+---------------+
+----+------------+----------+---------------+
| id | title      | quantity | price_at_time |
+----+------------+----------+---------------+
|  4 | Dune       |        1 |         14.99 |
| 19 | 1984       |        1 |          9.99 |
| 20 | The Hobbit |        2 |         10.99 |
+----+------------+----------+---------------+
+----------------+--------------+---------------+
| ROUTINE_NAME   | ROUTINE_TYPE | SECURITY_TYPE |
+----------------+--------------+---------------+
| add_order_item | PROCEDURE    | DEFINER       |
+----------------+--------------+---------------+
```

What the TODOs add: (1) the quantity rule as a named CHECK, because it looks at one row only; (2) the EXIT HANDLER (ROLLBACK, RESIGNAL) and FOR UPDATE on the order's row; (3) No such book / Book is out of stock; (4) Book is already on this order; (5) at most 3 different books per order. The price is copied into price_at_time on purpose (the price at the time of sale). Order 3 belongs to Bob and had one line (Dune); the cleanup removes the two new lines, the CHECK and the procedure.

- HANDOUT: LABS/MODULE-14/L8_2-FINAL-PROJECT/DEBUG_ADD_ITEM_RULES.SQL (EVERY RULE MUST FAIL)

It prints:

```
+---------+---------------+
| item_id | price_at_time |
+---------+---------------+
|      19 |          9.99 |
+---------+---------------+
+---------+---------------+
| item_id | price_at_time |
+---------+---------------+
|      20 |         10.99 |
+---------+---------------+
ERROR 1644 (45000): No such order
ERROR 1644 (45000): No such book
ERROR 1644 (45000): Book is out of stock
ERROR 1644 (45000): Book is already on this order
ERROR 1644 (45000): An order holds at most 3 different books
ERROR 3819 (HY000): Check constraint 'chk_items_quantity' is violated.
+-----------------+
| order_items_now |
+-----------------+
|              20 |
+-----------------+
```

Order 99 does not exist; book 77 does not exist; The Great Gatsby (4) is out of stock; Pride and Prejudice (5) is already on order 2; order 3 already holds three books; zero copies breaks the CHECK, which fires inside the procedure and is passed on by RESIGNAL as 3819. `order_items_now` is 20: 18 in the course data plus the two allowed calls; none of the refused calls left a row. With the STARTER's procedure, the Gatsby, Pride and Prejudice and fourth-book calls would all succeed.

- LAB 8.2-B: LABS/MODULE-14/L8_2-FINAL-PROJECT/LEAST_PRIVILEGE_STARTER.SQL → LEAST_PRIVILEGE_SOLUTION.SQL

As shipped, the audit shows the damage (ALL PRIVILEGES expands to 18 privileges, and both roles can DELETE):

```
+---------------------------------------------------------------------+
| Grants for kit_till_app@localhost                                   |
+---------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `kit_till_app`@`localhost`                    |
| GRANT ALL PRIVILEGES ON `bookstore`.* TO `kit_till_app`@`localhost` |
| GRANT `kit_shop_till`@`%` TO `kit_till_app`@`localhost`             |
+---------------------------------------------------------------------+
+----------------------+----------------+--------------------------+
| GRANTEE              | on_what        | PRIVILEGE_TYPE           |
+----------------------+----------------+--------------------------+
| 'kit_shop_owner'@'%' | whole database | DELETE                   |
| 'kit_shop_owner'@'%' | whole database | INSERT                   |
| 'kit_shop_owner'@'%' | whole database | SELECT                   |
| 'kit_shop_owner'@'%' | whole database | UPDATE                   |
| 'kit_shop_till'@'%'  | whole database | ALTER                    |
| 'kit_shop_till'@'%'  | whole database | ALTER ROUTINE            |
| 'kit_shop_till'@'%'  | whole database | CREATE                   |
| 'kit_shop_till'@'%'  | whole database | CREATE ROUTINE           |
| 'kit_shop_till'@'%'  | whole database | CREATE TEMPORARY TABLES  |
| 'kit_shop_till'@'%'  | whole database | CREATE VIEW              |
| 'kit_shop_till'@'%'  | whole database | DELETE                   |
| 'kit_shop_till'@'%'  | whole database | DROP                     |
| 'kit_shop_till'@'%'  | whole database | EVENT                    |
| 'kit_shop_till'@'%'  | whole database | EXECUTE                  |
| 'kit_shop_till'@'%'  | whole database | INDEX                    |
| 'kit_shop_till'@'%'  | whole database | INSERT                   |
| 'kit_shop_till'@'%'  | whole database | LOCK TABLES              |
| 'kit_shop_till'@'%'  | whole database | REFERENCES               |
| 'kit_shop_till'@'%'  | whole database | SELECT                   |
| 'kit_shop_till'@'%'  | whole database | SHOW VIEW                |
| 'kit_shop_till'@'%'  | whole database | TRIGGER                  |
| 'kit_shop_till'@'%'  | whole database | UPDATE                   |
+----------------------+----------------+--------------------------+
+-------------------+
| delete_privileges |
+-------------------+
|                 2 |
+-------------------+
```

The solution prints:

```
+-----------------------------------------------------------------------------------------+
| Grants for kit_till_app@localhost                                                       |
+-----------------------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `kit_till_app`@`localhost`                                        |
| GRANT SELECT ON `bookstore`.* TO `kit_till_app`@`localhost`                             |
| GRANT INSERT, UPDATE ON `bookstore`.`customers` TO `kit_till_app`@`localhost`           |
| GRANT EXECUTE ON PROCEDURE `bookstore`.`kit_restock` TO `kit_till_app`@`localhost`      |
| GRANT `kit_shop_till`@`%` TO `kit_till_app`@`localhost`                                 |
+-----------------------------------------------------------------------------------------+
+----------------------+----------------+----------------+
| GRANTEE              | on_what        | PRIVILEGE_TYPE |
+----------------------+----------------+----------------+
| 'kit_shop_owner'@'%' | whole database | SELECT         |
| 'kit_shop_till'@'%'  | customers      | INSERT         |
| 'kit_shop_till'@'%'  | customers      | UPDATE         |
| 'kit_shop_till'@'%'  | whole database | SELECT         |
+----------------------+----------------+----------------+
+-------------------+
| delete_privileges |
+-------------------+
|                 0 |
+-------------------+
```

Reading it: the till's account sees its role's grants through `USING`; EXECUTE on the procedure appears in SHOW GRANTS but not in SCHEMA_PRIVILEGES or TABLE_PRIVILEGES (procedure privileges live in mysql.procs_priv). TODO 3 (SET DEFAULT ROLE) changes nothing in this output: it only matters when the account logs in, which a SOURCEd lab cannot show (the Lesson 7.4 project tests it by logging in). Everything is dropped at the end.

- LAB 8.3-A: LABS/MODULE-14/L8_3-NEXT-STEPS/HEALTH_CHECK_STARTER.SQL → HEALTH_CHECK_SOLUTION.SQL

The solution prints (on the bookstore with the planted table):

```
+--------------------+
| table_without_pk   |
+--------------------+
| kit_legacy_refunds |
+--------------------+
+--------------------+-------------+--------------+--------------------+
| TABLE_NAME         | COLUMN_NAME | COLUMN_TYPE  | CHARACTER_SET_NAME |
+--------------------+-------------+--------------+--------------------+
| kit_legacy_refunds | amount      | float        | NULL               |
| kit_legacy_refunds | reason      | varchar(100) | latin1             |
+--------------------+-------------+--------------+--------------------+
+--------------------+-------------+
| TABLE_NAME         | COLUMN_NAME |
+--------------------+-------------+
| kit_legacy_refunds | order_id    |
+--------------------+-------------+
+-----------------+----------------------+-----------------------+-------------+
| TABLE_NAME      | CONSTRAINT_NAME      | REFERENCED_TABLE_NAME | DELETE_RULE |
+-----------------+----------------------+-----------------------+-------------+
| book_categories | fk_bc_book           | books                 | CASCADE     |
| book_categories | fk_bc_category       | categories            | CASCADE     |
| employees       | fk_employees_manager | employees             | SET NULL    |
| order_items     | fk_items_order       | orders                | CASCADE     |
+-----------------+----------------------+-----------------------+-------------+
```

Check 3 is new: a column named like a key that no foreign key protects (the legacy table's order_id could point at an order that does not exist). Check 4 lists only the delete rules that do more than refuse. Pointed at the reference tool_library, checks 1 to 3 print nothing and check 4 lists fk_mc_member CASCADE only. As shipped, check 4 lists all seven foreign keys (three RESTRICT).

- LAB 8.3-B: LABS/MODULE-14/L8_3-NEXT-STEPS/SAFE_SEARCH_STARTER.SQL → SAFE_SEARCH_SOLUTION.SQL

The solution prints:

```
+-----------------------------------------------------------------------------------------
| query_sent
+-----------------------------------------------------------------------------------------
| SELECT first_name, last_name, city FROM customers WHERE city = 'Chicago' OR 'x'='x' (
+-----------------------------------------------------------------------------------------
Statement prepared
+------------+-----------+-------------+
| first_name | last_name | city        |
+------------+-----------+-------------+
| Alice      | Johnson   | New York    |
| Bob        | Smith     | Chicago     |
| Carol      | Williams  | Los Angeles |
| Dave       | Brown     | Houston     |
| Eve        | Davis     | NULL        |
| Frank      | Miller    | New York    |
| Grace      | Wilson    | Chicago     |
| Hank       | Taylor    | NULL        |
+------------+-----------+-------------+
Statement prepared
Empty set
+------------+-----------+---------+
| first_name | last_name | city    |
+------------+-----------+---------+
| Bob        | Smith     | Chicago |
| Grace      | Wilson    | Chicago |
+------------+-----------+---------+
Statement prepared
+----+------------+
| id | order_date |
+----+------------+
|  2 | 2026-02-20 |
|  8 | 2026-04-05 |
+----+------------+
Empty set
```

As shipped, part 1 leaks all eight customers and the two TODO lines print placeholders. The second EXECUTE of orders_between with the crafted email prints Empty set (interactive client); Alice's orders between February and June are 2 and 8.

HANDOUT: LABS/MODULE-14/L8_3-NEXT-STEPS/DEBUG_UNSAFE_HABITS.SQL (DELIBERATELY BROKEN)

It prints:

```
ERROR 1175 (HY000): You are using safe update mode and you tried to update a table witl
ERROR 1175 (HY000): You are using safe update mode and you tried to update a table witl
+----+-------+-------+
| id | title | price |
+----+-------+-------+
|  1 | Dune  | 14.99 |
+----+-------+-------+
+----+------------+---------+
| id | first_name | city    |
+----+------------+---------+
|  2 | Bob        | Chicago |
+----+------------+---------+
+--------------+
| gatsby_lines |
+--------------+
|            0 |
+--------------+
```

1. **A, 1175:** no WHERE at all. Fix: `WHERE id = 1`.
2. **B, 1175:** first_name is not a key. Fix: `WHERE id = 2`.
3. **C gets through:** book_id leads an index (the foreign key's), so safe-update mode accepts it; it deletes nothing because The Great Gatsby was never sold.

After the fixes, inside the transaction Dune costs 16.49 and Bob lives in Boston; ROLLBACK restores both.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 8.2 · FINAL PROJECT: DESIGN AND BUILD A DATABASE

1. **B** In the procedure that creates loans, inside a transaction: The rule depends on other rows, so it belongs in the code that creates loans: a procedure that counts open loans inside a transaction before inserting.
2. **A** A junction table with member_id, course_id and completed_on: A many-to-many relationship needs a junction table, and the completion date is a fact about the pair, so it lives there too.
3. **C** CURDATE() changes over time, so it is not allowed in a CHECK: A constraint must give the same answer for a row at any time. CURDATE() changes every day, so MySQL disallows it in CHECK.
4. **A** So the results are repeatable and can be checked against known answers: With a fixed report date the results can be checked by hand and compared over time; in production the variable is simply set to CURDATE().
5. **C** The loan is created, because the procedure runs with its definer's rights: With SQL SECURITY DEFINER (the default) the procedure uses its definer's rights, so it can insert the loan, while the account itself still cannot touch loans directly.

1. **B** The value is sent separately from the SQL text, so it can never become part of the statement: The SQL text and the values travel separately, so the server parses the statement before it sees the input, and the input can only ever be a value.
2. **C** It fails with error 1524; update the app's driver instead: MySQL 9.x no longer has the mysql_native_password plugin (error 1524). Maintained drivers support caching_sha2_password, so the fix is a current driver.
3. **A** REFERENTIAL_CONSTRAINTS: REFERENTIAL_CONSTRAINTS has one row per foreign key, with its UPDATE_RULE and DELETE_RULE.
4. **B** Change the column to DECIMAL(8, 2): DECIMAL stores exact decimal digits, so sums of prices are exact to the cent.
5. **C** The current Long-Term Support release, such as 9.7 or 8.4: LTS releases receive bug and security fixes for years without changing behavior, which is what production systems need.

EXIT TICKET

1. **Definer's rights:** the procedure runs with the privileges of its definer (SQL SECURITY DEFINER is the default); the desk account only needs EXECUTE, and every loan passes the procedure's rules.
2. **Safely:** `PREPARE s FROM 'SELECT ... WHERE email = ?'; EXECUTE s USING @input; DEALLOCATE PREPARE s;` or, in a driver, a placeholder and the value passed separately.
3. **Next step:** any path from the table with a start date; check that the plan names one first topic and one small project.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 A (a loan is created).** Expects the procedure to check only availability. Ask which of the five rules reads member_courses.
- **Q2 C or D.** Mixes up SIGNAL, handlers and definer's rights: three separate tools. Draw the five-box shape again.
- **Q4 C (the procedures are missing).** True only without `--routines`; the lesson's command has it.
- **Q5 A ("faster").** Thinks of locks as optimizations. Show the two-terminal race once.
- **Q7 A (Empty set).** Has seen the safe version and assumes the database protects them. The leak is silent: that is the danger.
- **Q8 B (RESTRICT).** Generalizes from customers to orders. Read health check 3's output line by line.